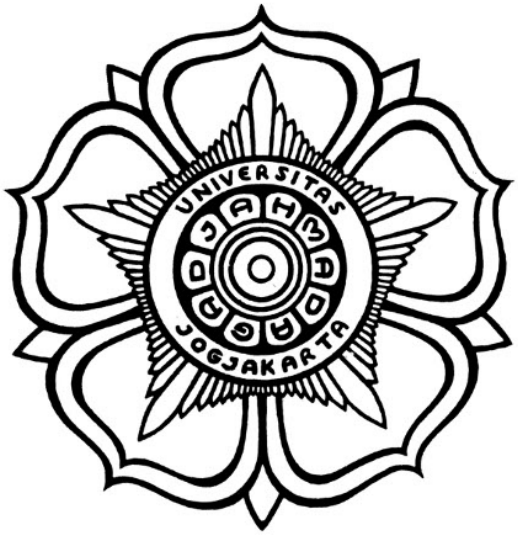
PERANCANGAN IP I2S TRANSMITTER AXI4-LITE PADA SOC MICROBLAZE V MENGGUNAKAN VERILOG

SKRIPSI



THE SUSTAINABLE DEVELOPMENT GOALS
Industry, Innovation and Infrastructure
Affordable and Clean Energy
Climate Action

Disusun oleh:

Muhammad Muqtada Alhaddad
22/500341/TK/54841

PROGRAM SARJANA PROGRAM STUDI TEKNOLOGI INFORMASI
DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

HALAMAN PENGESAHAN

PERANCANGAN IP I2S TRANSMITTER AXI4-LITE PADA SOC MICROBLAZE V MENGGUNAKAN VERILOG

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
pada Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

Muhammad Muqtada Alhaddad
22/500341/TK/54841

Telah disetujui dan disahkan

Pada tanggal 20 Juli 2026

Dosen Pembimbing I	Dosen Pembimbing II
Ir. Agus Bejo, S.T., M.Eng., D.Eng., IPM.	**Ir. Wahyu Dewanto, M.T.**
198001012015041002	**196111091986011001**

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

Nama	: Muhammad Muqtada Alhaddad
NIM	: 22/500341/TK/54841
Tahun terdaftar	: 2022
Program	: Sarjana
Program Studi	: Teknologi Informasi
Fakultas	: Teknik Universitas Gadjah Mada

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, 20 Juli 2026

Materai Rp10.000

(Tanda tangan)

Muhammad Muqtada Alhaddad
22/500341/TK/54841

HALAMAN PERSEMBAHAN

"Dan tidaklah kamu diberi ilmu melainkan hanya sedikit" (QS. Al-Isra: 85)

Skripsi ini penulis persembahkan kepada:

Orang tua tercinta atas doa, dukungan, dan pengorbanan tanpa henti.

Para dosen pembimbing yang telah membimbing perjalanan akademik penulis.

Semua pihak yang telah berkontribusi dalam perjalanan penyusunan skripsi ini.

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT atas rahmat, karunia, dan petunjuk-Nya sehingga penulis dapat menyelesaikan skripsi yang berjudul **"Perancangan IP I2S Transmitter AXI4-Lite pada SoC MicroBlaze V menggunakan Verilog"**. Shalawat serta salam semoga senantiasa tercurah kepada Rasulullah SAW, keluarga, sahabat, dan para pengikut beliau hingga akhir zaman.

Penyusunan skripsi ini tidak terlepas dari bantuan, bimbingan, dukungan, dan doa dari banyak pihak. Oleh karena itu, penulis menyampaikan terima kasih kepada:

1. Kedua orang tua dan keluarga besar penulis yang selalu memberikan doa, dukungan, perhatian, dan kasih sayang selama penulis menempuh masa studi.
2. Bapak Ir. Agus Bejo, S.T., M.Eng., D.Eng., IPM. selaku dosen pembimbing pertama yang mengenalkan penulis pada bidang IC design saat perkuliahan Sistem Berbasis Mikrokontroler, serta Bapak Ir. Wahyu Dewanto, M.T. selaku dosen pembimbing kedua yang mengajarkan dasar komunikasi data kepada penulis pada semester tiga sehingga menumbuhkan ketertarikan pada cara kerja komputer. Keduanya telah meluangkan waktu untuk memberikan arahan, masukan, dan bimbingan selama proses penelitian dan penyusunan skripsi ini. Serta Bapak Dani Adhipta, S.Si., M.T. dosen pembimbing akademik yang telah memberikan berbagai dukungan dan arahan sehingga penulis dapat menempuh studi dengan baik.
3. Segenap dosen dan staf Departemen Teknik Elektro dan Teknologi Informasi, Fakultas Teknik, Universitas Gadjah Mada, yang telah memberikan ilmu, pengalaman, serta bantuan administratif selama masa perkuliahan.
4. Teman-teman Teknologi Informasi angkatan 2022 dan teman-teman UGM lintas angkatan yang telah menjadi rekan belajar, berdiskusi, dan menjalani berbagai dinamika perkuliahan.
5. Om Asep yang telah memberikan dukungan, perhatian, dan menjadi wali selama penulis tinggal di Yogyakarta. Beserta Pak De, Bu De, Mas Rizky Bagaskara, Mba Lia serta teman-teman warga sumberan yang masih mengingat walau sudah lama tidak mengunjungi sejak kelas 2 SD .
6. Teman-teman Laboratorium IC, yaitu Ariq, Aji, Willi, Adit, Muhajir, Rakhel, Jonatan, Alphons, Mas Alfito, Mba Arifa, Ciel, Egi, Alex, Luqman, Rafinda serta Pak Sudiro, Mas Byan, Mas Cahyo, dan Mas Abid, atas diskusi, masukan, dan kebersamaan selama proses penelitian.
7. Mas Hilmi, Ustadz Nabyl, Ustadz Habibi, Ustadz Arif, Ustadz Minan, Ustadz Nurkholis, Ustadz Fawaid, serta teman-teman santri Pesantren Attaqwa Alawiyah yang telah memberikan ilmu, dukungan, pengalaman berharga, serta kesempatan mengajar kelas aljabar.
8. Teman-teman Alamin Yogyakarta yang telah menjadi keluarga dan teman berproses selama penulis menempuh pendidikan di Yogyakarta.
9. Anggana Putra, Gusti Syahrul, Nadya Biantri, dan Nicho Desvayawindra, teman sekolah dari Pontianak yang telah menemani, memberikan saran dan informasi seputar Yogyakarta, serta menjadi tempat berkonsultasi saat penulis awal pindah.

10. Mas Giga Hidjrika, Mas Karunia Perjuangan, Mas Lutfi Andriyanto, Mas Krisna Laksheta, Mas Afiq Zuhdi, Mas Orlando Ferrari, dan Bang Furqan yang telah menjadi mentor penulis selama masa perkuliahan.
11. Ridhwan Saputra, Adib Maulana, Arif Ibadurahman, Ega Baskara, serta teman-teman PPSMB Harjono Danoesastro 2 dan Sadewa 6 yang menjadi teman pertama penulis di UGM. Terima kasih pula kepada Mba Ifa dan Mba Ima sebagai pembimbing pada awal perkuliahan, serta Mas Damar dan Mba Dita sebagai mentor pada masa pengenalan kampus. Penulis juga berterima kasih kepada Mba Siti Malatania yang menjadi narahubung saat awal masuk kampus, serta semua pihak yang membantu penulis ketika pertama kali mendaftar perkuliahan.
12. Tim penelitian Psybot, yaitu Mba Khomsah, Virna, Anggita, Fina, dan Ibu Indrayanti, S.Psi., M.Si., Ph.D., Psikolog yang telah memberikan pengalaman penelitian dan kesempatan presentasi di PIMNAS beserta teman-teman kontingen, serta Bapak Muhammad Idham Ananta Timur, M.Kom., Mba Tata, Mba Riyanti, dan Mas Alfan dalam penelitian terkait kesehatan mental karyawan dengan skrining bot WhatsApp. Terima kasih juga kepada Dela Feby yang telah memberikan informasi terkait lomba yang paling berpengaruh dalam membuka kesempatan dan memperluas relasi penulis selama kuliah.
13. Teman-teman mahasiswa penggiat startup, yaitu Ridho Alfadri, Azfazaki Hilmi, Tantowi Shah, Achmad Luthfan, dan Mas Arief Rahman yang telah memberikan pengalaman berharga dalam berkarya dan mendorong penulis untuk berani mencoba hal baru.
14. Tim petugas TPS mahasiswa Karanggayam, yaitu Zahy, Farhan, Ipal, Astrid, Mba Safa, dan Mba Faiqoh yang bersama-sama telah menjalankan tugas sebagai petugas TPS pada Pemilu 2024 dengan bersih dan transparan.
15. Teman-teman Kelas Naskah, yaitu Kak Mark, Kak Alif, Manda, Mas Ai, dan Kak Maria, serta rekan-rekan dari divisi lain yang telah memberikan pengalaman kepada penulis sebagai developer volunteer untuk industri perfilman di Indonesia.
16. Bapak Dr. Guntur Dharma Putra, Bapak Dr. Bimo Sunarfri Hantono, S.T., M.Eng., Fathan Zahir, dan Mas Bagus Raka atas ilmu, pengalaman, dan kerja sama dalam penelitian sistem signage DTETI.
17. Bapak Dr. Ir. Ahmad Nasikun, S.T., M.Sc., Mas Iskan, Pak Hartanto, dan Pak Nur Muhammad yang telah memberikan bimbingan dan pengalaman berharga selama penulis bekerja paruh waktu sebagai developer di fakultas. Terima kasih pula kepada rekan-rekan kantor FT, yaitu Pak Purwoko, Pak Doni, Pak Bandi, Bu Desi, Bu Rahmi, Bu Nanda, Bu Acil, Bu Mega, Pak Untung, dan Pak Arief atas kebersamaan dan dukungannya, serta Bu Nuning yang telah memberikan kesempatan kepada penulis untuk mengembangkan sistem capaian kerja di FKKMK UGM. Serta Mas Deni dan Mbah yang sentiasa memberi dukungan dan yang saya selalu memintakan doa ketika bertemu di masjid teknik.
18. Rekan-rekan selama program kuliah magang IC Design Engineer di Polytron, Raphael, Fauzan, Kak Nur, Pande, Rizfa, Afgar, Andres, Ardian, Azriel, Bayu, Yavi, Chandra, Debora, Dwi, Faiq, Faisal, Ferhad, Hanif, Hisyam, Arif, Ivan, Mba Fazya, Fira, Ela, Renata, Aiman, Mas Andre, Mas Aziz, Galih, Mas Jabar, Jevon, Mas Priyo, Radit, Wisnu, Valen, Mas Wibaga, Mba Jihan, Mona, Fattah, Naumi, Nicky,

Panji, Putra, Qory, Raihan, Reza, Ridho, Surya, dan Tegar, serta Pak Arvin, Pak Riyad, dan Bu Dian atas pengalaman, kerja sama, dan dukungan selama kegiatan magang berlangsung. Terima kasih juga kepada Bapak Warsun Najib, M.T. yang menjadi pembimbing serta memberikan masukan saat laporan seminar kuliah praktik.

19. Sahabat satu tim Capstone D-02, yaitu Nanta, Titan, Laras, dan Nisa yang telah memberikan pengalaman dan kerja sama dalam menciptakan H.A.R.V.E.S.T. sebagai solusi pertanian bawang merah di Bantul.
20. Teman-teman KKN Minggir, yaitu Ayub, Subhan, Afifa, Anggun, Gama, Jasmin, Linggar, Akbar, Hanifah, Lea, Nisa, Padia, Rona, Iqbal, Amalia, Hilmi, Ilma, Irsyad, Lala, Safira, Berlean, Fajar, Farah, Hanif, Ifah, Sheila, Risti, dan Mba Vania, serta masyarakat Padukuhan Ngijon yang telah memberikan pengalaman, cerita, dan kebersamaan selama 51 hari pelaksanaan KKN.
21. Pak Yaenuri, A.Md., Pak Sunandar, S.T., Pak Nanang Muhammad Yusuf, S.T., Pak Heruwanto, S.T., M.M., Pak Nanang Dani Widyanto, S.T., Pak Rayuh Dhilah Hanggara, S.T., Pak Arwin Pamungkas, S.Pd., Pak Tri Wahyudiono, serta jajaran tenaga pendidik yang dengan penuh senyuman telah memberikan dukungan, bantuan, serta pengalaman selama masa perkuliahan dan kegiatan akademik maupun laboratorium.
22. Tim Senior Project Documind, yaitu Febri dan Yitzhak, Junior Project HarborFlow, yaitu Mirsad Alganawi dan Marseillo Satrian, serta Junior Project PaluGada, yaitu Luthfi Hanif dan Brian yang tetap melanjutkan pekerjaan tim ketika penulis menjalani kuliah magang.
23. Teman-teman Eternity Sistem Informasi UNTAN yang selalu memberikan dukungan kepada penulis selama menjalani dinamika kehidupan perkuliahan.
24. Teman-teman sobat DTI, yaitu Sulton Fatih, Hanif Riadi, Alfath Azka, Mas Adji Sudradjat, Mas Syafaq Abdallah, Mas Hafzhan Wisli, dan Fahmi Irfan Faiz yang telah menemani penulis selama pengerjaan skripsi di DTI.
25. Tetangga kos dekat apartemen, yaitu Mas Hasan, Mas Rama, Mas Choerul, Mas Kris, Mas Bagus, Mas Aldi, dan Mas Alvin yang selalu hadir saat dibutuhkan, termasuk ketika penulis perlu meminjam barang, serta Pak Ari selaku bapak kos, Pak Tumiran, Bu Mangkir, dan warga Pogung yang telah membantu penulis selama tinggal di sana.

Penulis menyadari bahwa skripsi ini masih memiliki kekurangan. Oleh karena itu, penulis terbuka terhadap saran dan masukan yang membangun. Penulis berharap skripsi ini dapat memberikan manfaat bagi pengembangan ilmu pengetahuan, khususnya dalam bidang sistem digital dan sistem embedded. Akhir kata, penulis mengucapkan terima kasih kepada seluruh pihak yang telah memberikan dukungan, bantuan, dan doa selama proses penyusunan skripsi ini. Semoga Allah subhanahu wa ta'ala membalas segala kebaikan yang telah diberikan. Aamiin.

Yogyakarta, 14 Mei 2026

Muhammad Muqtada Alhaddad

DAFTAR ISI

HALAMAN PENGESAHAN ii
PERNYATAAN BEBAS PLAGIASI iii
HALAMAN PERSEMBAHAN iv
KATA PENGANTAR vii
DAFTAR ISI viii
DAFTAR TABEL xii
DAFTAR GAMBAR xiii
DAFTAR SINGKATAN xvi
INTISARI xvii
ABSTRACT xviii
BAB I Pendahuluan 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 3
1.3 Tujuan Penelitian 3
1.3.1 Tujuan perancangan dan integrasi perangkat keras 3
1.3.2 Tujuan integrasi SoC dan perangkat lunak 3
1.3.3 Tujuan verifikasi dan validasi 4
1.4 Batasan Penelitian 4
1.4.1 Batasan perangkat keras dan RTL 4
1.4.2 Batasan SoC dan perangkat lunak 4
1.4.3 Batasan pengujian dan dokumentasi 5
1.5 Manfaat Penelitian 5
1.5.1 Manfaat akademik 5
1.5.2 Manfaat praktis 5
1.5.3 Manfaat pengembangan keahlian 5
1.6 Sistematika Penulisan 5
BAB II Tinjauan Pustaka dan Dasar Teori 7
2.1 Tinjauan Pustaka 7
2.1.1 Penelitian tentang implementasi I2S pada FPGA 7
2.1.2 Penelitian tentang prosesor soft-core dan integrasi SoC 7
2.1.3 Penelitian tentang sinkronisasi lintas domain clock (CDC) 7
2.1.4 Penelitian tentang penjadwalan firmware berbasis counter siklus .. 8
2.1.5 Gap penelitian dan kontribusi 8
2.2 Dasar Teori 8
2.2.1 Verilog Hardware Description Language (HDL) 8
2.2.2 Register Transfer Level (RTL) 9

2.2.3 Intellectual Property (IP) Core 10
2.2.4 Arsitektur MicroBlaze V dan Memory-Mapped I/O 11
2.2.5 Protokol Komunikasi AXI4-Lite 11
2.2.6 Protokol Komunikasi Audio I2S 12
2.2.7 Pembangkitan Clock Audio Berbasis NCO 13
2.2.8 DAC Audio PCM5102A 16
2.2.9 FPGA dan Board Basys3 16
2.2.10 Alur Pengembangan Vivado dan Vitis 17
BAB III Metode Penelitian 19
3.1 Alat dan Bahan 19
3.1.1 Perangkat keras 19
3.1.1.1 Perangkat pengembangan dan target hardware 19
3.1.1.2 Perangkat pengukuran dan pembantu 19
3.1.2 Perangkat lunak 20
3.2 Metode yang Digunakan 20
3.2.1 Metode perancangan hardware 20
3.2.2 Metode verifikasi dan simulasi 20
3.2.3 Metode integrasi SoC dan pengembangan software 20
3.3 Perancangan Block Design Sistem 20
3.3.1 Integrasi SoC MicroBlaze V 20
3.3.2 Perancangan Clocking Audio Berbasis NCO 21
3.3.3 Perancangan Register AXI4-Lite 21
3.3.4 Signal Generator dan Constraint FPGA 21
3.4 Alur Tugas Akhir 22
BAB IV Perancangan dan Implementasi 23
4.1 Implementasi IP I2S 23
4.1.1 Hierarki File dan Modul 23
4.1.2 Pembangunan Sistem Dasar MicroBlaze V 24
4.1.3 Struktur Modul RTL 26
4.1.4 Implementasi AXI4-Lite 29
4.1.5 Implementasi Register Konfigurasi 31
4.1.6 Implementasi Clock Domain Crossing 33
4.1.7 Implementasi Generator Clock Berbasis NCO 34
4.1.8 Implementasi Serializer Philips I2S 36
4.1.9 Implementasi Pengaturan Frekuensi Sampling Integer 39
4.1.10 Implementasi Enable, Mute, Reset, dan Output Assignment 40
4.1.11 Implementasi Integrasi pada Vivado Block Design 41
4.2 Implementasi Firmware 42
4.2.1 Struktur Program 43

4.2.2 Inisialisasi Periferal 43
4.2.3 Konfigurasi Register I2S 44
4.2.4 Pembangkitan Data Audio 45
4.2.5 Runtime Control Flow 47
4.2.6 Pengujian dan Validasi 49
4.3 Ringkasan Implementasi 50
BAB V Hasil dan Pembahasan 51
5.1 Verifikasi RTL dengan Testbench 51
5.2 Pengujian Timing dan Pembangkitan Clock I2S 52
5.3 Analisis Frekuensi Sampling 56
5.4 Validasi Audio pada Berbagai Frekuensi Sampling 56
5.5 Identifikasi dan Perbaikan Bug Firmware 56
5.5.1 Bug 1: Masker Bit Merusak Bit Tanda Sampel 57
5.5.2 Bug 2: Timing Loop Berbasis `usleep()` 57
5.5.3 Variabel dan Fungsi Kunci Firmware 58
5.6 Verifikasi Setelah Perbaikan Firmware 58
5.6.1 Akurasi Frekuensi Nada 58
5.6.2 Fungsionalitas Bit Kontrol 60
5.7 Verifikasi Integrasi dengan Filter Digital 60
5.7.1 Modul RTL `i2s_filter_bridge` 60
5.8 Analisis Penggunaan Sumber Daya dan Daya 64
5.9 Pembahasan terhadap Tujuan Penelitian 65
5.9.1 Perubahan Frekuensi Sampling 65
5.9.2 Variasi Lebar Data Sampel 67
5.9.3 Kontrol Kanal dan Status Mute 67
5.10 Perbandingan dengan Referensi Desain 69
5.11 Keterbatasan Desain 69
BAB VI Kesimpulan dan Saran 71
DAFTAR PUSTAKA 72
LAMPIRAN L-1
L.1 Hasil utilisasi resource L-1
L.1.1 Post-implementation resource report L-1
L.1.2 Timing report L-2
L.2 Timing diagram L-2
L.2.1 I2S audio transmission timing L-2
L.3 Format log hasil pengujian L-2
L.3.1 I2S audio playback test L-2
L.4 Deskripsi block design L-3
L.4.1 I2S IP Block (Standalone) L-3

L.4.2 Clocking View L-5
L.4.3 Interface View L-6
L.5 Hasil pengujian i2s transmitter - capture osiloskop L-7
L.5.1 Test Frekuensi Audio L-7
L.5.2 Test Kontrol Enable L-8
L.5.3 Test Kontrol Mute L-9
L.5.4 Test Routing Stereo L-9
L.5.5 Test Lebar Sample (Sample Width) L-11
L.6 Design Timing Summary L-12
L.7 Power Summary L-12
Ringkasan IP Block Design L-14
Kode Sumber dan Repositori L-21
L.8 Kode Testbench L-21

DAFTAR TABEL

Tabel 2.1 Perbandingan pembangkitan clock berbasis PLL dan NCO. 16

Tabel 4.1 Register map final IP I2S. 31
Tabel 4.2 Bitfield register `CONTROL`. 32

Tabel 5.1 Contoh penjumlahan pada *phase accumulator* BCLK untuk $F_s = 96$ kHz. 55
Tabel 5.2 Ringkasan penggunaan sumber daya FPGA Artix-7 XC7A35T (Basys3). 64
Tabel 5.3 Perbandingan fitur IP I2S Transmitter. 69

Tabel L.1 Ringkasan utilisasi resource FPGA L-1
Tabel L.2 Hasil analisis timing L-2
Tabel L.3 Segmen alamat penting pada block design `bdesign_clock_core` L-19

DAFTAR GAMBAR

Gambar 2.1 Diagram timing sinyal protokol I2S standar (Sumber: NXP). 13
Gambar 2.2 Alur pembangkitan frekuensi sampling menggunakan *phase accumulator* berbasis NCO. 15
Gambar 2.3 Arsitektur phase accumulator yang digunakan untuk menghasilkan BCLK. 15
Gambar 2.4 Spesifikasi arsitektur dan modul fisik DAC PCM5102A. 16
Gambar 2.5 Board FPGA Basys3 Artix-7 yang digunakan dalam penelitian. .. 17
Gambar 2.6 Alur pengembangan *hardware* dan *software* pada penelitian ini. .. 18

Gambar 3.1 Koneksi antarmuka pin antara FPGA Basys3 dan modul DAC PCM5102A. 19
Gambar 3.2 Komponen IP pendukung utama pada Block Design Vivado. 21
Gambar 3.3 Diagram alir pengerjaan penelitian tugas akhir skripsi. 22

Gambar 4.1 Arsitektur sistem akhir yang menghubungkan prosesor, UART, AXI4-Lite, dan IP I2S. 26
Gambar 4.2 Blok internal IP I2S yang terdiri dari AXI4-Lite, register file, NCO, latch sampel, dan serializer. 29
Gambar 4.3 Timing latching sampel kiri dan kanan pada awal frame stereo. .. 38
Gambar 4.4 Layout bit register `CONTROL` untuk konfigurasi frekuensi sampling integer. 39
Gambar 4.5 Rangkaian sistem yang digunakan pada Block Design Vivado. ... 42
Gambar 4.6 Koneksi pin PMOD FPGA ke modul DAC PCM5102A. 42

Gambar 5.1 Hasil simulasi internal IP I2S yang memperlihatkan pembangkitan clock berbasis *phase accumulator*, sinkronisasi frame stereo, dan serialisasi data sesuai spesifikasi Philips I2S. 53
Gambar 5.2 Keluaran osiloskop: nada sinus 1 kHz pada $F_s = 48$ kHz. 58
Gambar 5.3 Keluaran osiloskop: nada sinus 2 kHz pada $F_s = 48$ kHz. 59
Gambar 5.4 Keluaran osiloskop: nada sinus 5 kHz pada $F_s = 48$ kHz. 59
Gambar 5.5 Verifikasi fungsionalitas bit `ENABLE` dan `MUTE` pada keluaran audio analog. 60
Gambar 5.6 Keluaran stereo setelah perbaikan: kanal L dan R simetris dan identik. 61
Gambar 5.7 Integrasi IP I2S dengan blok filter IIR pada Vivado Block Design. 63
Gambar 5.8 Ringkasan timing desain pasca-implementasi dari Vivado Timing Analyzer. 64
Gambar 5.9 Analisis implementasi fisik pada FPGA Basys3. 64
Gambar 5.10 Perbandingan sinyal LRCK, BCLK, dan keluaran audio analog pada empat mode F_s standar. 66
Gambar 5.11 Efek variasi lebar data sampel terhadap sinyal serial DATA dan keluaran audio analog. 67
Gambar 5.12 Validasi rute kanal, bit `ENABLE`, dan bit `MUTE` melalui sinyal digital dan keluaran analog. 68

Gambar L.1 Ringkasan utilisasi resource (visual). L-1

Gambar L.2 Blok IP I2S standalone. L-4
Gambar L.3 Clocking view dan jalur clock sistem pada Vivado Block Design. L-5
Gambar L.4 Interface view yang menunjukkan pemetaan antarmuka I2S, PMOD, dan UART. L-6
Gambar L.5 Capture osiloskop: audio tone 1 kHz. L-7
Gambar L.6 Capture osiloskop: audio tone 2 kHz. L-7
Gambar L.7 Capture osiloskop: audio tone 5 kHz. L-8
Gambar L.8 Capture osiloskop: enable=0. ..... L-8
Gambar L.9 Capture osiloskop: enable=1. ..... L-8
Gambar L.10 Capture osiloskop: mute=0. ..... L-9
Gambar L.11 Capture osiloskop: mute=1. ..... L-9
Gambar L.12 Hasil capture osiloskop: Left channel only - hanya kanal left dioutput L-10
Gambar L.13 Hasil capture osiloskop: Right channel only - hanya kanal right dioutput L-10
Gambar L.14 Hasil capture osiloskop: Both channels - kedua kanal left dan right aktif simultan L-10
Gambar L.15 Hasil capture osiloskop: Sample width 16-bit - format audio PCM 16-bit L-11
Gambar L.16 Hasil capture osiloskop: Sample width 24-bit - format audio PCM 24-bit pada slot 32-bit L-11
Gambar L.17 Hasil capture osiloskop: Sample width 32-bit - full 32-bit slot I2S frame L-12
Gambar L.18 Design timing summary report - Worst Negative Slack (WNS), Total Negative Slack (TNS), dan status timing closure L-12
Gambar L.19 Power summary report - total on-chip power consumption, breakdown per resource (MMCM, Block RAM, Slice Logic), dan power supply margins L-13
Gambar L.20 Diagram block design berwarna yang menunjukkan hubungan prosesor, interconnect, clock, dan IP I2S. L-14
Gambar L.21 Tampilan clocking view pada block design untuk domain clock sistem dan jalur audio. L-14
Gambar L.22 Tampilan interface view yang memperlihatkan keterhubungan MicroBlaze, UARTLite, dan IP I2S melalui AXI SmartConnect. L-15
Gambar L.23 Tampilan awal konfigurasi Clocking Wizard sebelum penyesuaian frekuensi keluaran. L-15
Gambar L.24 Clocking Wizard setelah dikonfigurasi untuk menghasilkan clock sistem 100 MHz. L-15
Gambar L.25 Blok prosesor MicroBlaze V yang menjadi pengendali utama sistem. L-16
Gambar L.26 Block design reset controller yang membangkitkan sinyal reset sinkron untuk domain sistem. L-16
Gambar L.27 AXI SmartConnect yang merutekan transaksi dari MicroBlaze ke peripheral I2S dan UARTLite. L-16
Gambar L.28 Subsistem BRAM lokal untuk instruksi dan data MicroBlaze V. L-17
Gambar L.29 Microprocessor Debug Module yang digunakan untuk keperluan debug prosesor. L-17
Gambar L.30 Peripheral UARTLite sebagai antarmuka terminal firmware. L-18

Gambar L.31 Blok internal IP I2S yang digunakan sebagai peripheral utama. . L-18
Gambar L.32 Logo Hog (*HDL on Git*) yang digunakan untuk pengelolaan proyek HDL. L-26

DAFTAR SINGKATAN

AXI	=	Advanced eXtensible Interface
BCLK	=	Bit Clock
BRAM	=	Block RAM
CDC	=	Clock Domain Crossing
CPU	=	Central Processing Unit
DAC	=	Digital to Analog Converter
DDS	=	Direct Digital Synthesis
DMA	=	Direct Memory Access
DSP	=	Digital Signal Processor / Processing
FF	=	Flip-Flop
FIFO	=	First-In-First-Out
FPGA	=	Field Programmable Gate Array
HDL	=	Hardware Description Language
I2S	=	Inter-IC Sound
IP	=	Intellectual Property
IRQ	=	Interrupt Request
LRCK	=	Left/Right Clock (sama dengan WS)
LSB	=	Least Significant Bit
LUT	=	Look-Up Table
MCLK	=	Master Clock
MMCM	=	Mixed-Mode Clock Manager
MSB	=	Most Significant Bit
NCO	=	Numerically Controlled Oscillator
RISC-V	=	Reduced Instruction Set Computer V
RTL	=	Register Transfer Level
SoC	=	System on Chip
UART	=	Universal Asynchronous Receiver-Transmitter
WS	=	Word Select
XDC	=	Xilinx Design Constraints

INTISARI

Suara merupakan bagian penting dalam berbagai perangkat elektronik modern, mulai dari sistem hiburan hingga perangkat komunikasi. Untuk mentransmisikan data audio digital antarkomponen elektronik, digunakan protokol *Inter-IC Sound* (I2S) yang banyak diterapkan pada komunikasi antara prosesor atau mikrokontroler dengan perangkat audio digital seperti *Digital-to-Analog Converter* (DAC). Pada sistem berbasis *Field-Programmable Gate Array* (FPGA), IP (*Intellectual Property*) audio yang tersedia umumnya bersifat proprietary atau bergantung pada vendor tertentu sehingga membatasi fleksibilitas pengembangan. Oleh karena itu, perancangan IP transmitter I2S secara mandiri menggunakan bahasa deskripsi perangkat keras menjadi solusi yang relevan untuk membangun sistem audio digital yang lebih fleksibel dan mandiri.

Penelitian ini bertujuan merancang dan mengimplementasikan IP core transmitter I2S berbasis AXI4-Lite menggunakan Verilog HDL yang dapat diintegrasikan ke dalam *System-on-Chip* (SoC) berbasis prosesor MicroBlaze V pada FPGA Basys3. Kontribusi utama penelitian ini adalah pengembangan IP transmitter I2S yang mendukung frekuensi sampling integer dalam rentang operasi RTL menggunakan mekanisme *phase accumulator* berbasis *Numerically Controlled Oscillator* (NCO), sehingga tidak memerlukan IP audio berlisensi maupun pembangkit clock khusus untuk setiap frekuensi sampling.

Metodologi penelitian meliputi perancangan RTL transmitter I2S menggunakan Verilog HDL, implementasi antarmuka register AXI4-Lite, integrasi ke dalam SoC MicroBlaze V menggunakan AMD Vivado, pengembangan perangkat lunak bare-metal menggunakan AMD Vitis untuk konfigurasi register dan pengiriman data audio, serta verifikasi melalui simulasi RTL, pengamatan menggunakan osiloskop, dan pengujian perangkat keras menggunakan modul DAC PCM5102A.

Hasil penelitian menunjukkan bahwa IP transmitter I2S berhasil diintegrasikan ke dalam SoC berbasis MicroBlaze V dan dikendalikan melalui antarmuka AXI4-Lite. Sistem mampu membangkitkan sinyal BCLK, WS, dan DATA yang memenuhi spesifikasi Philips I2S, mentransmisikan data audio stereo dengan benar, serta mendukung frekuensi sampling integer standar maupun non-standar dalam rentang operasi RTL menggunakan satu clock referensi melalui mekanisme *phase accumulator*. Sistem SoC yang diimplementasikan pada FPGA Basys3 menggunakan 903 Slice, 2.435 LUT, 2.330 Flip-Flop, dan 32 BRAM dengan estimasi konsumsi daya total sebesar 0,209 W. Laporan timing menunjukkan WNS sebesar 0,064 ns pada clock 100 MHz, yang setara dengan estimasi frekuensi operasi maksimum sekitar 100,64 MHz.

Kata kunci: FPGA, Verilog, I2S, AXI4-Lite, MicroBlaze V, SoC, PCM5102A, Audio Digital.

ABSTRACT

Audio plays an essential role in modern electronic devices, ranging from entertainment systems to communication equipment. To transmit digital audio data between electronic components, the *Inter-IC Sound* (I2S) protocol is widely employed for communication between processors or microcontrollers and digital audio devices such as *Digital-to-Analog Converters* (DACs). In FPGA (*Field-Programmable Gate Array*)-based systems, commercially available audio Intellectual Property (IP) cores are often proprietary or vendor-dependent, limiting design flexibility and portability. Therefore, developing an independent I2S transmitter IP core using a hardware description language provides a flexible and reusable solution for FPGA-based embedded audio systems.

This research aims to design and implement an AXI4-Lite-based I2S transmitter IP core using Verilog HDL for integration into a *System-on-Chip* (SoC) built around the MicroBlaze V processor on the Basys3 FPGA platform. The main contribution of this work is the development of an I2S transmitter IP that supports programmable integer sampling rates within the RTL operating range through a *Numerically Controlled Oscillator* (NCO) based on a phase accumulator, eliminating the need for licensed audio IP cores or dedicated clock generators for each sampling rate.

The proposed methodology consists of RTL design of the I2S transmitter using Verilog HDL, implementation of an AXI4-Lite register interface, integration into a MicroBlaze V-based SoC using AMD Vivado, development of bare-metal software in AMD Vitis for register configuration and audio sample transmission, and verification through RTL simulation, oscilloscope measurements, and hardware validation using a PCM5102A DAC module.

The results demonstrate that the proposed I2S transmitter IP was successfully integrated into the MicroBlaze V-based SoC and controlled through the AXI4-Lite interface. The system correctly generated the BCLK, WS, and DATA signals in accordance with the Philips I2S specification, transmitted stereo audio samples reliably, and supported standard and non-standard integer sampling rates within the RTL operating range using a single reference clock. The implemented SoC used 903 slices, 2,435 LUTs, 2,330 flip-flops, and 32 BRAMs on the Basys3 FPGA, with an estimated total on-chip power consumption of 0.209 W. The timing report showed a WNS of 0.064 ns at 100 MHz, corresponding to an estimated maximum operating frequency of approximately 100.64 MHz.

Keywords: FPGA, Verilog HDL, Inter-IC Sound (I2S), AXI4-Lite, MicroBlaze V, System-on-Chip (SoC), PCM5102A, Digital Audio.

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan sistem embedded modern mendorong kebutuhan akan antarmuka audio digital yang fleksibel, dapat dikonfigurasi melalui perangkat lunak, dan mudah diintegrasikan ke dalam arsitektur System-on-Chip (SoC). Pada berbagai aplikasi seperti audio playback, perangkat edukasi, synthesizer sederhana, sistem notifikasi suara, dan eksperimen digital signal processing, data audio perlu dikirim dari prosesor ke perangkat audio eksternal secara sinkron dan andal. Dalam konteks ini, I2S (Inter-IC Sound) menjadi protokol yang sangat relevan karena dirancang khusus untuk transmisi data audio digital antarchip.

I2S merupakan standar antarmuka serial audio yang dikembangkan oleh Philips dan banyak digunakan untuk menghubungkan prosesor, codec, DAC, dan perangkat audio digital lain [1]. Protokol ini menggunakan tiga sinyal utama, yaitu *bit clock* (BCLK), *word select* (WS/LRCLK), dan *serial data* (SD). Relasi timing antar sinyal tersebut harus dipenuhi dengan ketat agar penerima audio dapat merekonstruksi sampel digital dengan benar. Dukungan terhadap berbagai *sample rate* dan *bit depth* membuat I2S cocok untuk implementasi audio pada FPGA, terutama ketika dibutuhkan fleksibilitas desain pada level register transfer.

FPGA menawarkan keunggulan utama dalam pengembangan sistem audio digital karena logika perangkat kerasnya dapat dikustomisasi sesuai kebutuhan aplikasi [2]. Dibandingkan pendekatan berbasis perangkat keras tetap, FPGA memungkinkan perancangan antarmuka komunikasi, logika buffer, pembagi clock, dan jalur data serial secara langsung dalam Verilog atau VHDL [3]. Ketika dikombinasikan dengan soft processor seperti MicroBlaze V, FPGA juga memberikan model kerja hardware-software co-design: fungsi real-time dan timing kritis ditangani oleh perangkat keras, sementara konfigurasi, pengiriman data, dan pengujian dikendalikan oleh perangkat lunak [4].

MicroBlaze V sebagai prosesor soft-core berbasis RISC-V [5] pada ekosistem AMD/Xilinx [6] menyediakan fondasi yang sesuai untuk integrasi peripheral kustom pada Vivado dan Vitis. Melalui antarmuka AXI4-Lite [7], peripheral dapat diakses dengan model memory-mapped I/O yang sederhana namun efektif. Dengan pendekatan ini, prosesor dapat menulis sampel audio, mengatur mode operasi, serta membaca status dari peripheral I2S menggunakan instruksi load/store biasa. Pendekatan tersebut memudahkan pengembangan perangkat lunak uji sekaligus menjaga modularitas desain perangkat keras.

Pemilihan MicroBlaze V sebagai prosesor pada penelitian ini didasarkan pada beberapa pertimbangan teknis. Dibandingkan dengan *ARM SoC* seperti Zynq-7000, MicroBlaze V tidak memerlukan prosesor ARM dan dapat dijalankan pada FPGA Artix-7 seperti Basys3 tanpa *hard processor*. Dibandingkan dengan *MicroBlaze Classic* berbasis ISA Xilinx proprietary, MicroBlaze V menggunakan arsitektur *RISC-V* standar terbuka sehingga memudahkan migrasi ke toolchain open-source. Dibandingkan pendekatan *bare-FPGA*, penggunaan MicroBlaze V memungkinkan konfigurasi register dan pembangkitan sinyal uji audio dikendalikan oleh perangkat lunak C. Kombinasi MicroBlaze V dan AXI4-Lite pada FPGA Artix-7 sesuai untuk pengembangan sistem tertanam pada penelitian ini.

Dalam praktik implementasi audio pada FPGA, permasalahan tidak hanya terletak pada pembangkitan sinyal I2S, tetapi juga pada kestabilan aliran data dan sinkronisasi antara perangkat lunak dengan serializer. Pada desain akhir, seluruh logika I2S berjalan pada domain `S_AXI_ACLK` 100 MHz sehingga register data dan serializer berada pada domain clock yang sama. Jika sampel berubah ketika word sedang diserialkan, keluaran audio dapat korup; oleh karena itu, sistem membutuhkan *shadow register* yang di-*latch* pada batas frame stereo serta mekanisme *handshake* melalui register `STATUS` agar firmware dapat mengirim sampel berikutnya tepat waktu. Pembangkitan `BCLK`, `WS`, dan `MCLK` dilakukan oleh akumulator fase DDS internal, sehingga frekuensi sampling dapat diprogram tanpa rekonfigurasi PLL eksternal. Kesesuaian pinout ke modul DAC eksternal [8] juga menjadi faktor penting yang memengaruhi keberhasilan sistem secara keseluruhan.

Berdasarkan kebutuhan tersebut, skripsi ini berfokus pada perancangan IP core transmitter I2S berbasis Verilog dengan antarmuka AXI4-Lite yang dapat diintegrasikan ke dalam SoC MicroBlaze V pada board Basys3 [9]. Kode sumber dan dokumentasi proyek ini tersedia secara terbuka pada repositori GitHub[1]. Target keluaran audio adalah modul PCM5102A sebagai DAC eksternal. Desain difokuskan pada mode TX-only, format Philips I2S, stereo, slot 32 bit, dan lebar sampel yang dapat dipilih hingga 32 bit. Penelitian ini mencakup evolusi rancangan dari empat mode frekuensi sampling standar menuju pengembangan berbasis DDS (*Direct Digital Synthesis*) untuk mendukung frekuensi sampling integer dalam rentang RTL. Tahapan penelitian meliputi perancangan RTL, integrasi Vivado Block Design [10], pengembangan perangkat lunak uji di Vitis, pengukuran sinyal, dan pengujian keluaran audio.

Kontribusi utama penelitian ini adalah perancangan Custom IP I2S Transmitter yang mendukung frekuensi sampling integer berbasis NCO dan dapat diintegrasikan sebagai peripheral AXI4-Lite pada SoC MicroBlaze V tanpa ketergantungan pada IP audio berlisensi pihak ketiga. RTL audio tersedia sebagai kode sumber, sedangkan pro-

[1] `https://github.com/adakhaddad/thesi2s`

ses packaging, integrasi, dan implementasi tetap bergantung pada toolchain FPGA yang digunakan.

Gambaran keseluruhan arsitektur sistem dibahas lebih rinci pada Bab IV. Prosesor MicroBlaze V berkomunikasi dengan IP I2S TX melalui bus AXI4-Lite, mengatur register kontrol dan mengirim sampel audio. IP I2S TX kemudian membangkitkan sinyal I2S (BCLK, WS, DATA) menuju DAC PCM5102A yang mengonversinya menjadi sinyal analog. UART digunakan untuk komunikasi terminal saat debugging.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah penelitian ini adalah:

1. Bagaimana merancang **IP core transmitter I2S** berbasis Verilog dengan antarmuka **AXI4-Lite** sehingga dapat dikontrol dari program Vitis melalui register memory-mapped dan mendukung frekuensi sampling integer melalui NCO?
2. Bagaimana melakukan **verifikasi fungsional dan timing** melalui simulasi bila tersedia, pengukuran sinyal, dan uji keluaran audio pada PCM5102A?

1.3 Tujuan Penelitian

Tujuan penelitian ini disusun agar selaras dengan rumusan masalah yang telah ditetapkan, sekaligus menegaskan bahwa skripsi ini menghasilkan **IP komunikasi sendiri** sehingga sistem tidak bergantung pada *license* atau IP komersial pihak ketiga.

1.3.1 Tujuan perancangan dan integrasi perangkat keras

1. Merancang dan mengimplementasikan **IP komunikasi audio I2S TX** berbasis Verilog dengan antarmuka **AXI4-Lite slave**, register kontrol dan data, serializer **Philips I2S**, serta mekanisme **shadow register** dan **status handshake** untuk pengiriman sampel audio stereo yang aman tanpa memakai IP komunikasi berlisensi dari pihak ketiga.
2. Mengimplementasikan pembangkitan detak audio berbasis **DDS/NCO** pada domain 100 MHz sehingga mendukung frekuensi sampling integer dalam rentang yang ditetapkan RTL, lalu mengintegrasikan IP ke dalam **Vivado Block Design** pada FPGA **Basys3** beserta pemetaan pin pada file `.xdc`.

1.3.2 Tujuan integrasi SoC dan perangkat lunak

1. Membangun SoC berbasis **MicroBlaze V** dengan AXI interconnect, BRAM, Clocking Wizard, dan akses ke peripheral I2S hasil rancangan sendiri melalui **Vitis**.

2. Mengembangkan aplikasi **Vitis** dalam bahasa C untuk konfigurasi register, pembangkitan sinyal uji audio, pengaturan kanal dan mode sampling, serta pengiriman sampel berbasis memory-mapped I/O ke IP komunikasi yang dibuat sendiri.

1.3.3 Tujuan verifikasi dan validasi

1. Memverifikasi sistem melalui pengamatan sinyal `i2s_bclk`, `i2s_ws`, `i2s_data`, status latch, serta bukti keluaran audio pada PCM5102A.
2. Melakukan synthesis dan implementation pada FPGA Xilinx Artix-7 serta menganalisis *resource utilization* seperti LUT, FF, dan BRAM.

1.4 Batasan Penelitian

Batasan penelitian dalam pengembangan **IP core I2S TX AXI4-Lite** ini meliputi:

1.4.1 Batasan perangkat keras dan RTL

1. Modul I2S dirancang sebagai **transmitter (TX-only)**; tidak mencakup receiver atau mode full-duplex.
2. Format audio dibatasi pada **Philips I2S**, stereo, **24 bit** per saluran dalam **slot 32 bit**; tidak mencakup TDM, left-justified, right-justified, atau multi-channel.
3. Dukungan frekuensi sampling dapat diprogram melalui register `SAMPLE_RATE_HZ` dalam rentang 1–195.312 Hz (batas aman DDS pada clock 100 MHz); pengujian utama difokuskan pada nilai standar 44,1 kHz, 48 kHz, 88,2 kHz, dan 96 kHz serta beberapa nilai non-standar.
4. Transfer data audio ke IP menggunakan **programmed I/O** berbasis register dan tidak mencakup integrasi **FIFO** maupun **DMA**.
5. Pengujian perangkat keras dilakukan pada board **Basys3** dengan modul **PCM5102A**; hasil pada platform lain dapat berbeda.

1.4.2 Batasan SoC dan perangkat lunak

1. Prosesor yang digunakan adalah **MicroBlaze V** pada ekosistem Vivado dan Vitis.
2. Lingkungan perangkat lunak yang digunakan adalah **bare-metal** tanpa RTOS atau Linux pada ruang lingkup utama.
3. UART digunakan untuk debug dan logging, bukan sebagai jalur streaming audio utama.

1.4.3 Batasan pengujian dan dokumentasi

1. Pengujian dibatasi pada simulasi *testbench*, osiloskop, dan uji dengar sederhana pada keluaran audio.
2. Dokumentasi disusun untuk kebutuhan skripsi dan tidak dimaksudkan sebagai datasheet komersial lengkap.

1.5 Manfaat Penelitian

1.5.1 Manfaat akademik

1. Menghasilkan contoh perancangan **IP I2S TX AXI4-Lite** berbasis Verilog yang dapat digunakan sebagai acuan pembelajaran audio digital pada FPGA.
2. Menyediakan studi kasus hardware-software co-design pada FPGA yang melibatkan RTL, integrasi SoC, dan pemrograman embedded C.
3. Memberikan dokumentasi register map, strategi *status handshake*, dan validasi sinyal I2S yang dapat digunakan kembali pada penelitian sejenis.

1.5.2 Manfaat praktis

1. Menyediakan dasar untuk proyek **audio output** berbasis FPGA seperti generator nada, alarm, dan sistem multimedia sederhana.
2. Menunjukkan pola desain **register-mapped I/O + status polling** yang dapat diadaptasi pada peripheral streaming sederhana lain.
3. Menjadi fondasi pengembangan lebih lanjut menuju sistem audio digital yang lebih kompleks, misalnya integrasi DMA atau dukungan Linux/ASoC.

1.5.3 Manfaat pengembangan keahlian

1. Memberikan pengalaman praktis dalam Verilog HDL, AXI4-Lite, integrasi Vivado Block Design, dan pengembangan aplikasi Vitis.
2. Mengembangkan kemampuan debugging hardware-software melalui simulasi *testbench*, pengukuran sinyal, dan analisis hasil implementasi.
3. Menjadi portofolio yang relevan untuk bidang FPGA, embedded systems, dan desain SoC.

1.6 Sistematika Penulisan

Bab I membahas pendahuluan yang mencakup latar belakang, rumusan masalah, tujuan, batasan, manfaat, dan sistematika penulisan.

Bab II memuat tinjauan pustaka dan dasar teori mengenai audio digital, protokol I2S, antarmuka AXI4-Lite, sinkronisasi lintas domain clock, Clocking Wizard, MicroBlaze V, dan PCM5102A.

Bab III menyajikan metode penelitian yang mencakup alat dan bahan, metode verifikasi, serta alur tahapan tugas akhir secara sistematis.

Bab IV membahas perancangan dan implementasi yang merinci spesifikasi fungsional IP, arsitektur register map, integrasi pada Vivado Block Design, serta detail logika RTL (DDS dan serializer).

Bab V menyajikan hasil dan pembahasan yang mencakup pengujian sinyal digital, analisis akurasi frekuensi sampling, identifikasi bug firmware, serta verifikasi akhir sistem.

Bab VI berisi kesimpulan dan saran pengembangan.

BAB II
TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Tinjauan Pustaka

Bagian ini membahas penelitian terdahulu yang relevan dengan implementasi protokol I2S pada FPGA, integrasi peripheral kustom pada SoC berbasis prosesor soft-core, serta penggunaan bus AXI4-Lite dan sinkronisasi lintas domain clock.

2.1.1 Penelitian tentang implementasi I2S pada FPGA

Implementasi antarmuka I2S pada FPGA telah dilaporkan dalam beberapa konteks. [11] merancang antarmuka I2S pada FPGA untuk menghubungkan prosesor dengan codec audio, menunjukkan dukungan *sample rate* hingga 96 kHz menggunakan pendekatan RTL langsung tanpa bus sistem. Sementara itu, [12] mengintegrasikan logika I2S bersama logic analyzer pada FPGA berbiaya rendah untuk kebutuhan instrumentasi.

Penelitian-penelitian tersebut mengonfirmasi kelayakan pembangkitan sinyal I2S pada FPGA, namun sebagian besar masih terbatas pada level simulasi atau pengujian RTL terisolasi. Penelitian ini berfokus pada pendekatan *hardware-software co-design* secara sistemik, yang mengintegrasikan register map, interkoneksi bus, dan validasi fungsional menggunakan perangkat lunak yang berjalan pada prosesor soft-core [13].

2.1.2 Penelitian tentang prosesor soft-core dan integrasi SoC

Pengembangan arsitektur RISC-V sebagai ISA terbuka [5] mendasari perancangan prosesor soft-core modern seperti MicroBlaze V pada ekosistem AMD/Xilinx [6]. Integrasi peripheral kustom ke dalam SoC berbasis bus AXI dieksplorasi oleh [14], yang menekankan aspek modularitas dan skalabilitas interkoneksi. Metodologi hardware-software co-design dan pengembangan driver bare-metal untuk peripheral kustom pada FPGA diulas secara komprehensif oleh [4].

2.1.3 Penelitian tentang sinkronisasi lintas domain clock (CDC)

Dalam sistem multi-clock, sinkronisasi data antar-domain (*Clock Domain Crossing*, CDC) sangat krusial. [15] menganalisis masalah metastabilitas dan memaparkan solusi flip-flop ganda. Pada rancangan final penelitian ini (`i2s_1_0`), serializer I2S berjalan sepenuhnya pada domain clock bus AXI 100 MHz, sehingga masalah CDC pada jalur data audio ditiadakan. Koordinasi aliran data diselesaikan melalui status handshake berbasis register.

2.1.4 Penelitian tentang penjadwalan firmware berbasis counter siklus

Penjadwalan periodik pada bare-metal sering kali memanfaatkan CSR `mcycle` pada arsitektur RISC-V sebagai referensi timing presisi tinggi [16]. Pada tahap awal penelitian ini, fungsi tunda `usleep()` dan register `mcycle` sempat digunakan untuk pengiriman sampel. Namun, sistem akhir beralih menggunakan polling bit register `STATUS` yang menyinkronkan pengiriman data secara fisik dengan detak konsumsi perangkat keras untuk mengeliminasi akumulasi keterlambatan (*drift*).

2.1.5 Gap penelitian dan kontribusi

Penelitian ini mengisi celah dokumentasi perancangan IP core I2S TX kustom untuk SoC MicroBlaze V yang mencakup seluruh alur RTL, register map, perangkat lunak bare-metal, analisis bug firmware (seperti kesalahan justifikasi bit dan sinkronisasi timing), serta verifikasi fisik pada platform hemat daya berbasis Basys3.

Kontribusi utama penelitian ini adalah:

1. Merancang IP core I2S transmitter AXI4-Lite berbasis **NCO** (*Numerically Controlled Oscillator*) desimal yang mendukung frekuensi sampling integer dalam RTL tanpa PLL eksternal.
2. Mengintegrasikan sistem SoC berbasis MicroBlaze V pada FPGA Artix-7 secara hemat sumber daya.
3. Menyediakan solusi hardware-software handshake berbasis status-polling untuk mencegah data corruption dan drift.
4. Mengidentifikasi dan mendokumentasikan penyelesaian tiga bug kritis pada lapisan firmware.

2.2 Dasar Teori

2.2.1 Verilog Hardware Description Language (HDL)

Verilog HDL merupakan bahasa formal yang digunakan untuk memodelkan, mendeskripsikan, dan mensimulasikan sistem digital [3]. Bahasa ini dikembangkan oleh Gateway Design Automation pada tahun 1984 dan distandarisasi sebagai IEEE 1364 pada tahun 1995. Sebagai *Hardware Description Language* (HDL), Verilog memungkinkan perancang untuk mendeskripsikan perilaku dan struktur rangkaian digital pada berbagai tingkat abstraksi, mulai dari level transistor (*switch-level*) hingga level pertukaran register (*Register Transfer Level*, RTL).

Pada tingkat RTL, Verilog mendeskripsikan aliran data antar register serta operasi logika kombinasional yang menghubungkannya, semuanya disinkronkan terhadap tepi clock tertentu. Pendekatan ini sangat sesuai untuk desain digital sinkron yang menjadi

fondasi sistem FPGA dan ASIC modern. Perbedaan mendasar antara pemrograman perangkat keras dengan Verilog dan pemrograman perangkat lunak konvensional (C, Python) terletak pada sifat eksekusinya: kode Verilog RTL menggambarkan koneksi fisik dan logika yang berjalan secara *konkuren*, bukan instruksi sekuensial *von Neumann*. Setiap blok `always` atau `assign` dalam Verilog mewakili jalur logika yang dieksekusi secara paralel oleh perangkat keras [17].

Pemilihan Verilog dalam penelitian ini didasarkan pada beberapa pertimbangan. Pertama, Verilog merupakan HDL yang didukung penuh oleh ekosistem Vivado, mencakup simulator, synthesizer, dan alat pengemasan IP. Kedua, sintaksis Verilog yang ringkas dan mirip dengan bahasa C memudahkan integrasi dengan kode yang dihasilkan oleh Vivado IP Packager serta memudahkan verifikasi fungsional melalui testbench [18]. Ketiga, adopsi Verilog yang luas di industri semikonduktor dan FPGA menjadikannya pilihan yang portabel dan relevan untuk publikasi penelitian. Seluruh logika IP I2S Transmitter pada penelitian ini, termasuk register map, akumulator NCO, serializer data, dan antarmuka AXI4-Lite slave, ditulis dalam Verilog RTL sintetis.

2.2.2 Register Transfer Level (RTL)

Register Transfer Level (RTL) merupakan gaya abstraksi perancangan digital di mana rangkaian dimodelkan sebagai aliran data antar register yang dipicu oleh tepi clock [17]. Dalam paradigma RTL, setiap sistem digital sinkron dapat dipandang sebagai kombinasi dari dua elemen utama: (1) elemen *sekuensial* berupa register atau *flip-flop* yang menyimpan status, dan (2) elemen *kombinasional* berupa gerbang logika yang menghitung nilai masukan register berikutnya berdasarkan status saat ini.

Hubungan antara logika kombinasional, logika sekuensial, register, dan clock merupakan inti dari desain RTL. Clock menyediakan referensi waktu diskrit yang menyelaraskan perpindahan data antar register. Pada setiap tepi clock (biasanya tepi naik), register menangkap nilai keluaran logika kombinasional, dan hasil tersebut menjadi masukan bagi logika kombinasional pada tahap berikutnya. Siklus ini membentuk pipa pemrosesan data yang sinkron dan deterministik [3].

Desain IP pada penelitian ini dikembangkan pada level RTL karena beberapa alasan. Pertama, RTL memberikan keseimbangan optimal antara portabilitas dan efisiensi sumber daya: kode RTL dapat disintesis untuk berbagai target FPGA atau ASIC tanpa modifikasi signifikan (*vendor-agnostic*), berbeda dengan desain level skematik yang terikat pada pustaka vendor tertentu. Kedua, desain RTL memungkinkan simulasi fungsional yang cepat untuk memverifikasi kebenaran logika sebelum implementasi fisik, yang sangat penting selama proses debugging. Ketiga, pendekatan RTL memfasilitasi modularitas dan *reuse*, di mana blok-blok fungsional seperti NCO, pencacah bit, dan AXI slave dapat dirancang, diuji, dan digabungkan secara independen. Seluruh komponen IP I2S

pada penelitian ini akumulator fase NCO, pencacah bit frame I2S, register map AXI4-Lite, dan serializer data diimplementasikan sebagai modul-modul RTL Verilog yang berdiri sendiri.

2.2.3 Intellectual Property (IP) Core

Intellectual Property (IP) *core* merupakan blok desain digital yang telah terverifikasi dan dapat digunakan kembali (*reusable*) dalam perancangan sistem terintegrasi skala besar seperti IC, SoC, maupun FPGA [18]. Konsep IP core lahir dari kebutuhan untuk mengatasi kompleksitas perancangan sistem digital modern: alih-alih merancang setiap blok dari awal (*start from scratch*), perancang dapat mengintegrasikan IP core yang sudah matang untuk mempercepat siklus pengembangan dan mengurangi risiko kesalahan desain.

Manfaat utama IP core meliputi *design reuse*, modularitas, dan percepatan pengembangan. Dengan menggunakan IP core, satu blok fungsional misalnya pengontrol I2S, antarmuka AXI, atau unit DSP dapat dirancang sekali dan digunakan kembali di berbagai proyek, menghemat waktu dan biaya desain. Pendekatan modular juga memudahkan pengujian dan verifikasi karena setiap blok dapat divalidasi secara independen [13].

Berdasarkan tingkat abstraksi dan portabilitasnya, IP core diklasifikasikan menjadi tiga kategori:

1. **Soft IP**: IP berupa kode HDL (Verilog/VHDL) pada level RTL. Soft IP bersifat paling portabel dan *vendor-agnostic*, tetapi hasil sintesis dan implementasinya (frekuensi maksimum, luas area) bergantung pada target FPGA serta alat sintesis yang digunakan. Desainer dapat memodifikasi kode sumber sesuai kebutuhan spesifik aplikasi.
2. **Firm IP**: IP berupa netlist atau representasi level gerbang yang telah dioptimasi untuk target teknologi tertentu. Firm IP menawarkan kinerja yang lebih dapat diprediksi dibandingkan soft IP, namun portabilitasnya lebih rendah dan modifikasi terbatas.
3. **Hard IP**: IP berupa *layout* fisik yang telah sepenuhnya ditempatkan dan di-*routing* pada silikon. Hard IP memberikan kinerja tertinggi dan konsumsi daya paling rendah, tetapi tidak portabel dan tidak dapat dimodifikasi. Contoh hard IP pada FPGA meliputi blok PLL, MMCM, pengontrol DDR, dan prosesor hard-core [2].

IP yang dirancang dalam penelitian ini tergolong **Soft IP** berbasis RTL Verilog. Seluruh kode sumber termasuk logika NCO, serializer I2S, register map konfigurasi, dan *slave* AXI4-Lite ditulis dalam Verilog portabel tanpa ketergantungan pada pustaka hardware vendor tertentu. Setelah proses verifikasi fungsional, IP ini dikemas menggunakan Vivado IP Packager menjadi format `.zip` yang dapat diimpor ke *IP Catalog* Vivado

untuk digunakan pada proyek FPGA lain [19]. Dengan demikian, IP I2S Transmitter yang dihasilkan bersifat *reusable* dan mendukung ekosistem perancangan SoC yang modular pada toolchain Vivado.

2.2.4 Arsitektur MicroBlaze V dan Memory-Mapped I/O

MicroBlaze V adalah soft-core processor 32-bit berbasis arsitektur RISC-V (RV32I) dari AMD/Xilinx [6]. Prosesor berkomunikasi dengan peripheral melalui model *Memory-Mapped I/O* (MMIO), di mana register kontrol dan data peripheral dipetakan ke alamat memori unik. Akses register dilakukan menggunakan instruksi load/store standar (`lw` / `sw`) melalui perantara fungsi pustaka `xil_io.h` seperti `Xil_Out32` dan `Xil_In32`.

Untuk menjaga integritas data tanpa FIFO atau DMA, dikembangkan mekanisme *Hardware-Software Handshake*. IP I2S Transmitter menyediakan bit status (`STATUS[0]`) yang berubah logika (*toggle*) setiap kali perangkat keras selesai memindahkan sampel dari register AXI ke *shadow register* internal untuk diserialisasikan. Perangkat lunak melakukan polling terhadap bit status ini sebelum mengirimkan sampel berikutnya, sehingga laju transfer data terkunci secara fisik dengan laju transmisi clock audio eksternal.

2.2.5 Protokol Komunikasi AXI4-Lite

AMBA (*Advanced Microcontroller Bus Architecture*) merupakan spesifikasi bus interkoneksi *on-chip* yang dirilis oleh ARM Limited untuk standarisasi komunikasi antar blok fungsional dalam suatu SoC [7]. Di dalam keluarga protokol AMBA, AXI (*Advanced eXtensible Interface*) menjadi pilihan utama untuk interkoneksi kinerja tinggi pada sistem FPGA modern. AXI memiliki tiga varian utama: AXI4 untuk transfer *burst* data berkecepatan tinggi, AXI4-Lite untuk transfer data tunggal (*single-beat*) yang disederhanakan, dan AXI4-Stream untuk aliran data tanpa alamat [20].

AXI4-Lite merupakan varian protokol AMBA yang dioptimalkan untuk akses memory-mapped register kontrol dan status pada peripheral kustom. Protokol ini mendukung lebar bus data 32-bit atau 64-bit dengan transaksi tunggal per alamat, tanpa dukungan *burst* maupun *out-of-order transaction*. Seluruh transaksi AXI4-Lite bersifat sinkron terhadap tepi naik clock dan menggunakan mekanisme *handshake* dua arah berbasis sinyal `VALID` (dinyalakan oleh pengirim) dan `READY` (dinyalakan oleh penerima). Data hanya ditransfer ketika kedua sinyal aktif dalam satu siklus clock yang sama.

Protokol AXI4-Lite terdiri atas lima kanal independen yang memisahkan alur penulisan dan pembacaan:

1. **Write Address Channel (AW)**: Mengirimkan alamat tujuan dari master ke slave.

2. **Write Data Channel (W)**: Mengirimkan data yang akan ditulis beserta strobe byte.
3. **Write Response Channel (B)**: Mengirimkan status respon penulisan (OKAY, EXOKAY, SLVERR, DECERR) dari slave ke master.
4. **Read Address Channel (AR)**: Mengirimkan alamat data yang akan dibaca.
5. **Read Data Channel (R)**: Mengirimkan data hasil pembacaan beserta status respon.

Mekanisme *read* dan *write* register pada AXI4-Lite berlangsung sebagai berikut. Pada transaksi *write*, master menempatkan alamat pada kanal AW dan data pada kanal W secara simultan atau berurutan. Slave mengakuisisi alamat dan data ketika `AWVALID` dan `AWREADY` (serta `WVALID` dan `WREADY`) aktif bersamaan, kemudian memproses penulisan dan mengembalikan respon melalui kanal B. Pada transaksi *read*, master menempatkan alamat pada kanal AR; slave merespon dengan menempatkan data yang diminta pada kanal R. Urutan ini memungkinkan pipeline transaksi yang efisien karena kanal alamat dan data terpisah.

Dalam konteks penelitian ini, MicroBlaze V berperan sebagai *master* AXI4-Lite yang mengakses register konfigurasi IP I2S Transmitter (*slave*) melalui bus AXI. Register IP terdiri atas `DATA_LEFT` dan `DATA_RIGHT` untuk data audio, `CONTROL` untuk `ENABLE`, `MUTE`, lebar sampel, dan frekuensi sampling, serta `STATUS` untuk indikator pergantian *frame latch*. Register-register tersebut dipetakan ke alamat memori unik dalam *address space* AXI4-Lite. Perangkat lunak bare-metal pada MicroBlaze V menulis dan membaca register-register tersebut menggunakan fungsi `Xil_Out32` dan `Xil_In32` yang membangkitkan transaksi AXI4-Lite secara otomatis.

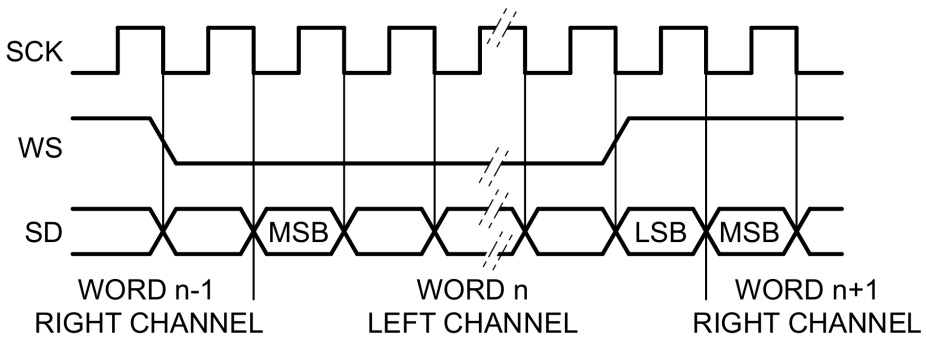
2.2.6 Protokol Komunikasi Audio I2S

I2S (*Inter-IC Sound*) merupakan protokol komunikasi serial sinkron tiga-kawat khusus untuk data audio digital PCM [1]. Tiga sinyal utama protokol ini meliputi:

- **Bit Clock (BCLK)**: Menentukan timing perpindahan bit data serial.
- **Word Select (WS / LRCLK)**: Menentukan kanal aktif (kiri = LOW, kanan = HIGH). Frekuensi WS sama dengan *sample rate* (F_s).
- **Serial Data (SD / DATA)**: Jalur transmisi bit data serial (MSB-first).

Gambar 2.1 menjadi acuan penjelasan format Philips I2S. Pada format tersebut, transisi `WS` terjadi satu pulsa BCLK sebelum bit MSB dikirim (*one-bit delay*) untuk sinkronisasi receiver. Penelitian ini menggunakan konfigurasi **stereo 24-bit di dalam slot 32-bit** per saluran (total 64 pulsa BCLK per frame stereo). Desain slot 32-bit menyederhanakan penyelarasan data pada bus AXI 32-bit. Hubungan frekuensi dinyatakan sebagai:

$$f_{\mathrm{BCLK}} = 64 \times F_s \tag{2-1}$$



Gambar 2.1. Diagram timing sinyal protokol I2S standar (Sumber: NXP).

2.2.7 Pembangkitan Clock Audio Berbasis NCO

Pembangkitan detak audio murni digital dilakukan menggunakan *Numerically Controlled Oscillator* (NCO) untuk mengurangi ketergantungan pada jalur clock audio khusus [21, 22]. NCO bekerja dengan prinsip akumulasi fase: suatu register (*phase accumulator*) bertambah secara periodik pada setiap siklus clock referensi ($f_{\text{ref}} = 100$ MHz) dengan nilai inkremen I hingga mencapai batas modulus M, lalu kembali ke nilai sisa dan memicu perubahan logika keluaran.

Akumulator fase pada penelitian ini menggunakan modulus numerik $M = 100.000.000$, sama dengan nilai frekuensi referensi dalam satuan Hz. Pemilihan ini membuat nilai inkremen berbasis frekuensi sampling integer mudah dihitung dari perangkat lunak. Modulus tersebut tidak menghapus kuantisasi atau jitter temporal; setiap luapan tetap terjadi pada tepi clock sistem terdekat.

Variabel-variabel dalam sistem NCO ini didefinisikan sebagai berikut:

1. f_{ref} : frekuensi clock referensi sistem (100 MHz), yaitu clock yang memicu pembaruan akumulator fase pada setiap siklusnya.
2. M : modulus akumulator fase, yaitu batas maksimum nilai fase. Dalam desain ini $M = f_{\text{ref}} = 100.000.000$, sehingga akumulator bertambah dari 0 hingga $M - 1$ dan kembali ke 0 setelah mencapai M.
3. I : inkremen fase (*phase increment*), yaitu nilai pertambahan akumulator pada setiap siklus clock. Semakin besar I, semakin cepat akumulator mencapai batas M dan semakin tinggi frekuensi keluaran.
4. $\theta(t)$: nilai akumulator fase pada waktu t (siklus clock ke-t).
5. f_{out} : frekuensi sinyal keluaran yang dihasilkan oleh NCO.

Persamaan pembaruan fase pada setiap siklus clock sistem adalah:

$$\theta(t) = \big(\theta(t-1) + I\big) \bmod M \tag{2-2}$$

Persamaan (2-2) menyatakan bahwa pada setiap siklus clock t, nilai fase saat ini ($\theta(t)$) diperoleh dengan menjumlahkan nilai fase sebelumnya ($\theta(t-1)$) dengan inkremen I, kemudian dimodulokan terhadap M. Ketika hasil penjumlahan $\theta(t-1)+I \geq M$, terjadi luapan (*overflow*) yang memicu perubahan logika (*toggle*) pada sinyal keluaran. Setiap luapan mewakili satu pertambahan setengah periode sinyal keluaran.

Frekuensi luapan akumulator fase dapat dihitung sebagai jumlah siklus per detik dikalikan probabilitas luapan per siklus, yaitu $f_{\text{ref}} \times (I/M)$. Karena satu periode penuh sinyal keluaran memerlukan dua kali luapan (satu untuk transisi naik dan satu untuk transisi turun), frekuensi keluaran NCO dinyatakan sebagai:

$$f_{\text{out}} = \frac{I}{2M} f_{\text{ref}} \tag{2-3}$$

Persamaan (2-3) merupakan relasi fundamental NCO yang menghubungkan seluruh variabel sistem. Semakin besar I, semakin tinggi f_{out}; semakin besar M, semakin rendah f_{out}.

Dari persamaan tersebut, nilai inkremen I yang diperlukan untuk menghasilkan frekuensi target tertentu dapat diturunkan sebagai:

$$I = \frac{2M\, f_{\text{out}}}{f_{\text{ref}}} \tag{2-4}$$

Dengan $M = f_{\text{ref}} = 100.000.000$, Persamaan (2-4) menyederhanakan menjadi $I = 2 \times f_{\text{out}}$. Untuk BCLK yang memiliki frekuensi $f_{\text{BCLK}} = 64 \times F_s$ (lihat Persamaan 2-1), nilai inkremennya adalah:

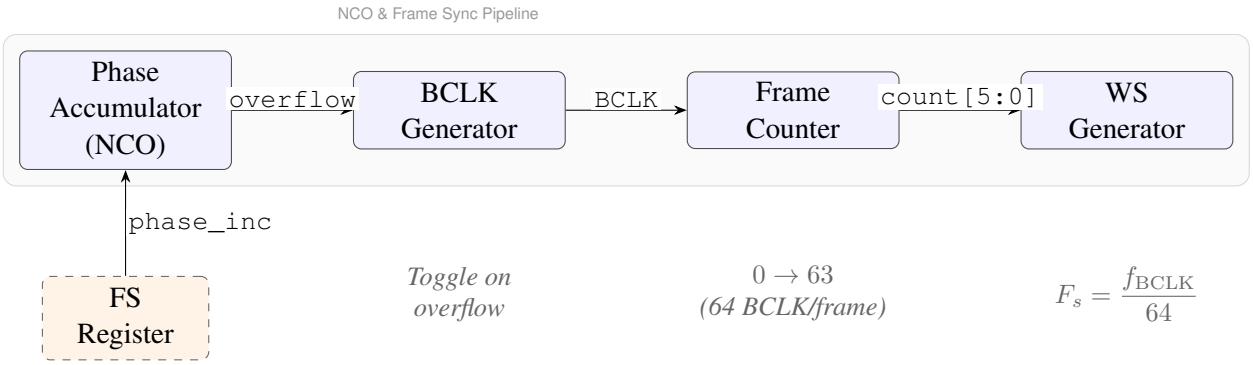
$$I_{\text{BCLK}} = 2 \times f_{\text{BCLK}} = 2 \times 64 \times F_s = 128 \times F_s \tag{2-5}$$

Untuk Master Clock (MCLK) dengan frekuensi $f_{\text{MCLK}} = 256 \times F_s$, nilai inkremennya adalah:

$$I_{\text{MCLK}} = 2 \times f_{\text{MCLK}} = 2 \times 256 \times F_s = 512 \times F_s \tag{2-6}$$

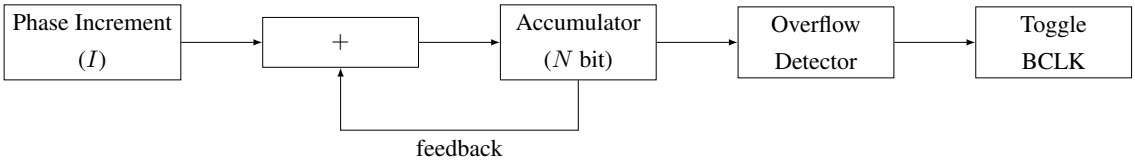
Dengan mekanisme ini, BCLK dan MCLK dibangkitkan dari clock sistem 100 MHz tanpa memerlukan PLL audio tambahan. Perangkat lunak pada MicroBlaze V menuliskan nilai F_s ke field `SAMPLE_RATE_HZ` pada register `CONTROL`, dan IP I2S menghitung inkremen BCLK serta MCLK melalui operasi geser bit (*bit-shift*) di dalam RTL. Nilai I_{BCLK} merupakan laju toggle BCLK, sehingga dua toggle membentuk satu periode BCLK. Sebaliknya, WS berubah setelah 64 periode BCLK, sedangkan MCLK memakai akumulator terpisah dengan inkremen I_{MCLK}. Contoh numerik pembaruan accumulator dan interpretasi waveform dibahas pada Bab V.

Gambar 2.2 menunjukkan alur pembangkitan frekuensi sampling dari register fre-



Gambar 2.2. Alur pembangkitan frekuensi sampling menggunakan *phase accumulator* berbasis NCO.

kuensi sampling hingga keluaran BCLK dan WS. Diagram ini menegaskan bahwa laju perubahan sinyal audio ditentukan oleh nilai inkremen pada akumulator fase.



Gambar 2.3. Arsitektur phase accumulator yang digunakan untuk menghasilkan BCLK.

Pada Gambar 2.3, nilai inkremen fase ditambahkan ke akumulator pada setiap siklus clock sistem. Ketika akumulator melampaui modulus, terjadi luapan yang digunakan untuk mengubah keluaran clock. Dengan demikian, laju perubahan BCLK dan MCLK ditentukan oleh frekuensi sampling yang ditulis pada register kontrol.

Dalam kode RTL Verilog, perkalian F_s dengan 128 dan 512 diimplementasikan secara efisien melalui pergeseran bit (*bit-shift*). Penyambungan tujuh bit nol di sebelah kanan menghasilkan $F_s \times 2^7 = F_s \times 128$, sedangkan penyambungan sembilan bit nol menghasilkan $F_s \times 2^9 = F_s \times 512$:

Cuplikan pada Kode II.1 menunjukkan implementasi pergeseran bit tersebut secara langsung di RTL Verilog.

```
1 wire [31:0] bclk_toggle_rate_hz = {sample_rate_sync, 7'b0}; // Fs * 128
2 wire [31:0] mclk_toggle_rate_hz = {sample_rate_sync, 9'b0}; // Fs * 512
```

Kode II.1. Implementasi pergeseran bit untuk laju toggle BCLK dan MCLK.

Variabel `bclk_toggle_rate_hz` menyimpan laju toggle BCLK, sedangkan `mclk_toggle_rate_hz` menyimpan laju toggle MCLK. Karena setiap periode clock memerlukan dua toggle, nilai tersebut digunakan sebagai laju perubahan level oleh logika NCO.

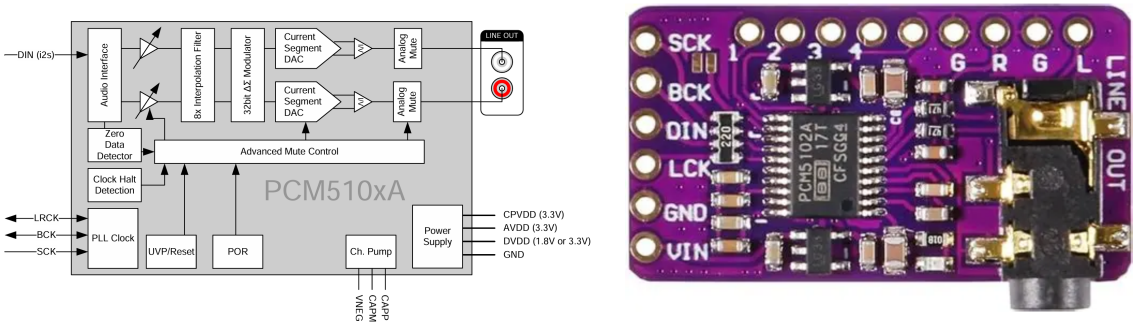
Tabel 2.1. Perbandingan pembangkitan clock berbasis PLL dan NCO.

Fitur	**PLL / MMCM (Analog)**	**NCO (Digital)**
Jitter	Bergantung pada karakteristik PLL/MMCM dan konfigurasi clock.	Tepi keluaran terkuantisasi terhadap periode clock sistem ($1/f_{\text{ref}}$).
Fleksibilitas	Bergantung pada konfigurasi dan resolusi clocking device.	Mendukung frekuensi sampling integer dalam rentang RTL.
Portabilitas	Menggunakan primitive atau IP clocking vendor.	Logika audio ditulis dalam RTL portabel, tetapi integrasi sistem tetap menggunakan toolchain vendor.

Perbandingan kedua teknik disajikan pada Tabel 2.1. Pendekatan NCO dipilih karena memberikan fleksibilitas penuh untuk memprogram frekuensi sampling secara dinamis dari perangkat lunak.

2.2.8 DAC Audio PCM5102A

PCM5102A adalah chip Digital-to-Analog Converter (DAC) 24-bit stereo dari Texas Instruments dengan keluaran analog level-line langsung tanpa filter eksternal [8]. DAC ini dapat beroperasi tanpa master clock (MCLK) eksternal dengan membangkitkan detak internal dari BCLK melalui unit PLL internalnya.



(a) Diagram blok internal fungsional. (b) Foto modul fisik PMOD PCM5102A.

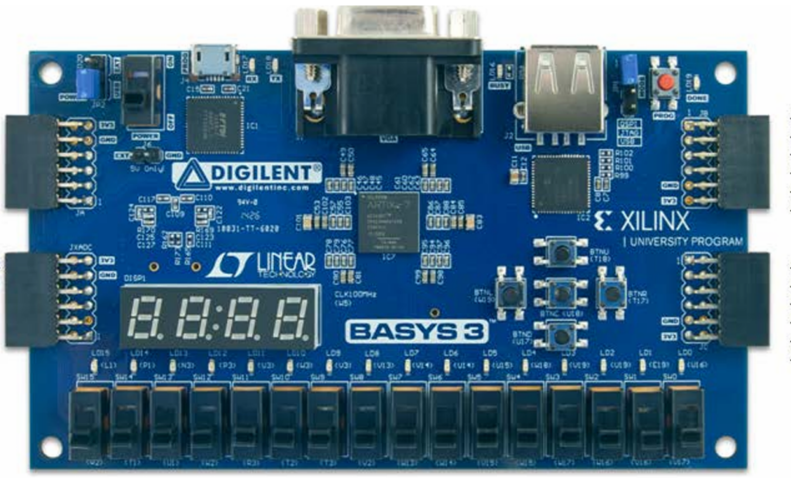
Gambar 2.4. Spesifikasi arsitektur dan modul fisik DAC PCM5102A.

Diagram blok fungsional dan foto fisik modul DAC PCM5102A disajikan pada Gambar 2.4. Modul eksternal ini dihubungkan langsung ke pin PMOD Basys3 untuk menerima data I2S dan menghasilkan keluaran audio stereo.

2.2.9 FPGA dan Board Basys3

FPGA (*Field-Programmable Gate Array*) memungkinkan kustomisasi jalur data perangkat keras secara langsung melalui deskripsi bahasa pemrograman RTL [23].

Pengujian hardware pada penelitian ini menggunakan board pengembangan **Basys3** berbasis chip Xilinx Artix-7 (XC7A35T-1CPG236C) [9].



Gambar 2.5. Board FPGA Basys3 Artix-7 yang digunakan dalam penelitian.

Board Basys3 (Gambar 2.5) menyediakan osilator clock 100 MHz, koneksi micro-USB untuk pemograman JTAG dan UART, serta empat port PMOD standar 12-pin untuk ekspansi periferal eksternal seperti DAC.

2.2.10 Alur Pengembangan Vivado dan Vitis

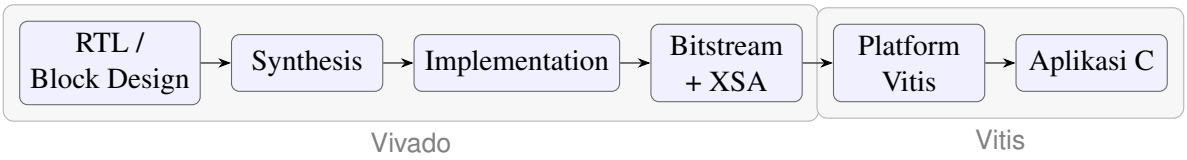
AMD Vivado Design Suite digunakan untuk membangun bagian perangkat keras sistem FPGA, mulai dari penulisan dan simulasi RTL, pengemasan custom IP, penyusunan Block Design, hingga pemetaan pin melalui file constraint `.xdc` [24]. Setelah rancangan hardware selesai, Vivado menjalankan tiga tahap utama:

1. **Synthesis**: menerjemahkan RTL dan Block Design menjadi netlist yang tersusun dari sumber daya logika FPGA, seperti LUT, flip-flop, BRAM, dan DSP.
2. **Implementation**: menempatkan sumber daya hasil sintesis pada FPGA (*placement*), menghubungkan jalurnya (*routing*), dan memeriksa pemenuhan batasan timing.
3. **Generate Bitstream**: menghasilkan file konfigurasi `.bit` yang digunakan untuk memprogram FPGA.

Untuk pengembangan perangkat lunak, Vivado mengekspor deskripsi hardware sebagai file `.xsa` (*Xilinx Support Archive*). File ini memuat informasi prosesor MicroBlaze V, peta alamat peripheral AXI4-Lite, konfigurasi interrupt, clock, serta bitstream yang terkait dengan platform hardware. AMD Vitis mengimpor file `.xsa` tersebut untuk membentuk *platform project* dan *Board Support Package* (BSP) [25]. Berdasarkan platform itu, aplikasi bare-metal ditulis dalam bahasa C pada file `.c`, lalu dikompilasi

menjadi berkas executable `.elf`. Saat pengujian, FPGA diprogram menggunakan bitstream dan Vitis memuat aplikasi `.elf` ke memori sistem melalui JTAG.

Dengan demikian, alur pengembangan hardware dan software pada penelitian ini diringkas pada Gambar 2.6:



Gambar 2.6. Alur pengembangan *hardware* dan *software* pada penelitian ini.

BAB III
METODE PENELITIAN

3.1 Alat dan Bahan

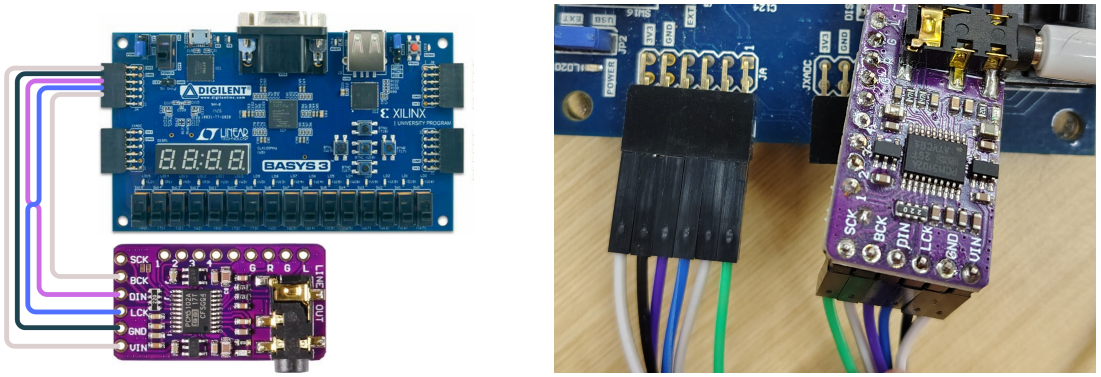
Penelitian ini menggunakan kombinasi perangkat keras dan perangkat lunak untuk perancangan, integrasi SoC, dan validasi fisik sistem I2S Transmitter.

3.1.1 Perangkat keras

3.1.1.1 Perangkat pengembangan dan target hardware

- **Laptop Lenovo ThinkPad**: Intel Core i5, RAM 8 GB, Windows 10 Pro. Digunakan untuk merancang RTL, menulis kode driver, dan menyusun laporan.
- **Board FPGA Basys3**: Berbasis AMD/Xilinx Artix-7 (XC7A35T), sebagai platform SoC soft-core utama [9].
- **Modul DAC Audio PCM5102A**: Berperan sebagai receiver I2S eksternal yang dihubungkan ke PMOD Basys3 untuk menghasilkan sinyal audio analog [8].

Konfigurasi koneksi antara FPGA Basys3 dan modul DAC PCM5102A diperlihatkan pada Gambar 3.1. Sinyal I2S (BCLK, WS, DATA) dan MCLK dihubungkan dari pin PMOD Basys3 ke input yang sesuai pada modul DAC, dengan jalur ground bersama untuk referensi sinyal.



(a) Diagram koneksi pin. (b) Implementasi fisik.

Gambar 3.1. Koneksi antarmuka pin antara FPGA Basys3 dan modul DAC PCM5102A.

3.1.1.2 Perangkat pengukuran dan pembantu

- **Osiloskop Digital TEXIO DCS-1102B**: Mengukur dan memverifikasi integritas sinyal BCLK, WS, dan DATA.
- **Headset stereo dan kabel jumper**: Untuk pengujian dengar audio analog langsung dari keluaran DAC.

3.1.2 Perangkat lunak

- **AMD Vivado 2025.2**: Tool utama untuk perancangan RTL Verilog, simulasi behavioral (XSim), sintesis, implementasi, dan perancangan SoC (Block Design) [10].
- **AMD Vitis 2025.2**: Platform pengembangan perangkat lunak C bare-metal untuk prosesor MicroBlaze V.
- **PuTTY dan Visual Studio Code IDE**: PuTTY digunakan sebagai terminal UART logging, sedangkan Visual Studio Code IDE untuk penyuntingan kode.

3.2 Metode yang Digunakan

3.2.1 Metode perancangan hardware

RTL dirancang secara modular menggunakan Verilog HDL yang memisahkan bagian datapath dan control path [3]. Desain difokuskan pada serializer I2S standar Philips, register data, shadow register untuk perlindungan data, serta modul clock desimal NCO. RTL audio tidak menggunakan primitif audio vendor-spesifik dan mendukung *sample rate* integer dalam rentang yang ditetapkan oleh RTL.

3.2.2 Metode verifikasi dan simulasi

Verifikasi dilakukan lewat simulasi testbench di Vivado Simulator untuk memastikan fungsionalitas sinyal I2S. Selanjutnya dilakukan validasi fisik dengan mengamati timing clock dan bit data serial pada osiloskop serta mendengarkan hasil reproduksi suara dari DAC.

3.2.3 Metode integrasi SoC dan pengembangan software

IP kustom dikemas menggunakan Vivado IP Packager lalu dimasukkan ke dalam Block Design untuk dihubungkan dengan MicroBlaze V melalui AXI SmartConnect [20]. Firmware bare-metal dikembangkan di Vitis untuk mengontrol peripheral, menginisialisasi parameter register, menjalankan generator DDS software, dan mengelola sinkronisasi pengiriman data lewat status polling.

3.3 Perancangan Block Design Sistem

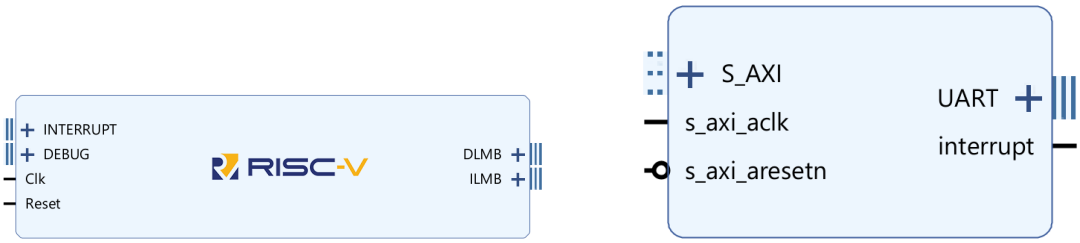
Perancangan Block Design dilakukan untuk menyatukan custom IP I2S ke dalam sistem SoC berbasis prosesor MicroBlaze V agar dapat diakses oleh program perangkat lunak.

3.3.1 Integrasi SoC MicroBlaze V

Vivado IP Integrator digunakan untuk menghubungkan komponen SoC: prosesor soft-core MicroBlaze V [6], memori lokal BRAM, Processor System Reset, AXI Smar-

tConnect, AXI UARTLite untuk komunikasi terminal, dan custom IP I2S. Alamat dasar (Base Address) peripheral ditentukan secara otomatis oleh Vivado dan dipetakan ke dalam header file `xparameters.h` di Vitis.

Komponen IP pendukung pada Block Design diperlihatkan pada Gambar 3.2. MicroBlaze V berperan sebagai pengendali utama, AXI UARTLite menyediakan jalur komunikasi serial untuk debugging, dan keduanya terhubung melalui AXI SmartConnect ke memori BRAM serta peripheral I2S.



(a) IP MicroBlaze V. (b) IP AXI UARTLite.

Gambar 3.2. Komponen IP pendukung utama pada Block Design Vivado.

3.3.2 Perancangan Clocking Audio Berbasis NCO

Clock audio diturunkan secara sinkron dari clock sistem AXI 100 MHz menggunakan teknik *Numerically Controlled Oscillator* (NCO) desimal modulo 100.000.000 (penjelasan matematis rinci dibahas pada Bab II). Sinyal detak yang dihasilkan meliputi **BCLK** ($64 \times F_s$) untuk serializer, serta **MCLK** ($256 \times F_s$) sebagai referensi clock DAC eksternal.

3.3.3 Perancangan Register AXI4-Lite

Akses register IP I2S oleh MicroBlaze V dilakukan melalui alamat memory-mapped (MMIO) berbasis antarmuka bus AXI4-Lite. Register internal IP terdiri dari `DATA_LEFT` (0x00), `DATA_RIGHT` (0x04), `CONTROL` (0x08), dan `STATUS` (0x0C). Peta register dan penjelasan fungsi bitfield sepenuhnya identik dengan detail perancangan yang disajikan pada Tabel 4.1 dan Tabel 4.2 di Bab IV (halaman 31).

Untuk sinkronisasi pengiriman data, firmware melakukan polling terhadap bit status `STATUS[0]` yang berubah logika setiap kali perangkat keras memindahkan sampel dari register AXI ke shadow register internal di awal frame I2S.

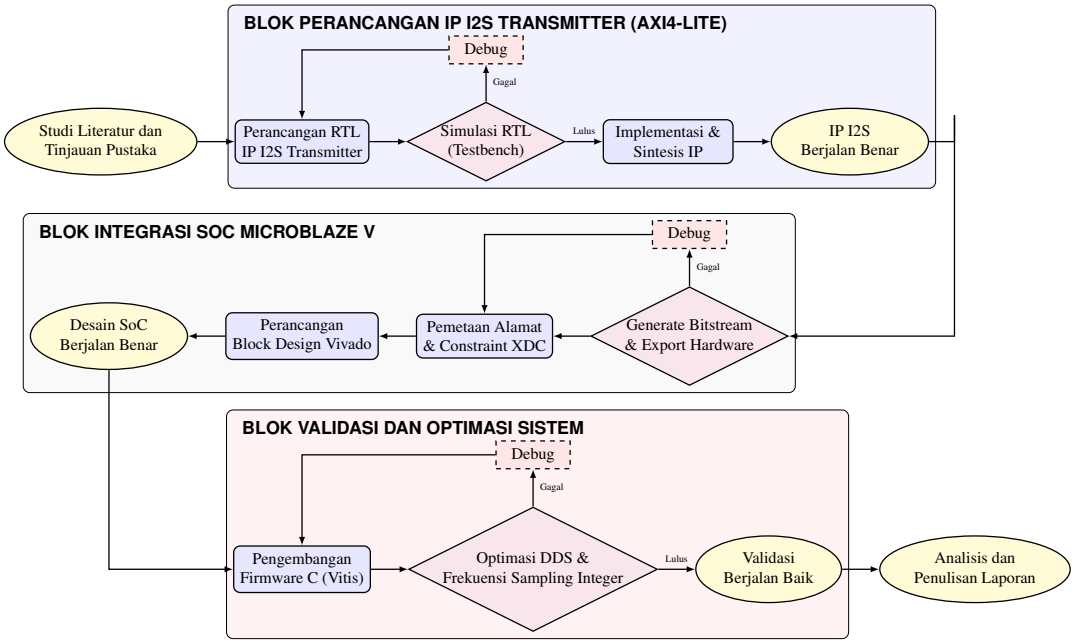
3.3.4 Signal Generator dan Constraint FPGA

Signal generator digital dibangun pada sisi software untuk menghasilkan sampel sinus atau kotak yang kemudian disalurkan ke register data peripheral. Pin eksternal I2S dipetakan ke port PMOD Basys3 menggunakan file constraint `.xdc` dengan standar level

tegangan LVCMOS33. Setelah Block Design divalidasi, dilakukan sintesis, implementasi, generate bitstream, dan ekspor hardware platform (`.xsa`) ke Vitis.

3.4 Alur Tugas Akhir

Penelitian ini mengikuti alur pengerjaan yang terbagi dalam tiga blok utama: perancangan IP I2S, integrasi SoC MicroBlaze V, serta validasi dan optimasi sistem. Diagram alir lengkap disajikan pada Gambar 3.3.



Gambar 3.3. Diagram alir pengerjaan penelitian tugas akhir skripsi.

BAB IV
PERANCANGAN DAN IMPLEMENTASI

4.1 Implementasi IP I2S

Bab ini mendokumentasikan realisasi akhir IP I2S berdasarkan source code aktual pada modul RTL dan firmware. Implementasi akhir terdiri atas wrapper `i2s.v`, core `i2s_slave_lite_v1_0_S00_AXI.v`, dan aplikasi bare-metal `freshi2s/sw/helloworld.c`. Berbeda dari rancangan awal yang masih menempatkan generator gelombang di RTL, implementasi final memindahkan pembangkitan sampel audio ke firmware, sedangkan RTL difokuskan pada antarmuka AXI4-Lite, pengaturan register, pembangkitan clock audio, dan serializer Philips I2S.

4.1.1 Hierarki File dan Modul

Source code implementasi terbagi menjadi dua lapisan: RTL perangkat keras dan firmware perangkat lunak. Seluruh file RTL ditulis dalam Verilog HDL, sedangkan firmware ditulis dalam bahasa C. Berikut adalah hierarki file beserta hubungan instansiasi antar modul:

1. **`i2s.v`** — *Wrapper top-level IP*. Berada di `ip_repo/i2s_1_0/hdl/`. Modul `i2s` tidak berisi logika fungsional; ia hanya meneruskan sinyal AXI4-Lite dan keluaran I2S ke modul inti. Instansiasi: `i2s_slave_lite_v1_0_S00_AXI`.
2. **`i2s_slave_lite_v1_0_S00_AXI.v`** — *Core IP I2S*. Berada di direktori yang sama. Modul ini mencakup seluruh logika: handshake AXI4-Lite, register file (DATA_LEFT, DATA_RIGHT, CONTROL, STATUS), phase accumulator NCO untuk pembangkitan BCLK/MCLK/WS, latch sampel, dan fungsi `i2s_next_serial_bit` yang mengimplementasikan serializer Philips I2S. Tidak ada sub-module yang diinstansiasi; seluruh fungsi terkandung dalam satu modul (*self-contained*).
3. **`freshi2s/sw/helloworld.c`** — *Firmware bare-metal*. Berisi fungsi `main()` yang menginisialisasi sistem, serta fungsi-fungsi statis: `control_word()` dan `apply_control()` untuk menulis register CONTROL, `frame_status_bit()` untuk membaca sinyal latch hardware, `pack_sample()` untuk menyesuaikan lebar sampel, `write_channels()` untuk mengirim data ke DATA_LEFT dan DATA_RIGHT, `stream_software_dds_sample()` untuk pembangkitan nada sinus/kotak, dan `handle_key()` untuk kontrol runtime melalui UART.
4. **`i2s.h/i2s.c`** — *Driver peripheral*. Makro `I2S_mWriteReg` dan `I2S_mReadReg` menyediakan antarmuka baca/tulis register, serta `I2S_Reg_SelfTest` untuk ve-

rifikasi dasar.

Hubungan instansiasi antar modul RTL adalah sebagai berikut:

```
i2s.v (top-level wrapper)
  +-- i2s_slave_lite_v1_0_S00_AXI.v (core)
        +-- AXI4-Lite handshake logic
        +-- Register file (slv_reg0--slv_reg3)
        +-- NCO phase accumulator (bclk/mclk/MCLK)
        +-- Latch sample (sample_left_q, sample_right_q)
        +-- Serializer (bit_count_q, i2s_next_serial_bit)
        +-- Output assignment (i2s_mclk, i2s_bclk, i2s_ws, i2s_data)
```

Sementara itu, firmware `freshi2s/sw/helloworld.c` mengakses core melalui alamat memory-mapped AXI4-Lite tanpa perantara driver tambahan. Fungsi `Xil_Out32` dan `Xil_In32` dari pustaka Vitis digunakan untuk menulis dan membaca register IP I2S. Dengan demikian, seluruh alur data dari firmware ke DAC dapat dirunut: `freshi2s/sw/helloworld.c` → AXI4-Lite → `i2s_slave_lite_v1_0_S00_AXI` → pin FPGA → PCM5102A.

4.1.2 Pembangunan Sistem Dasar MicroBlaze V

Sebagai landasan implementasi, penelitian ini terlebih dahulu membangun platform dasar System-on-Chip (SoC) berbasis MicroBlaze V dengan mengacu pada modul tutorial MicroBlaze V [26]. Platform tersebut kemudian dikembangkan lebih lanjut melalui integrasi IP I2S berbasis AXI4-Lite yang dirancang pada penelitian ini. Tutorial tersebut digunakan hanya untuk membentuk platform komputasi awal pada Vivado dan Vitis, sedangkan pengembangan utama pada penelitian ini tetap berfokus pada IP I2S kustom, register kontrol, pembangkitan frekuensi sampling, serializer Philips I2S, serta firmware pengujian. Desain akhir menggunakan satu domain clock 100 MHz, sehingga tidak memerlukan mekanisme *clock domain crossing* asinkron pada jalur audio.

Pembangunan dimulai dari pembuatan proyek Vivado untuk target papan Basys3 dan penetapan desain puncak yang akan menampung prosesor, memori lokal, interkoneksi AXI, serta periferal dasar. Pada tahap ini, struktur sistem belum memuat logika audio kustom, melainkan hanya menyediakan lingkungan pemrosesan yang stabil agar blok-blok berikutnya dapat diintegrasikan secara terukur. Melalui pendekatan tersebut, platform dasar berfungsi sebagai fondasi yang konsisten bagi seluruh eksperimen perangkat keras dan perangkat lunak pada penelitian ini.

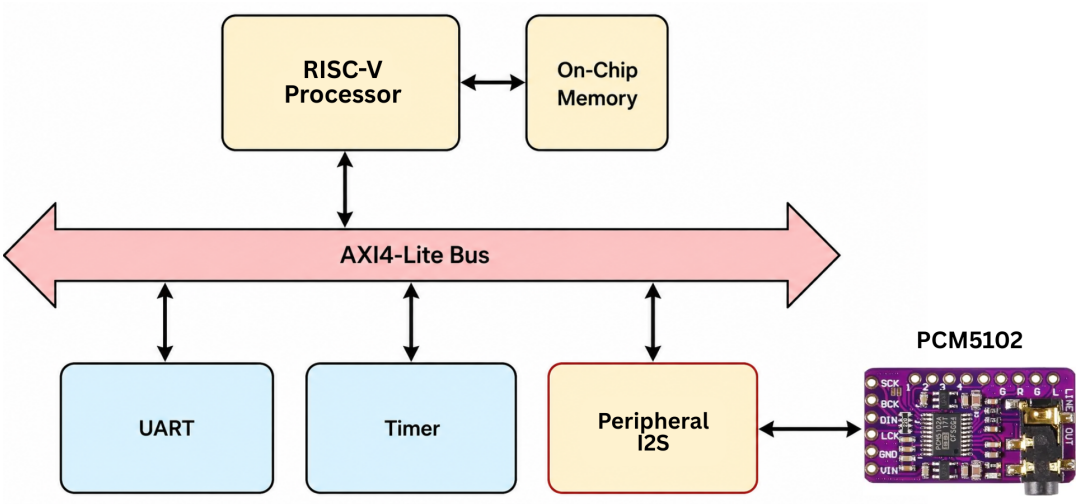
Di dalam *block design*, tutorial mengarahkan pembentukan subsistem prosesor yang memuat instansiasi MicroBlaze V, *Clocking Wizard*, memori lokal berbasis BRAM,

AXI SmartConnect, dan *AXI UARTLite*. *Clocking Wizard* disiapkan untuk menyediakan clock sistem 100 MHz yang seragam bagi subsistem prosesor dan periferal. Memori lokal dipakai sebagai ruang instruksi dan data awal bagi firmware, sedangkan AXI SmartConnect menangani perutean transaksi *memory-mapped* antara prosesor dan periferal. UARTLite dipakai sebagai kanal keluaran teks untuk verifikasi dasar eksekusi firmware. *Block automation* kemudian menyatukan parameter-parameter koneksi tersebut sehingga sistem dasar dapat dibangun sebagai satu kesatuan yang siap dipakai oleh perangkat lunak.

Setelah struktur internal selesai, Vivado menghasilkan *HDL wrapper* agar desain dapat diperlakukan sebagai satu entitas top-level. Langkah ini penting karena wrapper menjadi antarmuka antara *block design* dan implementasi fisik pada papan FPGA, termasuk pemetaan pin serta ekspor perangkat keras ke Vitis dalam bentuk berkas XSA. Dari titik ini, platform MicroBlaze V dapat dimanfaatkan untuk membuat firmware bare-metal yang menguji konektivitas prosesor, memori, dan UART sebelum peripheral audio kustom diintegrasikan.

Komponen yang diadopsi dari tutorial terbatas pada infrastruktur dasar, yaitu proyek Vivado, *block design*, *Clocking Wizard*, MicroBlaze V, *block automation*, memori lokal, AXI SmartConnect, AXI UARTLite, *HDL wrapper*, dan ekspor hardware ke Vitis. Sebaliknya, komponen yang menjadi kontribusi penelitian ini adalah IP I2S berbasis AXI4-Lite beserta implementasi register, logika NCO untuk pembangkitan frekuensi sampling integer dalam rentang RTL, serializer Philips I2S, serta firmware yang mengendalikan pengiriman sampel audio dan memvalidasi keluaran perangkat keras.

Setelah sistem dasar MicroBlaze V berhasil dibangun, penelitian ini melanjutkan proses dengan mengintegrasikan IP I2S berbasis AXI4-Lite yang dikembangkan secara mandiri. Seluruh mekanisme komunikasi audio, register konfigurasi, generator clock berbasis NCO, serializer Philips I2S, serta firmware pengujian merupakan hasil pengembangan pada penelitian ini.



Gambar 4.1. Arsitektur sistem akhir yang menghubungkan prosesor, UART, AXI4-Lite, dan IP I2S.

Gambar 4.1 menunjukkan arsitektur keseluruhan sistem yang dibangun pada penelitian ini. Prosesor MicroBlaze V berkomunikasi dengan IP I2S melalui antarmuka AXI4-Lite, sedangkan keluaran audio diteruskan menuju DAC melalui sinyal I2S.

4.1.3 Struktur Modul RTL

Struktur RTL dibangun dari wrapper yang mempertahankan nama IP `i2s` dan modul inti `i2s_slave_lite_v1_0_S00_AXI`. Wrapper ini digunakan agar integrasi pada Vivado Block Design tetap kompatibel dengan repository IP yang sudah ada.

```
module i2s #
(
    parameter integer C_S00_AXI_DATA_WIDTH = 32,
    parameter integer C_S00_AXI_ADDR_WIDTH = 4
)
(
    input  wire audio_clk,

    output wire i2s_mclk,
    output wire i2s_bclk,
    output wire i2s_ws,
    output wire i2s_data,

    input  wire                          s00_axi_aclk,
    input  wire                          s00_axi_aresetn,
    input  wire [C_S00_AXI_ADDR_WIDTH-1:0] s00_axi_awaddr,
    input  wire [2:0]                      s00_axi_awprot,
    input  wire                          s00_axi_awvalid,
```

```
    output wire                    s00_axi_awready,

    input  wire [C_S00_AXI_DATA_WIDTH-1:0] s00_axi_wdata,
    input  wire [(C_S00_AXI_DATA_WIDTH/8)-1:0] s00_axi_wstrb,
    input  wire                    s00_axi_wvalid,
    output wire                    s00_axi_wready,

    output wire [1:0]              s00_axi_bresp,
    output wire                    s00_axi_bvalid,
    input  wire                    s00_axi_bready,

    input  wire [C_S00_AXI_ADDR_WIDTH-1:0] s00_axi_araddr,
    input  wire [2:0]              s00_axi_arprot,
    input  wire                    s00_axi_arvalid,
    output wire                    s00_axi_arready,

    output wire [C_S00_AXI_DATA_WIDTH-1:0] s00_axi_rdata,
    output wire [1:0]              s00_axi_rresp,
    output wire                    s00_axi_rvalid,
    input  wire                    s00_axi_rready
);

i2s_slave_lite_v1_0_S00_AXI #(
    .C_S_AXI_DATA_WIDTH(C_S00_AXI_DATA_WIDTH),
    .C_S_AXI_ADDR_WIDTH(C_S00_AXI_ADDR_WIDTH)
)
i2s_slave_lite_v1_0_S00_AXI_inst
(
    .i2s_mclk     (i2s_mclk),
    .i2s_bclk     (i2s_bclk),
    .i2s_ws       (i2s_ws),
    .i2s_data     (i2s_data),

    .S_AXI_ACLK   (s00_axi_aclk),
    .S_AXI_ARESETN (s00_axi_aresetn),

    .S_AXI_AWADDR  (s00_axi_awaddr),
    .S_AXI_AWPROT  (s00_axi_awprot),
    .S_AXI_AWVALID (s00_axi_awvalid),
    .S_AXI_AWREADY (s00_axi_awready),

```

```
60      .S_AXI_WDATA   (s00_axi_wdata),
61      .S_AXI_WSTRB   (s00_axi_wstrb),
62      .S_AXI_WVALID  (s00_axi_wvalid),
63      .S_AXI_WREADY  (s00_axi_wready),
64
65      .S_AXI_BRESP   (s00_axi_bresp),
66      .S_AXI_BVALID  (s00_axi_bvalid),
67      .S_AXI_BREADY  (s00_axi_bready),
68
69      .S_AXI_ARADDR  (s00_axi_araddr),
70      .S_AXI_ARPROT  (s00_axi_arprot),
71      .S_AXI_ARVALID (s00_axi_arvalid),
72      .S_AXI_ARREADY (s00_axi_arready),
73
74      .S_AXI_RDATA   (s00_axi_rdata),
75      .S_AXI_RRESP   (s00_axi_rresp),
76      .S_AXI_RVALID  (s00_axi_rvalid),
77      .S_AXI_RREADY  (s00_axi_rready)
78  );
79
80  endmodule
```

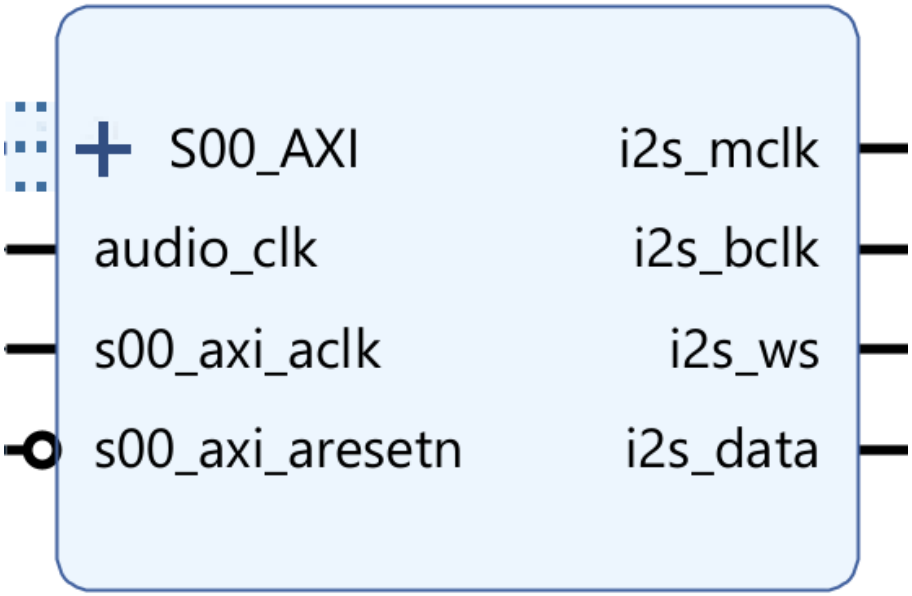
Kode IV.1. Wrapper IP `i2s.v` yang meneruskan antarmuka AXI4-Lite dan keluaran I2S ke modul inti.

Potongan kode pada kode IV.1 menunjukkan bahwa wrapper hanya berfungsi sebagai lapisan kompatibilitas antara Vivado Block Design dan modul inti. Seluruh transaksi AXI4-Lite diteruskan langsung ke modul `i2s_slave_lite_v1_0_S00_AXI`, sedangkan sinyal keluaran I2S (`i2s_mclk`, `i2s_bclk`, `i2s_ws`, dan `i2s_data`) diteruskan tanpa modifikasi. Port `audio_clk` tetap dipertahankan pada wrapper untuk menjaga kompatibilitas antarmuka IP dengan desain sebelumnya, meskipun pada implementasi akhir pembangkitan clock audio dilakukan secara internal menggunakan phase accumulator.

Struktur ini membuat seluruh logika fungsional berada di core, sehingga wrapper tetap sederhana dan mudah dipetakan pada IP Packager Vivado. Pada Gambar 4.1 hubungan antara prosesor, UART, dan IP I2S menjadi jelas: firmware menulis register melalui AXI4-Lite, lalu core menghasilkan sinyal `i2s_mclk`, `i2s_bclk`, `i2s_ws`, dan `i2s_data`.

Blok internal IP I2S diperlihatkan pada Gambar 4.2. Modul AXI4-Lite menangani transaksi bus dari prosesor, register file menyimpan konfigurasi dan data sampel, NCO

membangkitkan clock audio berdasarkan nilai register, latch sampel menyalin data pada batas frame stereo, dan serializer mengeluarkan bit serial dalam format Philips I2S.



Gambar 4.2. Blok internal IP I2S yang terdiri dari AXI4-Lite, register file, NCO, latch sampel, dan serializer.

4.1.4 Implementasi AXI4-Lite

Implementasi AXI4-Lite mengikuti pola handshake standar template Xilinx. Register handshake dan register file dideklarasikan sebagai berikut.

```
reg [C_S_AXI_ADDR_WIDTH-1:0] axi_awaddr;
reg                          axi_awready;
reg                          axi_wready;
reg [1:0]                    axi_bresp;
reg                          axi_bvalid;
reg                          aw_en;

reg [C_S_AXI_ADDR_WIDTH-1:0]  axi_araddr;
reg                          axi_arready;
reg [1:0]                    axi_rresp;
reg                          axi_rvalid;
reg [C_S_AXI_DATA_WIDTH-1:0]   axi_rdata;

reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg0;
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg1;
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg2;
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg3;
```

Kode IV.2. Register internal kanal AXI4-Lite dan register file empat register.

`slv_reg0` dan `slv_reg1` menyimpan sampel kanal kiri dan kanan. `slv_reg2` menyimpan register kontrol, sedangkan `slv_reg3` digunakan sebagai register status. Dengan struktur ini, semua komunikasi firmware-ke-hardware berlangsung melalui memory-mapped I/O, sehingga firmware tidak perlu mengakses sinyal internal secara langsung.

Keluaran AXI hanya merupakan penyalinan dari register internal.

```
assign S_AXI_AWREADY = axi_awready;
assign S_AXI_WREADY  = axi_wready;
assign S_AXI_BRESP   = axi_bresp;
assign S_AXI_BVALID  = axi_bvalid;
assign S_AXI_ARREADY = axi_arready;
assign S_AXI_RDATA   = axi_rdata;
assign S_AXI_RRESP   = axi_rresp;
assign S_AXI_RVALID  = axi_rvalid;
```

Kode IV.3. Pengaitan output AXI4-Lite terhadap register internal.

Strobe tulis dan baca dibentuk sebagai pulsa satu siklus agar setiap transaksi mempunyai batas yang jelas.

```
assign slv_reg_wren = axi_wready && S_AXI_WVALID &&
                      axi_awready && S_AXI_AWVALID;

assign slv_reg_rden  = axi_arready && S_AXI_ARVALID && !axi_rvalid;
```

Kode IV.4. Strobe tulis dan baca pada AXI4-Lite.

Pola `aw_en` dipakai untuk memastikan hanya satu transaksi tulis yang aktif pada satu waktu.

```
always @(posedge S_AXI_ACLK) begin
  if (!S_AXI_ARESETN) begin
    axi_awready <= 1'b0;
    aw_en       <= 1'b1;
  end else begin
    if (!axi_awready && S_AXI_AWVALID && S_AXI_WVALID && aw_en) begin
      axi_awready <= 1'b1;
      axi_awaddr  <= S_AXI_AWADDR;
      aw_en       <= 1'b0;
    end else if (S_AXI_BREADY && axi_bvalid) begin
      aw_en       <= 1'b1;
      axi_awready <= 1'b0;
    end else begin
```

```
        axi_awready <= 1'b0;
      end
    end
end
```

Kode IV.5. Handshake tulis menggunakan token `aw_en`.

Kanal baca bekerja dengan prinsip yang sama: alamat dilatch, data register dipilih, lalu `axi_rdata` dan `axi_rvalid` diaktifkan sampai master menerima data. Seluruh handshake ini penting karena firmware bergantung pada register memory-mapped sebagai satu-satunya jalur konfigurasi dan pengiriman sampel.

4.1.5 Implementasi Register Konfigurasi

Register map final terdiri atas empat register 32 bit. Tabel 4.1 menunjukkan pemetaan offset yang benar-benar dipakai pada source code. Rincian bitfield register `CONTROL` beserta fungsi setiap field dijelaskan pada Tabel 4.2.

Tabel 4.1. Register map final IP I2S.

Offset	Nama	Akses	Fungsi
`0x00`	`DATA_LEFT`	R/W	Sampel kanal kiri, right-justified.
`0x04`	`DATA_RIGHT`	R/W	Sampel kanal kanan, right-justified.
`0x08`	`CONTROL`	R/W	Enable, mute, sample width, dan sample rate.
`0x0C`	`STATUS`	R/W	Bit 0 toggle saat awal frame stereo.

Empat register digunakan karena IP harus menyimpan sampel kiri dan kanan secara terpisah, menerima konfigurasi operasi, serta menyediakan umpan balik sinkronisasi kepada firmware. Register `STATUS` melengkapi register data dan `CONTROL`: bit 0 berubah nilai setiap kali pasangan sampel kiri-kanan dilatch ke dalam serializer pada awal frame stereo. Firmware menggunakan perubahan nilai tersebut untuk mengetahui bahwa register data dapat diperbarui tanpa menimpa sampel yang masih dikirim.

Struktur register tersebut sesuai dengan komentar header pada source code. Alamat dasar ditentukan dari index register, bukan dari logika alamat kompleks, sehingga dekode register menjadi sederhana dan efisien.

```
localparam [1:0] REG_DATA_LEFT  = 2'h0;
localparam [1:0] REG_DATA_RIGHT = 2'h1;
localparam [1:0] REG_CONTROL    = 2'h2;
localparam [1:0] REG_STATUS     = 2'h3;

localparam [31:0] CONTROL_MASK    = 32'h07FF_FFFF;
```

Tabel 4.2. Bitfield register `CONTROL`.

Bit	Field	Keterangan
0	ENABLE	1 = output aktif; 0 = seluruh keluaran ditahan rendah.
1	MUTE	1 = data dipaksa nol, clock tetap berjalan.
6:2	SAMPLE_WIDTH	1–31 merepresentasikan lebar bit literal; nilai 0 dikodekan sebagai 32 bit.
26:7	SAMPLE_RATE_HZ	Target frekuensi sampling dalam Hz.
31:27	RESERVED	Diabaikan saat tulis dan dibaca sebagai nol.

```
localparam [31:0] DEFAULT_CONTROL = (32'd48000 << 7);
```

Kode IV.6. Deklarasi indeks register, mask baca, dan nilai default.

Pada saat reset, register dikembalikan ke keadaan aman: sampel nol, kontrol default 48 kHz dengan output nonaktif, dan status nol.

```
if (!S_AXI_ARESETN) begin
    slv_reg0 <= 32'd0;
    slv_reg1 <= 32'd0;
    slv_reg2 <= DEFAULT_CONTROL;
    slv_reg3 <= 32'd0;
end
```

Kode IV.7. Keadaan reset register file.

Penulisan register memperhatikan `S_AXI_WSTRB`, sehingga perubahan parsial per byte tetap dimungkinkan.

```
REG_DATA_LEFT: begin
    for (byte_index = 0;
        byte_index <= (C_S_AXI_DATA_WIDTH/8)-1;
        byte_index = byte_index + 1)
       if (S_AXI_WSTRB[byte_index])
          slv_reg0[(byte_index*8) +: 8] <=
             S_AXI_WDATA[(byte_index*8) +: 8];
end
```

Kode IV.8. Penulisan register `DATA_LEFT` berbasis byte strobe.

Register `CONTROL` memperoleh perlakuan tambahan karena bit cadangan harus tetap nol. Pada baca kembali, bit `[31:27]` dimask agar kontrak register tidak dilanggar oleh firmware.

```
REG_CONTROL:   reg_data_out = slv_reg2 & CONTROL_MASK;
REG_STATUS:    reg_data_out = {slv_reg3[31:1],
                               current_latch_status_q};
```

Kode IV.9. Masking bit cadangan pada register `CONTROL` dan `STATUS`.

Bagian `STATUS` tidak sepenuhnya berupa register biasa, karena bit 0 diganti oleh sinyal toggle `current_latch_status_q`. Inilah mekanisme debug dan sinkronisasi yang dipakai firmware untuk mengetahui bahwa frame baru telah dimulai.

4.1.6 Implementasi Clock Domain Crossing

Implementasi akhir tidak menggunakan clock domain crossing asinkron. Seluruh logika utama, termasuk register AXI, dekode kontrol, NCO, latch sampel, dan serializer, berjalan pada domain `S_AXI_ACLK`. Port `audio_clk` yang ada pada wrapper tidak dipakai pada core, sehingga arsitektur akhir menjadi single-clock design.

```
always @(posedge S_AXI_ACLK or negedge S_AXI_ARESETN) begin
  if (!S_AXI_ARESETN) begin
    bclk_phase_acc_q      <= 32'd0;
    mclk_phase_acc_q      <= 32'd0;
    bit_count_q           <= 6'd0;
    sample_left_q         <= 32'd0;
    sample_right_q        <= 32'd0;
    mclk_q                <= 1'b0;
    bclk_q                <= 1'b0;
    ws_q                  <= 1'b0;
    data_q                <= 1'b0;
    current_latch_status_q <= 1'b0;
  end else if (!enable_sync) begin
    bclk_phase_acc_q <= 32'd0;
    mclk_phase_acc_q <= 32'd0;
    bit_count_q      <= 6'd0;
    mclk_q           <= 1'b0;
    bclk_q           <= 1'b0;
    ws_q             <= 1'b0;
    data_q           <= 1'b0;
  end else begin
    if (mclk_phase_wrap) begin
      mclk_phase_acc_q <= mclk_phase_sum - AXI_CLK_HZ;
      mclk_q           <= !mclk_q;
    end else begin
      mclk_phase_acc_q <= mclk_phase_sum;
```

```
    end

    if (bclk_phase_wrap) begin
      bclk_phase_acc_q <= bclk_phase_sum - AXI_CLK_HZ;

      if (bclk_q) begin
        bclk_q      <= 1'b0;
        bit_count_q <= bit_count_q + 6'd1;
        ws_q        <= bit_count_q[5];
        data_q      <= i2s_next_serial_bit(
                         sample_for_i2s_left,
                         sample_for_i2s_right,
                         bit_count_q,
                         sample_width_sync);
      end else begin
        bclk_q <= 1'b1;

        if (bit_count_q == 6'd0) begin
          sample_left_q         <= slv_reg0;
          sample_right_q        <= slv_reg1;
          current_latch_status_q <= !current_latch_status_q;
        end
      end
    end else begin
      bclk_phase_acc_q <= bclk_phase_sum;
    end
  end
end
```

Kode IV.10. Core serializer berjalan pada satu domain clock yang sama.

Implementasi ini menghilangkan kebutuhan synchronizer dua flip-flop atau FIFO asinkron antara domain AXI dan domain audio. Dari sisi implementasi, hasilnya lebih sederhana dan lebih mudah diverifikasi, karena setiap perubahan register kontrol dan sampel terjadi pada clock yang sama dengan logika keluaran I2S.

4.1.7 Implementasi Generator Clock Berbasis NCO

Generator clock audio direalisasikan memakai akumulator fasa. Nilai referensi clock sistem adalah 100 MHz, sesuai asumsi pada source code, dan struktur dasarnya telah dijelaskan pada Bab II melalui Persamaan (2-3) serta Gambar 2.2 dan Gambar 2.3.

```
localparam [31:0] AXI_CLK_HZ = 32'd100_000_000;
```

```
localparam [19:0] DEFAULT_SAMPLE_RATE_HZ = 20'd48000;
localparam [19:0] MAX_SAFE_FS_HZ       = 20'd195312;
```

Kode IV.11. Parameter clock sistem dan batas sample rate.

Field `SAMPLE_RATE_HZ` pada register `CONTROL` dikonversi menjadi `sample_rate_sync`. Jika input bernilai nol maka digunakan default 48 kHz, sedangkan nilai di atas batas aman dibatasi ke `MAX_SAFE_FS_HZ`. Nilai inilah yang kemudian membentuk increment akumulator BCLK dan MCLK.

```
wire [31:0] bclk_toggle_rate_hz = {sample_rate_sync, 7'b0};
wire [31:0] mclk_toggle_rate_hz = {sample_rate_sync, 9'b0};

wire [31:0] bclk_phase_sum = bclk_phase_acc_q + bclk_toggle_rate_hz;
wire [31:0] mclk_phase_sum = mclk_phase_acc_q + mclk_toggle_rate_hz;

wire bclk_phase_wrap = (bclk_phase_sum >= AXI_CLK_HZ);
wire mclk_phase_wrap = (mclk_phase_sum >= AXI_CLK_HZ);
```

Kode IV.12. Pembentukan increment akumulator BCLK dan MCLK.

Secara fungsional, increment BCLK dikalikan dua kali lipat lebih tinggi daripada frekuensi keluaran efektif karena satu wrap akumulator hanya menghasilkan satu transisi logika. Akibatnya, BCLK efektif menjadi $64 \times F_s$ dan MCLK efektif menjadi $256 \times F_s$, sesuai kebutuhan I2S dan DAC.

Pembangkitan MCLK dan BCLK dilakukan independen, tetapi keduanya tetap bersumber dari `S_AXI_ACLK`.

```
if (mclk_phase_wrap) begin
    mclk_phase_acc_q <= mclk_phase_sum - AXI_CLK_HZ;
    mclk_q           <= !mclk_q;
end else begin
    mclk_phase_acc_q <= mclk_phase_sum;
end
```

Kode IV.13. Toggle MCLK berbasis wrap akumulator.

Penggunaan NCO membuat frekuensi sampling integer dapat diprogram dalam rentang RTL tanpa PLL audio tambahan [22]. Dari sudut implementasi, pendekatan ini menyederhanakan block design dan mengurangi ketergantungan pada clocking IP audio vendor-spesifik. Tepi keluaran tetap terkuantisasi terhadap clock sistem 100 MHz, sehingga klaim ini tidak berarti bahwa jitter temporal hilang.

4.1.8 Implementasi Serializer Philips I2S

Serializer memakai penghitung bit 6 bit agar satu frame stereo memiliki 64 slot bit. Lima bit bawah menunjukkan posisi di dalam kanal, sedangkan bit paling signifikan membedakan kanal kiri dan kanan.

```
reg [5:0]  bit_count_q;
reg [31:0] sample_left_q;
reg [31:0] sample_right_q;
reg        mclk_q;
reg        bclk_q;
reg        ws_q;
reg        data_q;
reg        current_latch_status_q;
```

Kode IV.14. Register state utama serializer I2S.

Fungsi `i2s_next_serial_bit` menentukan bit yang akan dikirim berikutnya. Fungsi ini memuat aturan Philips I2S: slot pertama bernilai nol sebagai delay satu bit setelah transisi WS, payload dikirim MSB-first, dan sisanya diisi nol jika lebar sampel lebih kecil dari 32 bit.

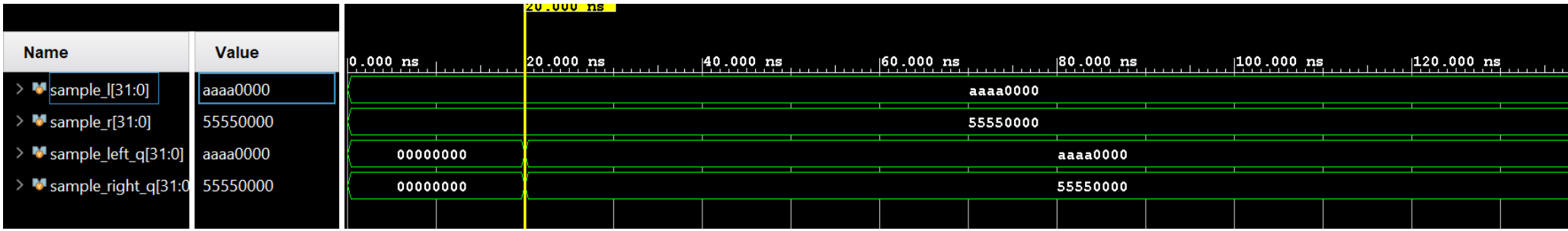
```
function i2s_next_serial_bit;
   input [31:0] left_samp;
   input [31:0] right_samp;
   input [5:0]  bc;
   input [5:0]  width;
   reg          in_left;
   reg [4:0]    pos;
   reg [5:0]    bit_idx;
   begin
      in_left = (bc < 6'd32);
      pos     = bc[4:0];

      if (pos == 5'd0)
         i2s_next_serial_bit = 1'b0;
      else if (pos <= width) begin
         bit_idx = width - {1'b0, pos};
         i2s_next_serial_bit = in_left ? left_samp[bit_idx[4:0]]
                                       : right_samp[bit_idx[4:0]];
      end else
         i2s_next_serial_bit = 1'b0;
   end
endfunction
```

Kode IV.15. Fungsi pembentuk bit serial Philips I2S.

Perubahan `data_q`, `ws_q`, dan `bit_count_q` dilakukan pada tepi jatuh BCLK. Pada tepi naik BCLK, receiver dianggap melatch data, sehingga data tidak diubah pada saat itu. Pola ini menjaga timing Philips I2S: WS berubah satu siklus sebelum MSB kanal berikutnya.

```
if (bclk_q) begin
    bclk_q      <= 1'b0;
    bit_count_q <= bit_count_q + 6'd1;
    ws_q        <= bit_count_q[5];
    data_q      <= i2s_next_serial_bit(
                     sample_for_i2s_left,
                     sample_for_i2s_right,
                     bit_count_q,
                     sample_width_sync);
end else begin
    bclk_q <= 1'b1;
    if (bit_count_q == 6'd0) begin
        sample_left_q         <= slv_reg0;
        sample_right_q        <= slv_reg1;
        current_latch_status_q <= !current_latch_status_q;
    end
end
```

Kode IV.16. Latching sampel dan pembaruan WS/DATA pada tepi BCLK yang sesuai.



Gambar 4.3. Timing latching sampel kiri dan kanan pada awal frame stereo.

Gambar 4.3 untuk menunjukkan *waveform* bahwa pasangan sampel kiri dan kanan tidak diambil secara acak per bit, tetapi dilatch utuh pada awal frame. Dengan demikian, satu frame stereo selalu menggunakan pasangan sampel yang konsisten.

4.1.9 Implementasi Pengaturan Frekuensi Sampling Integer

Pengaturan frekuensi sampling dimasukkan melalui bit `SAMPLE_RATE_HZ` pada register `CONTROL`. Di source code, nilai ini didefinisikan pada bit `[26:7]` dan dikonversi menjadi frekuensi yang digunakan NCO.

```
wire        enable_sync = slv_reg2[0];
wire        mute_sync   = slv_reg2[1];

wire [4:0]  sample_width_raw  = slv_reg2[6:2];
wire [5:0]  sample_width_sync = (sample_width_raw == 5'd0) ? 6'd32
                                      : {1'b0, sample_width_raw};

wire [19:0] sample_rate_raw     = slv_reg2[26:7];
wire [19:0] sample_rate_nonzero = (sample_rate_raw == 20'd0)
                          ? DEFAULT_SAMPLE_RATE_HZ
                          : sample_rate_raw;
wire [19:0] sample_rate_sync    = (sample_rate_nonzero > MAX_SAFE_FS_HZ)
                          ? MAX_SAFE_FS_HZ
                          : sample_rate_nonzero;
```

Kode IV.17. Dekode control register untuk enable, mute, width, dan sample rate.

Konstruk ini penting karena `SAMPLE_WIDTH` tidak dapat menyimpan nilai 32 secara literal dalam 5 bit, sehingga nilai 0 dipakai sebagai kode untuk 32 bit. Konversi semacam ini juga menjelaskan mengapa firmware dan RTL harus memakai pemetaan yang sama agar interpretasi register tetap identik.

Gambar 4.4. Layout bit register `CONTROL` untuk konfigurasi frekuensi sampling integer.

Gambar 4.4 memperlihatkan posisi field `ENABLE`, `MUTE`, `SAMPLE_WIDTH`, dan `SAMPLE_RATE_HZ`. Field ini merupakan jembatan antara firmware dan NCO di RTL, sehingga perubahan frekuensi sampling cukup dilakukan melalui satu transaksi AXI4-Lite.

4.1.10 Implementasi Enable, Mute, Reset, dan Output Assignment

Fungsi `ENABLE` memutus atau mengaktifkan seluruh keluaran I2S. Ketika `ENABLE=0`, semua register output dan akumulator direset ke nol agar keluaran berhenti secara bersih.

```
end else if (!enable_sync) begin
    bclk_phase_acc_q <= 32'd0;
    mclk_phase_acc_q <= 32'd0;
    bit_count_q      <= 6'd0;
    mclk_q           <= 1'b0;
    bclk_q           <= 1'b0;
    ws_q             <= 1'b0;
    data_q           <= 1'b0;
end else begin
    if (mclk_phase_wrap) begin
        mclk_phase_acc_q <= mclk_phase_sum - AXI_CLK_HZ;
        mclk_q           <= !mclk_q;
    end else begin
        mclk_phase_acc_q <= mclk_phase_sum;
    end

    if (bclk_phase_wrap) begin
        bclk_phase_acc_q <= bclk_phase_sum - AXI_CLK_HZ;

        if (bclk_q) begin
            bclk_q      <= 1'b0;
            bit_count_q <= bit_count_q + 6'd1;
            ws_q        <= bit_count_q[5];
            data_q      <= i2s_next_serial_bit(
                             sample_for_i2s_left,
                             sample_for_i2s_right,
                             bit_count_q,
                             sample_width_sync);
        end else begin
            bclk_q <= 1'b1;

            if (bit_count_q == 6'd0) begin
                sample_left_q         <= slv_reg0;
                sample_right_q        <= slv_reg1;
                current_latch_status_q <= !current_latch_status_q;
            end
        end
```

```
    end else begin
        bclk_phase_acc_q <= bclk_phase_sum;
    end
end
```

Kode IV.18. Perilaku saat `ENABLE` dimatikan.

Mute tidak mematikan clock, tetapi mengganti sampel yang akan diserialisasi menjadi nol.

```
wire [31:0] sample_for_i2s_left  = mute_sync ? 32'd0 : sample_left_q;
wire [31:0] sample_for_i2s_right = mute_sync ? 32'd0 : sample_right_q;
```

Kode IV.19. Logika mute pada jalur sampel serializer.

Pendekatan ini dipilih karena clock tetap stabil ketika mute aktif, sehingga DAC tidak kehilangan lock dan transisi kembali ke audio aktif menjadi lebih halus. Setelah seluruh proses serialisasi selesai, keluaran fisik dihubungkan secara langsung.

```
assign i2s_mclk = mclk_q;
assign i2s_ws   = ws_q;
assign i2s_bclk = bclk_q;
assign i2s_data = data_q;
```

Kode IV.20. Pengaitan register output ke pin I2S.

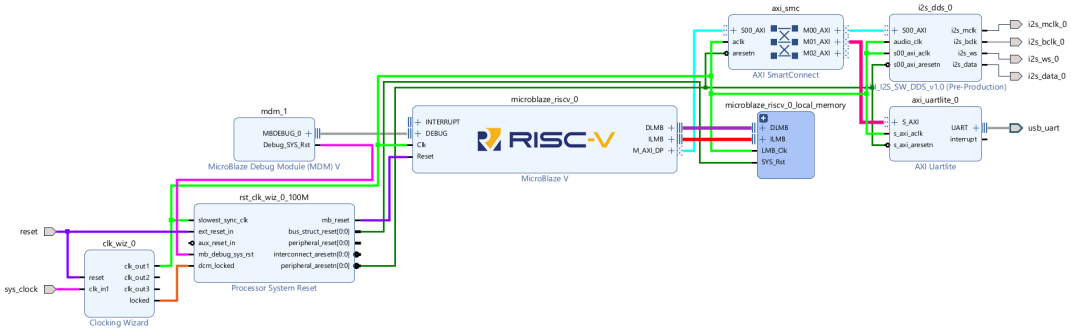
Reset menggunakan `S_AXI_ARESETN` aktif rendah dan membersihkan semua state penting, termasuk akumulator, penghitung bit, sampel latch, dan bit status. Di sisi integrasi, sinyal `AWPROT` dan `ARPROT` tidak dipakai secara fungsional, namun tetap diterima untuk kompatibilitas pin AXI.

```
wire _unused_ok = S_AXI_AWPROT[0] ^ S_AXI_AWPROT[1] ^ S_AXI_AWPROT[2] ^
                  S_AXI_ARPROT[0] ^ S_AXI_ARPROT[1] ^ S_AXI_ARPROT[2];
```

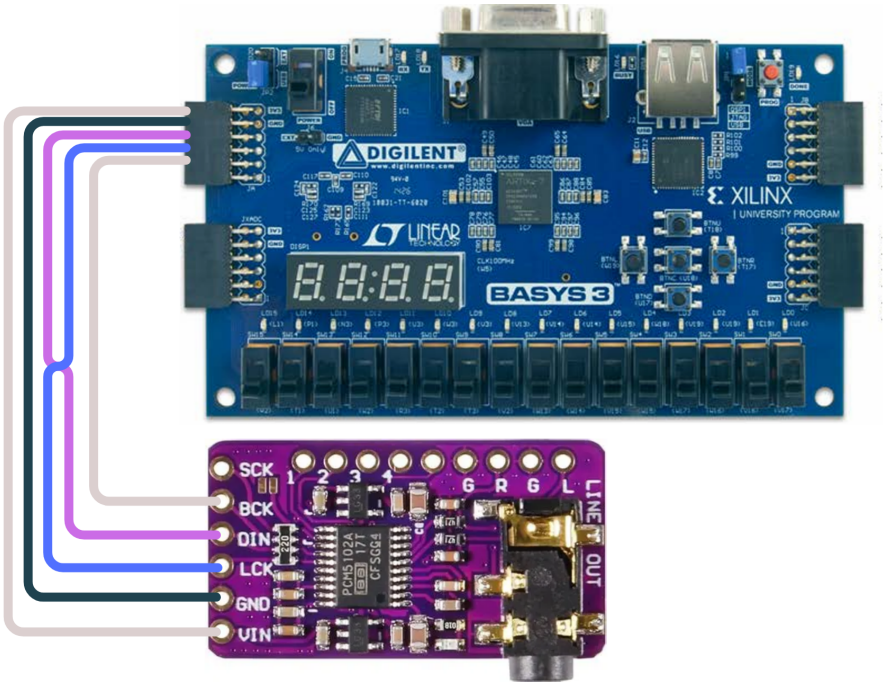
Kode IV.21. Dummy logic untuk sinyal AXI yang tidak dipakai.

4.1.11 Implementasi Integrasi pada Vivado Block Design

Integrasi ke Vivado dilakukan dengan mempertahankan nama modul `i2s` pada wrapper, menggunakan parameter lebar bus standar, dan menghubungkan core ke AXI interconnect serta pin keluar PMOD/DAC. Pada level Block Design, IP ini menerima clock sistem dari prosesor dan memancarkan clock I2S langsung dari internal NCO.



Gambar 4.5. Rangkaian sistem yang digunakan pada Block Design Vivado.



Gambar 4.6. Koneksi pin PMOD FPGA ke modul DAC PCM5102A.

Gambar 4.5 dan Gambar 4.6 menjelaskan konteks integrasi hardware. Reset sistem memastikan IP berada pada keadaan stabil saat inisialisasi, sedangkan wiring PMOD menunjukkan hubungan fisik antara keluaran I2S dan DAC audio.

4.2 Implementasi Firmware

Firmware `freshi2s/sw/helloworld.c` merupakan bagian yang menghasilkan sampel audio uji dan mengendalikan IP I2S melalui register memory-mapped. Di implementasi akhir ini, firmware tidak hanya menulis data, tetapi juga menyinkronkan penulisan sampel dengan status latch hardware dan menyediakan antarmuka UART untuk mengubah parameter runtime.

4.2.1 Struktur Program

Struktur program dimulai dari definisi base address, register offset, macro bitfield, variabel state global, helper function, handler UART, dan loop utama.

```
#if defined(XPAR_I2S_DDS_0_BASEADDR)
#define I2S_BASE XPAR_I2S_DDS_0_BASEADDR
#elif defined(XPAR_I2S_0_BASEADDR)
#define I2S_BASE XPAR_I2S_0_BASEADDR
#else
#error "Cannot find I2S base address in xparameters.h"
#endif

#define DATA_LEFT   (I2S_BASE + 0x00U)
#define DATA_RIGHT  (I2S_BASE + 0x04U)
#define CONTROL     (I2S_BASE + 0x08U)
#define STATUS_REG  (I2S_BASE + 0x0CU)
```

Kode IV.22. Definisi base address dan register offset firmware.

Makro tersebut harus identik dengan register map RTL agar transaksi AXI4-Lite yang dilakukan firmware dipetakan ke register yang benar. Pendekatan ini membuat firmware tetap fleksibel terhadap perubahan nama instance IP pada `xparameters.h`.

```
#define CTRL_ENABLE     (1U << 0)
#define CTRL_MUTE       (1U << 1)
#define CTRL_WIDTH(w)   (((uint32_t)((w) >= 32U ? 0U : ((w) & 0x1FU))) << 2U)
#define CTRL_RATE(hz)   (((uint32_t)(hz) & 0x000FFFFFU) << 7U)
```

Kode IV.23. Bitfield kontrol firmware yang sesuai dengan RTL.

Makro `CTRL_WIDTH(w)` mengimplementasikan konvensi yang sama dengan RTL, yaitu nilai 0 pada field lebar sampel berarti 32 bit. Hal ini penting agar firmware dan RTL menafsirkan register `CONTROL` secara konsisten.

4.2.2 Inisialisasi Periferal

Program utama melakukan inisialisasi platform, validasi nilai awal, penulisan sampel nol, dan penerapan kontrol awal.

```
int main(void)
{
    init_platform();
    xil_printf("\r\n=== I2S Software DDS Source ===\r\n");

```

```
    g_sample_rate_hz = clamp_sample_rate(g_sample_rate_hz);
    g_width = clamp_width(g_width);
    write_channels(0U);
    apply_control();
    g_last_frame_bit = frame_status_bit();
    print_state();
    while (1) {
        uint32_t frame_bit = frame_status_bit();

        if (g_enable && frame_bit != g_last_frame_bit) {
            g_last_frame_bit = frame_bit;
            stream_software_dds_sample();
        }

        if (!XUartLite_IsReceiveEmpty(STDIN_BASEADDRESS))
            handle_key((uint8_t)XUartLite_RecvByte(STDIN_BASEADDRESS));
    }

    cleanup_platform();
    return 0;
```

Kode IV.24. Urutan inisialisasi pada fungsi `main()`.

Urutan tersebut menunjukkan bahwa firmware memastikan konfigurasi aman sebelum streaming dimulai. Sampel data ditulis nol terlebih dahulu, lalu kontrol diterapkan, dan status frame awal dibaca agar sinkronisasi berikutnya memiliki referensi yang benar.

4.2.3 Konfigurasi Register I2S

Firmware membentuk sebuah control word dari state global, lalu menulisnya ke register `CONTROL`. Fungsi ini menjadi titik utama interaksi firmware dengan IP.

```
static uint32_t control_word(void)
{
    uint32_t ctrl = 0U;
    if (g_enable)
        ctrl |= CTRL_ENABLE;
    if (g_mute)
        ctrl |= CTRL_MUTE;
    ctrl |= CTRL_WIDTH(g_width);
    ctrl |= CTRL_RATE(g_sample_rate_hz);
    return ctrl;
}
```

```
12
13 static void apply_control(void)
14 {
15     uint32_t ctrl = control_word();
16     Xil_Out32(CONTROL, ctrl);
17     xil_printf("   CTRL = 0x%08lX\r\n", (unsigned long)ctrl);
18 }
```

Kode IV.25. Pembentukan dan penulisan control word.

Penulisan register ini tidak hanya mengaktifkan output, tetapi juga mengubah sample rate, mute, dan lebar sampel pada RTL. Dengan demikian, satu transaksi AXI4-Lite cukup untuk memperbarui seluruh mode operasional I2S.

```
1 static uint32_t frame_status_bit(void)
2 {
3     return Xil_In32(STATUS_REG) & 1U;
4 }
```

Kode IV.26. Pembacaan status latch dari register `STATUS`.

Bit ini dipakai sebagai sinyal sinkronisasi sederhana antara software dan hardware. Ketika bit berubah, berarti RTL sudah masuk awal frame stereo baru dan siap menerima sampel berikutnya.

4.2.4 Pembangkitan Data Audio

Firmware menghasilkan sampel audio menggunakan software DDS. Tabel sinus 32 titik dipakai agar pembangkitan tetap ringan tanpa floating point berat.

```
1 static const int16_t SINE[32] = {
2         0,   6393,  12539,  18204,  23170,  27245,  30273,  32137,
3     32767,  32137,  30273,  27245,  23170,  18204,  12539,   6393,
4         0,  −6393, −12539, −18204, −23170, −27245, −30273, −32137,
5     −32767, −32137, −30273, −27245, −23170, −18204, −12539,  −6393
6 };
```

Kode IV.27. Tabel sinus 32 titik yang dipakai untuk software DDS.

Sebelum ditulis ke register audio, sampel disesuaikan dengan lebar bit yang sedang aktif.

```
1 static uint32_t pack_sample(int16_t sample)
2 {
3     int32_t value = (int32_t)sample;
```

```
    if (g_width >= 32U)
        value <<= 16;
    else if (g_width > 16U)
        value <<= (g_width - 16U);
    else if (g_width < 16U)
        value >>= (16U - g_width);

    return (uint32_t)value;
}
```

Kode IV.28. Penyesuaian lebar sampel sebelum ditulis ke register data.

Fungsi `write_channels` kemudian menulis sampel ke `DATA_LEFT` dan `DATA_RIGHT`. Jika `ENABLE` atau `MUTE` aktif, firmware mengirim nol. Jika routing kanal diubah, firmware dapat mengirim data hanya ke kanal kiri atau kanan.

```
static void write_channels(uint32_t sample)
{
    uint32_t left = sample;
    uint32_t right = sample;

    if (!g_enable || g_mute) {
        left = 0U;
        right = 0U;
    } else {
        if (g_channel == CH_RIGHT)
            left = 0U;
        if (g_channel == CH_LEFT)
            right = 0U;
    }

    Xil_Out32(DATA_LEFT, left);
    Xil_Out32(DATA_RIGHT, right);
}
```

Kode IV.29. Penulisan sampel ke register kanal kiri dan kanan.

Penghasilan sampel nada dilakukan oleh software DDS. Frekuensi nada dihitung dari perbandingan frekuensi target terhadap frekuensi sampling aktif, kemudian phase accumulator diperbarui setiap frame.

```
static void stream_software_dds_sample(void)
{
```

```
    uint32_t step = (uint32_t)(((uint64_t)g_tone_freq_hz << 32) / g_sample_rate_hz);
    int16_t sample = g_square
                ? (g_phase & 0x80000000U ? (int16_t)AMPLITUDE : (int16_t)-AMPLITUDE)
                : (int16_t)(((int32_t)SINE[g_phase >> 27] * AMPLITUDE) / 32767);

    write_channels(pack_sample(sample));
    g_phase += step;
}
```

Kode IV.30. Pembangkitan sampel audio dengan software DDS.

Teknik ini dipilih karena sederhana, deterministik, dan cukup presisi untuk pengujian audio. Pada tingkat implementasi sistem, firmware menghasilkan konten audio, sedangkan RTL hanya mengurus timing serialisasinya.

4.2.5 Runtime Control Flow

Loop utama firmware membaca status frame dari hardware, menulis sampel baru ketika bit toggle berubah, dan secara paralel memproses input UART.

```
while (1) {
    uint32_t frame_bit = frame_status_bit();

    if (g_enable && frame_bit != g_last_frame_bit) {
        g_last_frame_bit = frame_bit;
        stream_software_dds_sample();
    }

    if (!XUartLite_IsReceiveEmpty(STDIN_BASEADDRESS))
        handle_key((uint8_t)XUartLite_RecvByte(STDIN_BASEADDRESS));
}
```

Kode IV.31. Alur runtime utama firmware.

Urutan ini menunjukkan bahwa firmware tidak bergantung pada `usleep()` atau delay kasar. Penulisan sampel disinkronkan dengan hardware melalui polling status frame, sehingga laju update mengikuti boundary frame I2S.

```
static void handle_key(uint8_t ch)
{
    uint8_t lower = (uint8_t)tolower((int)ch);
    xil_printf("%c\r\n", (char)ch);

    switch (lower) {
```

```
        case 's': set_wave_square(!g_square); break;
        case '4': set_sample_rate(48000U); break;
        case '5': set_sample_rate(44100U); break;
        case '6': set_sample_rate(96000U); break;
        case 'w': set_width(16U); break;
        case 'x': set_width(24U); break;
        case 'y': set_width(32U); break;
        case 'e': set_enable(!g_enable); break;
        case 'm': set_mute(!g_mute); break;
        case 'l': set_channel(CH_LEFT); break;
        case 'r': set_channel(CH_RIGHT); break;
        case 'b': set_channel(CH_BOTH); break;
        case '?': print_state(); break;
        default: break;
    }
}
```

Kode IV.32. Handler UART untuk kontrol runtime.

Fungsi setter yang dipanggil dari handler ini memperbarui state firmware, menulis ulang register kontrol, dan memastikan nilai yang dikirim ke hardware sudah divalidasi.

```
static void set_sample_rate(uint32_t hz)
{
    g_sample_rate_hz = clamp_sample_rate(hz);
    xil_printf(">> %-14s = %lu Hz\r\n", "SAMPLE RATE", (unsigned long)g_sample_rate_hz);
    apply_control();
    g_last_frame_bit = frame_status_bit();
}

static void set_enable(uint8_t value)
{
    g_enable = value ? 1U : 0U;
    xil_printf(">> %-14s = %u\r\n", "ENABLE", (unsigned)g_enable);
    apply_control();
    if (!g_enable)
        write_channels(0U);
    g_last_frame_bit = frame_status_bit();
}
```

Kode IV.33. Setter sample rate dan enable yang terhubung ke AXI4-Lite.

4.2.6 Pengujian dan Validasi

Pengujian firmware dilakukan melalui antarmuka UART dan observasi register serta keluaran audio. Program menampilkan status runtime ke terminal agar konfigurasi yang aktif dapat diverifikasi secara langsung.

```
static void print_state(void)
{
    xil_printf("\r\n==============================\r\n");
    xil_printf("  I2S Software DDS Test\r\n");
    xil_printf("==============================\r\n");
    xil_printf(">> %-14s = %u\r\n", "ENABLE", (unsigned)g_enable);
    xil_printf(">> %-14s = %s\r\n", "MUTE", g_mute ? "1 (silence)" : "0 (audio)");
    xil_printf(">> %-14s = %s\r\n", "WAVE", g_square ? "Square" : "Sine");
    xil_printf(">> %-14s = %u Hz\r\n", "TONE FREQ", (unsigned)g_tone_freq_hz);
    xil_printf(">> %-14s = %lu Hz\r\n", "SAMPLE RATE", (unsigned long)g_sample_rate_hz);
    xil_printf(">> %-14s = %u bit\r\n", "SAMPLE WIDTH", (unsigned)g_width);
    xil_printf(" L/R/B  left/right/both channels\r\n");
    xil_printf(" ?      show this menu\r\n");
    xil_printf("==============================\r\n");
}
```

Kode IV.34. Output status runtime firmware untuk debugging.

Pencetakan nilai `CTRL` setelah penulisan register juga memudahkan validasi bitfield terhadap register map RTL.

```
static void apply_control(void)
{
    uint32_t ctrl = control_word();
    Xil_Out32(CONTROL, ctrl);
    xil_printf("   CTRL = 0x%08lX\r\n", (unsigned long)ctrl);
}
```

Kode IV.35. Pengiriman control word ke hardware dan echo ke UART.

Pengujian yang dilakukan dengan beberapa kombinasi sample rate, lebar sampel, mute, dan routing kanal menunjukkan bahwa hubungan antara firmware dan RTL berjalan sesuai desain. Pada saat firmware menulis sampel, RTL melatch sampel di awal frame berikutnya, lalu toggle status dibaca kembali oleh firmware sebagai penanda bahwa data sebelumnya telah dikonsumsi.

4.3 Ringkasan Implementasi

Realisasi akhir IP I2S dibangun sebagai periferal AXI4-Lite empat register dengan clock generator berbasis NCO, serializer Philips I2S, mekanisme latch sampel, enable dan mute, serta status toggle sebagai sinkronisasi firmware. Firmware pada `freshi2s/sw/helloworld.c` bertugas menghasilkan audio uji, menulis register kontrol dan data, serta menyediakan antarmuka UART untuk validasi runtime. Dengan pembagian tugas tersebut, sistem menjadi sederhana, deterministik, dan sesuai dengan source code yang benar-benar diimplementasikan.

BAB V
HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil verifikasi dan pengujian fungsional IP I2S Transmitter. Urutan pengembangan dimulai dari simulasi RTL menggunakan *testbench*, dilanjutkan dengan sintesis, implementasi, dan pembuatan bitstream setelah pemeriksaan fungsional dinyatakan lulus. Pengujian pada Basys3 (FPGA Artix-7 XC7A35T) dengan DAC PCM5102A kemudian digunakan sebagai validasi akhir pada perangkat keras, bukan sebagai tahap awal untuk mencari kesalahan RTL secara coba-coba.

5.1 Verifikasi RTL dengan Testbench

Sebelum IP dikemas dan diintegrasikan ke dalam *block design*, modul I2S diverifikasi menggunakan Vivado Simulator melalui *testbench* mandiri. Testbench memberikan clock sistem 100 MHz dan reset, menjalankan transaksi AXI4-Lite untuk menulis serta membaca register `DATA\_LEFT`, `DATA\_RIGHT`, `CONTROL`, dan `STATUS`, kemudian memeriksa respons `BRESP` dan `RRESP`. Dengan demikian, fungsi register, mekanisme handshaking AXI4-Lite, pembangkitan clock berbasis *phase accumulator*, serta proses serialisasi data I2S dapat diverifikasi tanpa ketergantungan pada firmware maupun board fisik.

Kode lengkap *testbench* yang digunakan pada simulasi disertakan pada Listing 1 di lampiran. Listing tersebut menunjukkan pembangkitan clock dan reset, koneksi modul I2S, transaksi baca-tulis AXI4-Lite, serta pemeriksaan otomatis terhadap data yang dibaca kembali dan sinyal keluaran I2S. Dengan demikian, hasil verifikasi pada bagian ini dapat direproduksi dari kode sumber yang dicantumkan.

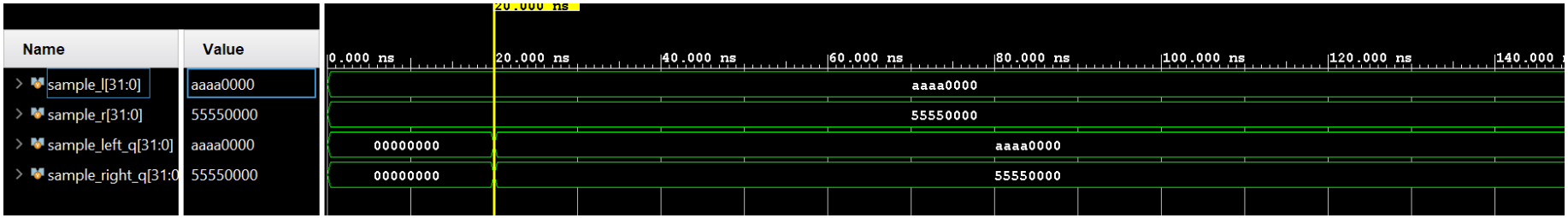
Vektor uji menggunakan sampel kiri dan kanan 24-bit yang berbeda. Setelah bit `ENABLE` diaktifkan pada register `CONTROL`, simulator memantau pembangkitan BCLK, perubahan WS, serta keluaran `i2s\_data` pada setiap tepi naik BCLK sebagaimana dilakukan oleh penerima I2S. Pemeriksaan meliputi kondisi reset, kondisi `ENABLE=0`, pembangkitan clock menggunakan *phase accumulator*, sinkronisasi frame stereo, urutan pengiriman bit MSB–LSB, *one-bit delay* sesuai spesifikasi Philips I2S, padding nol pada slot 32-bit, serta fungsi `MUTE`. Simulasi dinyatakan berhasil apabila seluruh pemeriksaan tersebut terpenuhi.

Hasil simulasi menjadi dasar sebelum proses sintesis dan implementasi. Setelah seluruh *timing constraints* terpenuhi, bitstream dan *hardware platform* diekspor ke Vitis untuk pengujian firmware. Selanjutnya dilakukan validasi pada perangkat keras menggunakan Integrated Logic Analyzer (ILA), osiloskop, dan pengujian audio pada DAC PCM5102A untuk memastikan perilaku yang telah diverifikasi melalui simulasi tetap

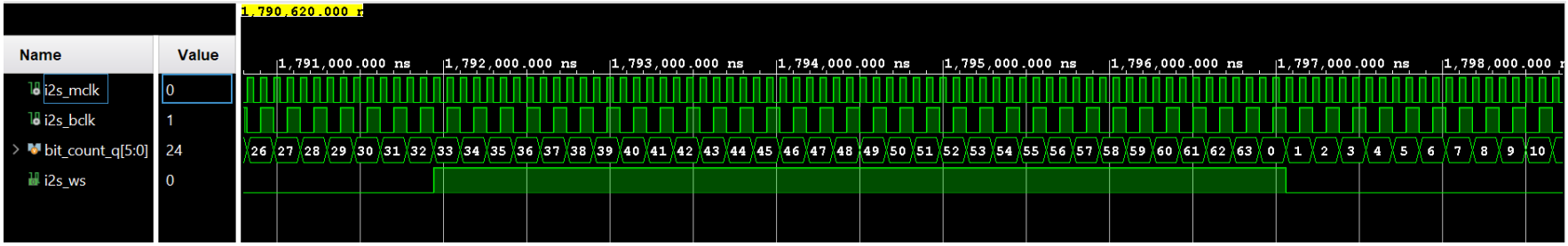
konsisten pada implementasi fisik.

5.2 Pengujian Timing dan Pembangkitan Clock I2S

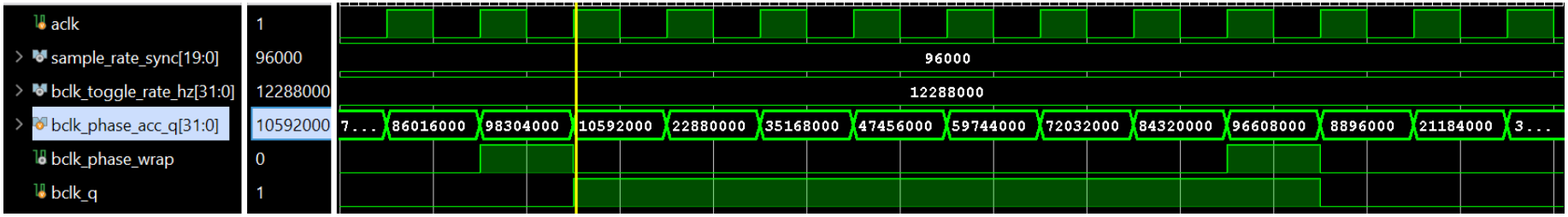
Verifikasi timing dilakukan dengan mengamati sinyal internal hasil simulasi RTL. Selain keluaran `i2s\_bclk`, `i2s\_ws`, dan `i2s\_data`, diamati pula sinyal internal `bclk\_phase\_acc\_q`, `bclk\_phase\_wrap`, `bit\_count\_q`, `ws\_q`, dan `data\_q`. Pengamatan terhadap sinyal-sinyal tersebut bertujuan untuk memverifikasi mekanisme pembangkitan clock menggunakan *phase accumulator*, sinkronisasi frame stereo, serta proses serialisasi data sesuai protokol Philips I2S.



(a) Timing sinyal BCLK, WS, dan DATA selama satu frame stereo (64 siklus BCLK).



(b) Perilaku penghitung bit (`bit_count_q`) dan sinkronisasi pergantian kanal melalui sinyal WS.



(c) Pembangkitan BCLK menggunakan *phase accumulator*. Accumulator bertambah sebesar `bclk_toggle_rate_hz` pada setiap siklus clock sistem dan menghasilkan sinyal `bclk_phase_wrap` yang men-*toggle* BCLK saat mencapai batas frekuensi clock sistem.

Gambar 5.1. Hasil simulasi internal IP I2S yang memperlihatkan pembangkitan clock berbasis *phase accumulator*, sinkronisasi frame stereo, dan serialisasi data sesuai spesifikasi Philips I2S.

Subgambar 5.1c memperlihatkan mekanisme pembangkitan BCLK menggunakan *phase accumulator*. Pada setiap tepi naik clock sistem 100 MHz, RTL menjumlahkan `bclk\_toggle\_rate\_hz` dengan nilai accumulator sebelumnya. Jika hasil penjumlahan mencapai atau melampaui modulus 100.000.000, sinyal `bclk\_phase\_wrap` aktif selama satu siklus, accumulator menyimpan nilai sisa setelah dikurangi modulus, dan BCLK melakukan satu toggle. Contoh urutan penjumlahan tersebut ditunjukkan pada Tabel 5.1.

Pada waveform, empat sinyal perlu dibedakan. Clock sistem menjadi sumbu waktu diskrit. Accumulator membentuk deret nilai yang meningkat lalu turun ke nilai sisa ketika terjadi wrap. Pulsa `bclk\_phase\_wrap` menandai peristiwa toggle, bukan satu periode penuh BCLK. Dua peristiwa toggle membentuk satu periode BCLK, dan 64 periode BCLK membentuk satu frame stereo sehingga WS kembali ke kanal berikutnya. Akumulator MCLK bekerja secara terpisah menggunakan `mclk\_toggle\_rate\_hz`; karena itu, waveform tidak perlu menunjukkan MCLK dan BCLK berubah pada tepi yang sama.

Untuk menghubungkan bentuk waveform dengan perhitungan numerik, Tabel 5.1 memberikan contoh $F_s = 96$ kHz. Pada konfigurasi ini, inkremen akumulator BCLK adalah $I_{\mathrm{BCLK}} = 128 \times 96.000 = 12.288.000$. Kolom nilai penjumlahan menampilkan hasil sebelum operasi modulo. Ketika hasil mencapai atau melewati $M = 100.000.000$, RTL menghasilkan satu pulsa `bclk\_phase\_wrap`, melakukan satu toggle pada BCLK, lalu menyimpan sisa setelah dikurangi M.

Tabel 5.1. Contoh penjumlahan pada *phase accumulator* BCLK untuk $F_s = 96$ kHz.

Siklus	**Nilai penjumlahan**	**Keterangan**
0	0	Nilai awal
1	12 288 000	
2	24 576 000	
3	36 864 000	
4	49 152 000	
5	61 440 000	
6	73 728 000	
7	86 016 000	
8	98 304 000	
9	110 592 000	$\geq M$: toggle 1; sisa 10 592 000
10	10 592 000	Nilai setelah wrap
11	22 880 000	
12	35 168 000	
13	47 456 000	
14	59 744 000	
15	72 032 000	
16	84 320 000	
17	96 608 000	
18	108 896 000	$\geq M$: toggle 2; sisa 8 896 000

Sementara itu, Subgambar 5.1a dan Subgambar 5.1b mengonfirmasi karakteristik timing Philips I2S [1] sebagai berikut.

1. Satu frame stereo terdiri atas tepat 64 siklus BCLK, masing-masing 32 bit untuk kanal kiri dan 32 bit untuk kanal kanan.
2. Pergantian sinyal WS terjadi tepat pada batas antara kanal kiri dan kanal kanan.
3. Bit data pertama dikirim satu siklus BCLK setelah perubahan WS (*one-bit delay*) sesuai spesifikasi Philips I2S.
4. Penghitung bit (`bit_count_q`) kembali ke nol pada awal setiap frame stereo sehingga sinkronisasi kanal kiri selalu terjaga.

5.3 Analisis Frekuensi Sampling

Frekuensi sampling diprogram melalui field `SAMPLE\_RATE\_HZ` pada register `CONTROL`. Pada clock sistem 100 MHz, DDS menambahkan nilai $128 \times F_s$ ke akumulator BCLK pada setiap siklus. Saat jumlahnya mencapai 100.000.000, akumulator melakukan *wrap-around* dan BCLK berubah keadaan. Karena satu frame stereo memiliki 64 pulsa BCLK dan BCLK berubah keadaan dua kali dalam satu periode, sinyal WS memiliki frekuensi target F_s.

Validasi numerik dilakukan pada *testbench*, bukan dengan menginterpretasikan gambar osiloskop. Setelah testbench menulis nilai target melalui AXI4-Lite, ia mencatat waktu 16 periode WS penuh, menghitung $f_{\mathrm{WS}} = 16/\Delta t$, lalu membandingkannya dengan nilai yang diprogram. Konfigurasi yang diperiksa meliputi 44,1 kHz, 48 kHz, 88,2 kHz, 96 kHz, serta nilai non-standar 10 kHz dan 12,345 kHz. Simulasi gagal apabila galat relatif melebihi 0,1%.

Pendekatan ini menghasilkan bukti yang dapat diulang dari keluaran simulator dan tidak mengklaim angka hasil pengukuran perangkat keras yang tidak lagi tersedia. Capture osiloskop dan ILA tetap digunakan pada tahap integrasi untuk menunjukkan keberadaan dan relasi timing BCLK, WS, serta DATA pada board, sedangkan nilai frekuensi kuantitatif dilaporkan dari checker testbench.

5.4 Validasi Audio pada Berbagai Frekuensi Sampling

Pengujian fungsional dilakukan dengan memutar nada sinus menggunakan firmware berbasis DDS perangkat lunak (*software DDS*) pada empat mode F_s standar secara bergantian. Peralihan F_s dilakukan melalui field `SAMPLE\_RATE\_HZ` pada register `CONTROL` tanpa reset hardware.

Pada konfigurasi yang diuji, DAC PCM5102A menerima keluaran I2S setelah perubahan F_s dan tidak terdengar distorsi *pitch* maupun *audio pop* pada headset. Pengamatan ini bersifat kualitatif; tesis tidak mengklaim pengukuran waktu *lock* atau pembuktian bahwa setiap perubahan `ENABLE` selalu terjadi pada batas frame.

5.5 Identifikasi dan Perbaikan Bug Firmware

Selama tahap integrasi perangkat keras ditemukan dua bug pada firmware (`freshi2s/sw/helloworld.c`) yang memengaruhi kualitas audio dan akurasi frekuensi nada. Keduanya berhasil diidentifikasi dan diperbaiki sebelum pengambilan data final.

Pada rancangan ini, *status polling* berarti firmware membaca bit 0 pada register `STATUS` secara berulang dan membandingkan nilainya dengan nilai pembacaan sebelumnya. Perubahan nilai menandai dimulainya frame stereo baru setelah hardware

melatch sampel sebelumnya. Firmware kemudian menulis pasangan sampel berikutnya. Mekanisme ini menjaga sinkronisasi antara perangkat lunak dan serializer tanpa FIFO atau interupsi.

5.5.1 Bug 1: Masker Bit Merusak Bit Tanda Sampel

Gejala. Gelombang audio terdengar terdistorsi dengan amplitudo asimetris antara setengah siklus positif dan negatif.

Penyebab. Operasi masker bitwise pada sampel `int16_t` dilakukan setelah pengeseran kiri ke posisi bit slot 24-bit. Masker berbasis lebar bit (`(1 « g_width) − 1`) memotong bit tanda MSB hasil pengeseran, sehingga sampel negatif kehilangan bit tandanya dan diinterpretasikan sebagai nilai besar positif oleh DAC.

Solusi. Masker dihapus. Sampel `int16_t` di-*cast* ke `int32_t` sebelum pengeseran agar ekstensi tanda terjaga, kemudian dikirim langsung ke register data tanpa pemotongan bit tanda.

5.5.2 Bug 2: Timing Loop Berbasis `usleep()`

Gejala. Frekuensi nada keluaran tidak stabil (*jitter* tinggi) dan konsisten lebih rendah dari target. Pada mode 48 kHz firmware yang menggunakan `usleep(20)` seharusnya menghasilkan rate 50.000 sampel/s, namun overhead fungsi `usleep()` menyebabkan rate aktual lebih rendah sehingga frekuensi nada ikut turun.

Penyebab. Fungsi `usleep()` pada MicroBlaze bare-metal memiliki resolusi dan akurasi yang bergantung pada implementasi BSP, dan tidak tersinkronisasi dengan transisi frame hardware I2S.

Solusi. Penulisan sampel disinkronkan dengan sinyal latch hardware menggunakan *status-bit polling* pada bit `STATUS[0]` (Listing V.1). Firmware membaca status secara nonblokir, mengirim sampel ketika bit berubah, lalu tetap memeriksa masukan UART pada loop yang sama.

```
while (1) {
  uint32_t frame_bit = frame_status_bit();
  if (g_enable && frame_bit != g_last_frame_bit) {
    g_last_frame_bit = frame_bit;
    stream_software_dds_sample();
  }
  if (!XUartLite_IsReceiveEmpty(STDIN_BASEADDRESS))
    handle_key((uint8_t)XUartLite_RecvByte(STDIN_BASEADDRESS));
}
```

Kode V.1. Sinkronisasi nonblokir penulisan sampel menggunakan status-bit polling.

5.5.3 Variabel dan Fungsi Kunci Firmware

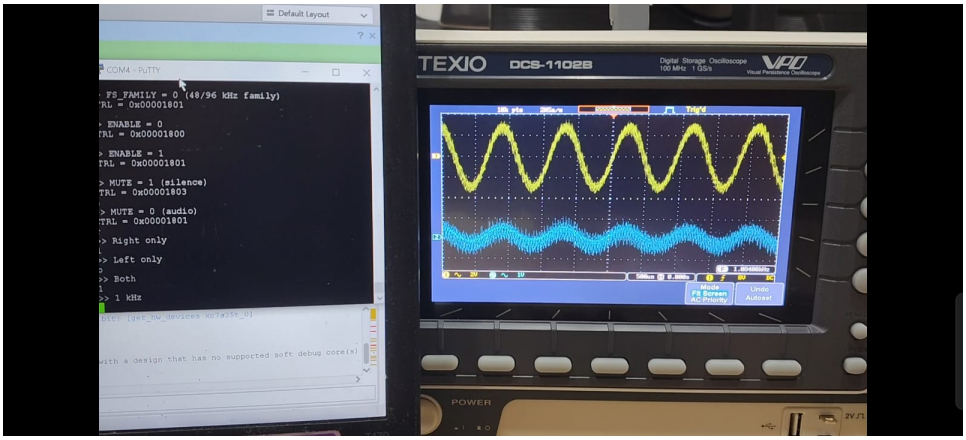
Fungsionalitas utama firmware dikendalikan melalui beberapa komponen berikut:

- `g\_phase` — akumulator fase 32-bit untuk DDS perangkat lunak; bertambah sebesar *phase step* setiap sampel.
- `g\_sample\_rate\_hz` — menyimpan nilai F_s aktif; digunakan untuk menghitung *phase step* agar akurasi frekuensi nada tidak bergantung pada mode F_s yang dipilih.
- `frame\_status\_bit()` — membaca bit[0] register `STATUS` yang berubah setiap kali hardware melakukan latch sampel stereo; digunakan sebagai sinyal sinkronisasi.
- `pack\_sample()` — melakukan penskalaan dan pengeseran sampel sesuai lebar bit yang aktif tanpa memotong bit tanda.
- `apply\_control()` — mengemas seluruh bit konfigurasi ke dalam satu nilai 32-bit dan menulis ke register `CONTROL` dalam satu transaksi AXI4-Lite.

5.6 Verifikasi Setelah Perbaikan Firmware

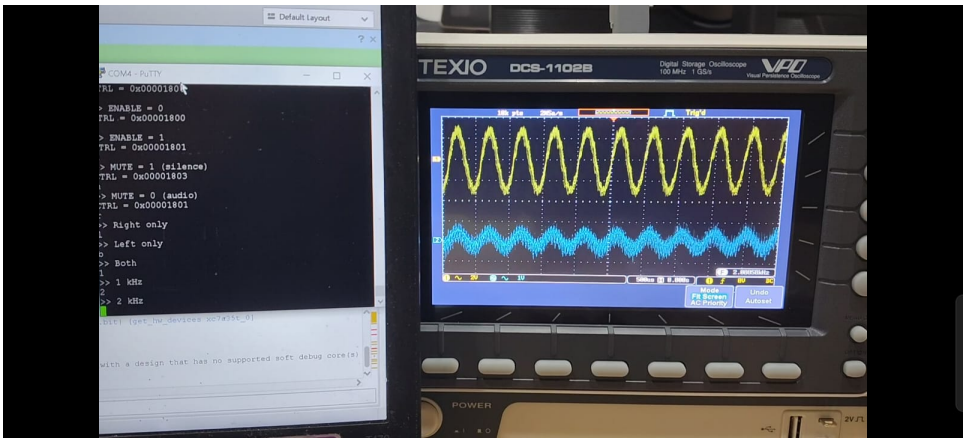
5.6.1 Akurasi Frekuensi Nada

Nada sinus diuji pada tiga frekuensi target (1 kHz, 2 kHz, dan 5 kHz) dengan mode F_s 48 kHz sebagai basis pengukuran. Gambar 5.2–5.4 menunjukkan keluaran audio analog yang diukur pada LOUT DAC PCM5102A menggunakan osiloskop.

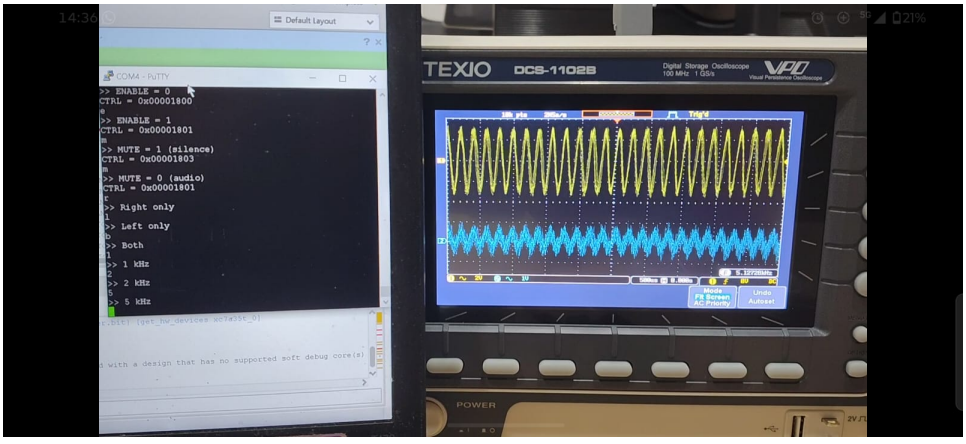


Gambar 5.2. Keluaran osiloskop: nada sinus 1 kHz pada $F_s = 48$ kHz.

Bentuk gelombang sinus pada ketiga frekuensi menunjukkan amplitudo yang stabil dan simetris tanpa distorsi yang terlihat, mengonfirmasi bahwa perbaikan bug bit tanda telah menyelesaikan masalah asimetri amplitudo yang sebelumnya ditemukan.



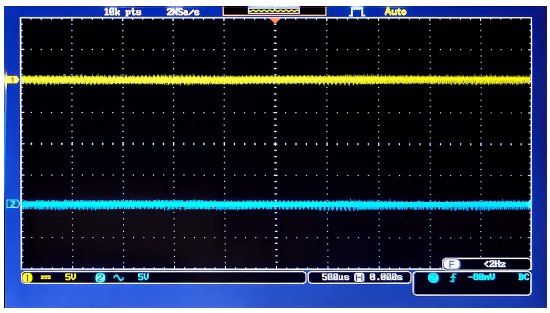
Gambar 5.3. Keluaran osiloskop: nada sinus 2 kHz pada $F_s = 48$ kHz.



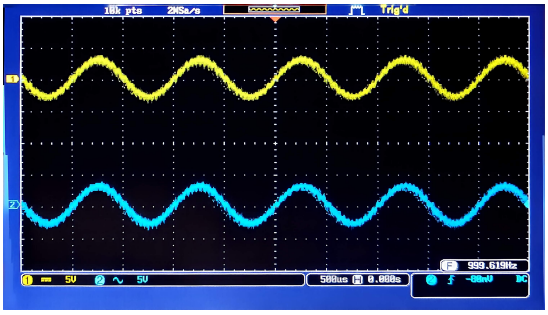
Gambar 5.4. Keluaran osiloskop: nada sinus 5 kHz pada $F_s = 48$ kHz.

5.6.2 Fungsionalitas Bit Kontrol

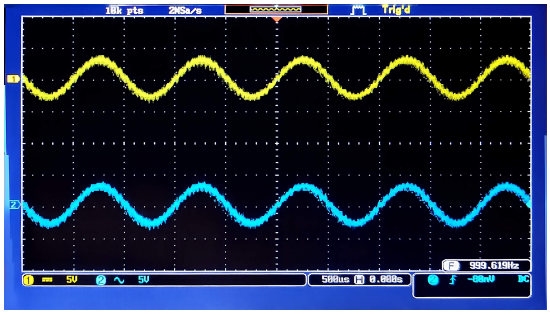
Pengujian bit `ENABLE` dan `MUTE` dilakukan dengan mengamati respons sinyal analog pada LOUT/ROUT (Gambar 5.5).



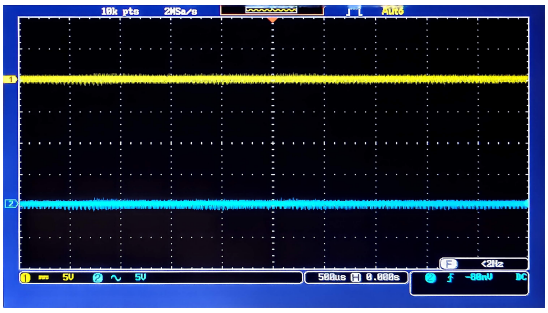
(a) `ENABLE` = 0: seluruh keluaran mati.



(b) `ENABLE` = 1: keluaran aktif.



(c) `MUTE` = 0: audio normal.



(d) `MUTE` = 1: audio senyap, clock tetap berjalan.

Gambar 5.5. Verifikasi fungsionalitas bit `ENABLE` dan `MUTE` pada keluaran audio analog.

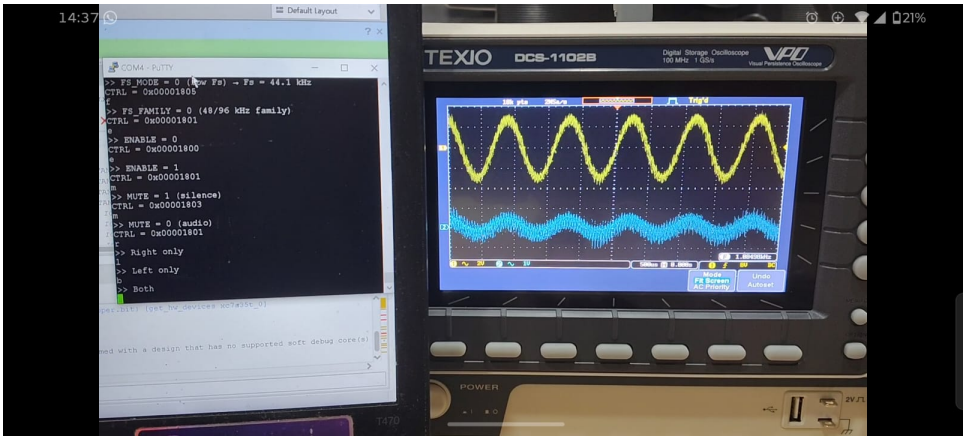
Hasil menunjukkan bahwa `ENABLE` = 0 menghentikan seluruh sinyal keluaran termasuk BCLK dan WS, sesuai perilaku yang dirancang dalam RTL. `MUTE` = 1 mematikan audio dengan memaksa jalur data ke nol, namun BCLK dan WS tetap berjalan sehingga DAC PCM5102A mempertahankan kondisi *locked* dan tidak memerlukan waktu *lock* ulang saat audio diaktifkan kembali.

Setelah perbaikan bug bit tanda, kanal kiri (L) dan kanan (R) menghasilkan bentuk gelombang sinus yang identik dan seimbang (Gambar 5.6).

5.7 Verifikasi Integrasi dengan Filter Digital

5.7.1 Modul RTL `i2s_filter_bridge`

IP `i2s_0` menerima sampel PCM melalui register AXI4-Lite dan menghasilkan aliran serial I2S, sedangkan modul `filter_coeff` memakai antarmuka paralel 16 bit berupa `x_in`, `valid`, dan `y_out`. Kedua antarmuka tersebut tidak dapat dihubungkan dengan sebuah net pada *block design*. Penelitian ini menambahkan modul `i2s_filter_bridge` pada keluaran serial IP I2S tanpa mengubah RTL IP I2S maupun RTL filter.



Gambar 5.6. Keluaran stereo setelah perbaikan: kanal L dan R simetris dan identik.

Pada tepi naik BCLK, bridge mendeteksi transisi WS, membuang satu bit tunda sesuai format Philips I2S, lalu menggeser 16 bit payload ke register sampel. Implementasi terbaru hanya memproses slot kiri melalui satu instans `filter_coeff`. Sinyal `right_valid` dan jalur sampel kanan dipertahankan pada antarmuka untuk kompatibilitas, tetapi tidak digunakan pada Block Design final.

Bridge menyimpan keluaran filter dan mengirimkannya pada frame berikutnya melalui `i2s_data_out`. Nilai hasil filter mono tersebut dikirim pada kedua slot I2S. Proses serialisasi memperbarui SDATA pada tepi turun BCLK agar DAC membaca nilai yang stabil pada tepi naik. Jalur MCLK, BCLK, dan WS menggunakan koneksi tembus; bridge hanya mengganti SDATA dengan data hasil filter. Konsekuensinya, integrasi menambah latensi satu frame stereo tanpa mengubah frekuensi sampling atau relasi fase antarsinyal I2S.

Tiga switch pemilih koefisien berasal dari pin FPGA yang asinkron terhadap clock sistem. Dua tingkat flip-flop pada bridge menyinkronkan nilai switch sebelum filter membacanya. Listing V.2 memperlihatkan antarmuka dan bagian utama jalur data bridge.

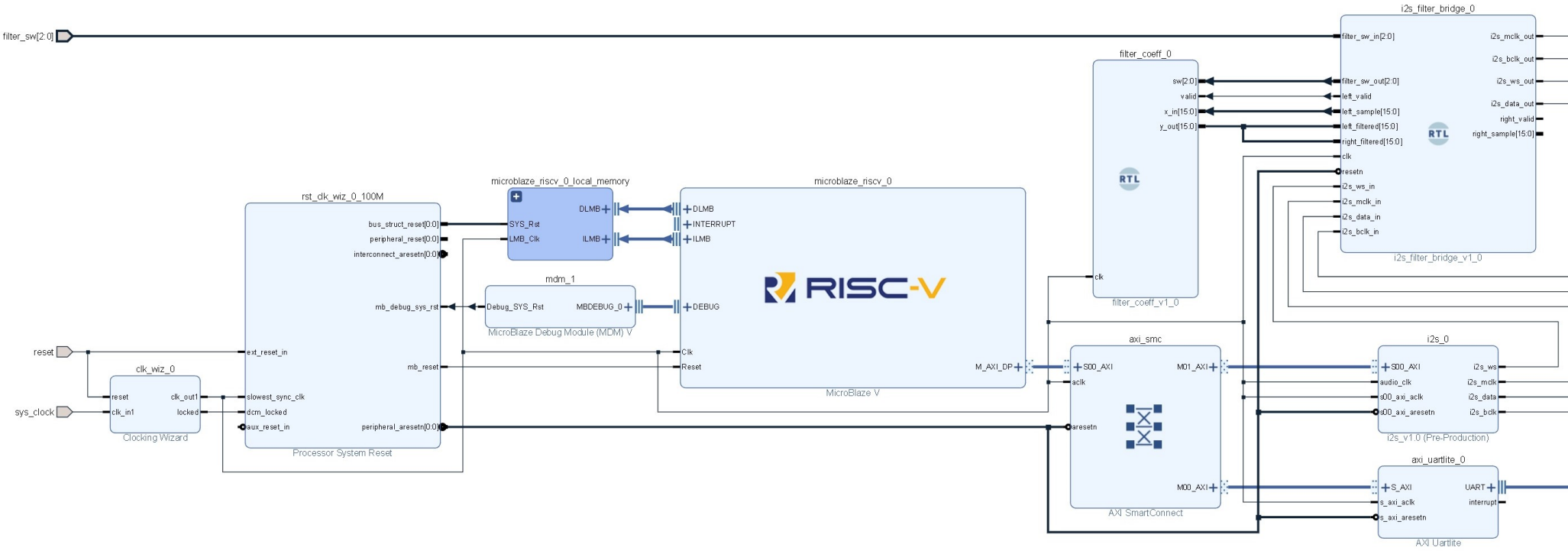
```
module i2s_filter_bridge (
    input  wire              clk,
    input  wire              resetn,
    input  wire [2:0]          filter_sw_in,
    input  wire              i2s_bclk_in,
    input  wire              i2s_ws_in,
    input  wire              i2s_data_in,
    output reg               i2s_data_out,
    output reg               left_valid,
    output reg signed [15:0]  left_sample,
    input  wire signed [15:0] left_filtered
);
```

```
13
14 wire bclk_rise = i2s_bclk_in & ~bclk_d;
15 wire bclk_fall = ~i2s_bclk_in & bclk_d;
16
17 always @(posedge clk) begin
18     /* The final Block Design connects only left_valid and left_filtered. */
19     if (bclk_rise && !i2s_ws_in && rx_bit_pos == 6'd15) begin
20         left_sample <= {rx_shift[14:0], i2s_data_in};
21         left_valid  <= 1'b1;
22     end
23
24     if (filter_result_d2)
25         tx_sample <= left_filtered;
26
27     /* The filtered mono result is serialized into both I2S slots. */
28     if (bclk_fall) begin
29         if (tx_bit_pos < 6'd16)
30             i2s_data_out <= tx_sample[15-tx_bit_pos];
31         else
32             i2s_data_out <= 1'b0;
33     end
34 end
35 endmodule
```

Kode V.2. Bagian utama RTL `i2s_filter_bridge`.

Pengujian mengirim nada sinus 1 kHz dari firmware ke register data IP I2S. Pada desain final, pulsa `left_valid` memicu pemrosesan sampel dari slot kiri, sedangkan hasil filter mono dikirim ulang pada slot kiri dan kanan. BCLK dan WS pada sisi DAC tetap mengikuti keluaran IP I2S. Deskripsi ini membatasi klaim integrasi pada satu jalur filter dan tidak menyatakan bahwa kanal kiri dan kanan memiliki state biquad independen.

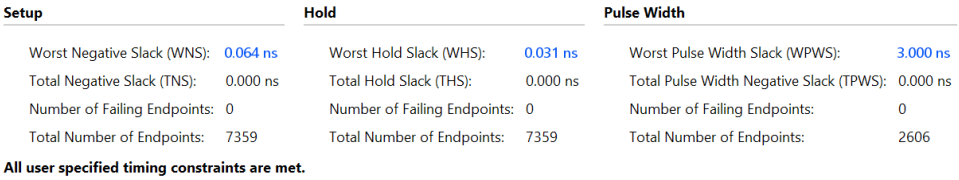
Integrasi IP I2S dengan satu blok filter IIR pada Block Design Vivado diperlihatkan pada Gambar 5.7. IP I2S (`i2s_0`) terhubung ke modul `i2s_filter_bridge` yang mendeserialisasi slot kiri, memprosesnya melalui filter IIR (`filter_coeff`), dan mengembalikan hasil mono yang dimirror ke jalur data serial menuju DAC.



Gambar 5.7. Integrasi IP I2S dengan blok filter IIR pada Vivado Block Design.

5.8 Analisis Penggunaan Sumber Daya dan Daya

Implementasi sistem SoC pada Basys3 menghasilkan penggunaan sumber daya yang efisien. Tabel 5.2 merangkum utilitas FPGA Artix-7 dari laporan implementasi Vivado, Gambar 5.8 menunjukkan ringkasan timing pasca-implementasi, dan Gambar 5.9 menyajikan visualisasi persentase utilitas resource serta estimasi konsumsi daya sistem.

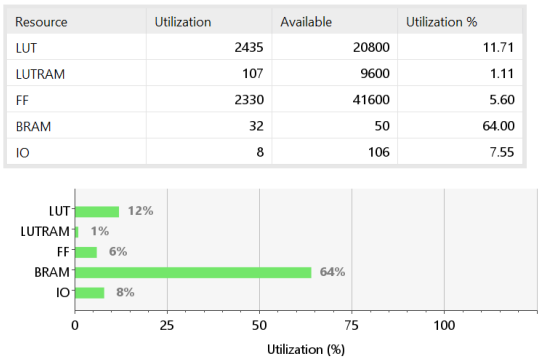


Gambar 5.8. Ringkasan timing desain pasca-implementasi dari Vivado Timing Analyzer.

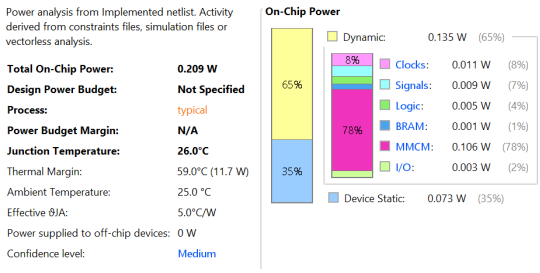
Laporan timing pasca-implementasi menunjukkan *Worst Negative Slack* (WNS) sebesar 0,064 ns, *Worst Hold Slack* (WHS) sebesar 0,031 ns, serta *Worst Pulse Width Slack* (WPWS) sebesar 3,000 ns. Nilai TNS, THS, dan TPWS sama dengan 0, sehingga seluruh constraint timing yang ditetapkan terpenuhi.

Tabel 5.2. Ringkasan penggunaan sumber daya FPGA Artix-7 XC7A35T (Basys3).

Resource	Utilized	Available	Utilization (%)
LUT	2.435	20.800	11,71
LUTRAM	107	9.600	1,11
FF	2.330	41.600	5,60
BRAM	32	50	64,00
IO	8	106	7,55
MMCM	1	5	20,00



(a) Persentase utilitas resource FPGA.



(b) Estimasi konsumsi daya sistem.

Gambar 5.9. Analisis implementasi fisik pada FPGA Basys3.

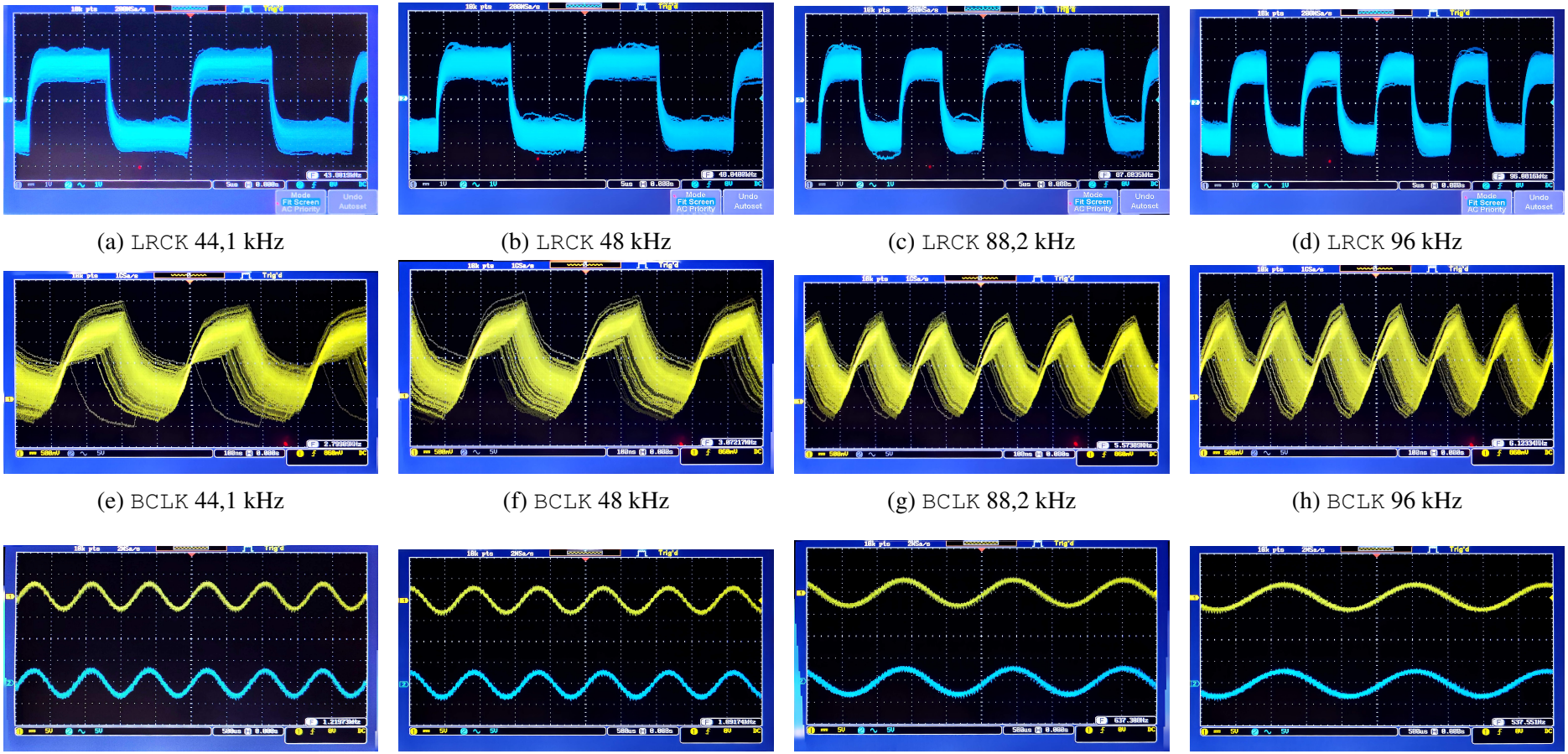
Penggunaan LUT sebesar 11,71% dan FF sebesar 5,60% menunjukkan bahwa sistem masih memiliki kapasitas logika untuk pengembangan lanjutan. Penggunaan BRAM

sebesar 64% terutama berasal dari memori program dan data pada sistem prosesor serta blok filter yang terintegrasi. Vivado Power Analyzer memperkirakan daya total sebesar 0,209 W, yang terdiri atas daya dinamis 0,135 W dan daya statis 0,073 W. Laporan tersebut menggunakan tingkat keyakinan *Medium* tanpa berkas aktivitas simulasi, sehingga angka daya diperlakukan sebagai estimasi implementasi, bukan hasil pengukuran langsung pada *board*.

5.9 Pembahasan terhadap Tujuan Penelitian

5.9.1 Perubahan Frekuensi Sampling

Pengujian pada empat mode F_s standar (44,1 kHz, 48 kHz, 88,2 kHz, dan 96 kHz) menunjukkan bahwa perubahan konfigurasi register `CONTROL` langsung memengaruhi timing BCLK dan WS secara *real-time* tanpa memerlukan reset hardware (Gambar 5.10).



(a) LRCK 44,1 kHz (b) LRCK 48 kHz (c) LRCK 88,2 kHz (d) LRCK 96 kHz

(e) BCLK 44,1 kHz (f) BCLK 48 kHz (g) BCLK 88,2 kHz (h) BCLK 96 kHz

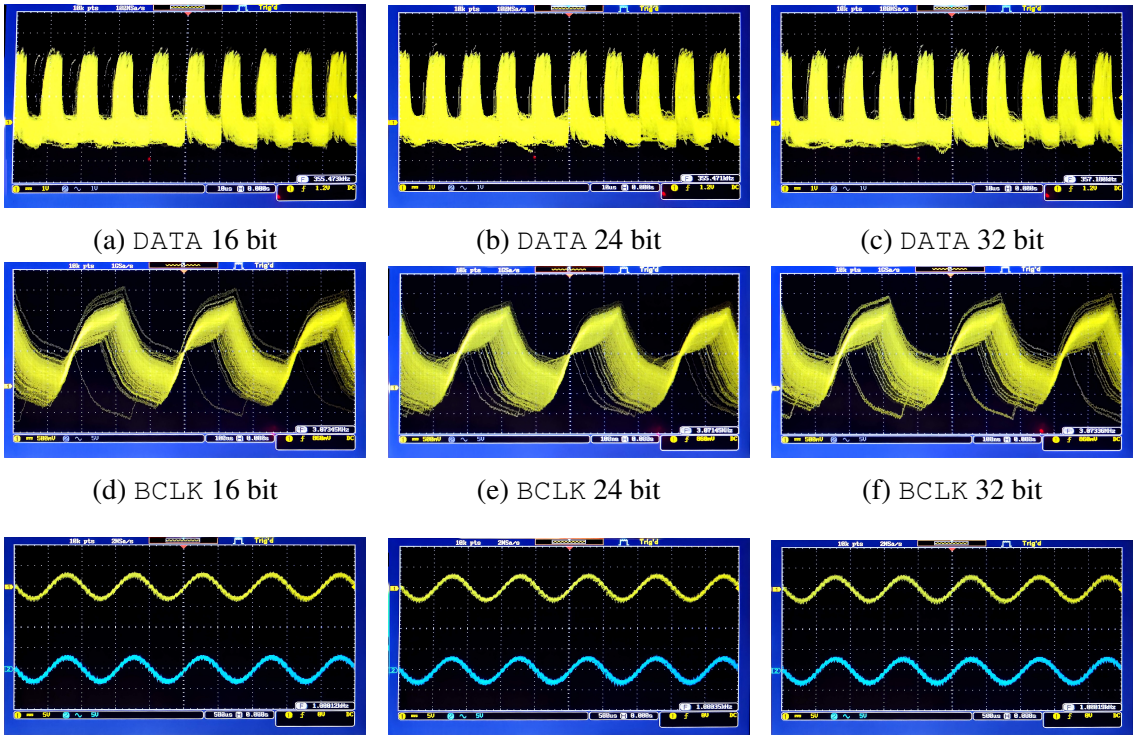
(i) LOUT/ROUT 44,1 kHz (j) LOUT/ROUT 48 kHz (k) LOUT/ROUT 88,2 kHz (l) LOUT/ROUT 96 kHz

Gambar 5.10. Perbandingan sinyal LRCK, BCLK, dan keluaran audio analog pada empat mode F_s standar.

Pada setiap mode, checker testbench memverifikasi frekuensi WS terhadap nilai F_s yang diprogram, sedangkan frekuensi BCLK mengikuti relasi $f_{\mathrm{BCLK}} = 64 \times F_s$ sebagaimana ditetapkan oleh protokol Philips I2S.

5.9.2 Variasi Lebar Data Sampel

Konfigurasi lebar bit (16, 24, dan 32 bit) mengubah letak bit valid yang dikirim di dalam slot 32-bit per kanal (Gambar 5.11). Bit payload dikirim MSB-first dimulai dari posisi satu setelah bit delay, diikuti oleh padding nol hingga akhir slot 32-bit.



(a) DATA 16 bit (b) DATA 24 bit (c) DATA 32 bit

(d) BCLK 16 bit (e) BCLK 24 bit (f) BCLK 32 bit

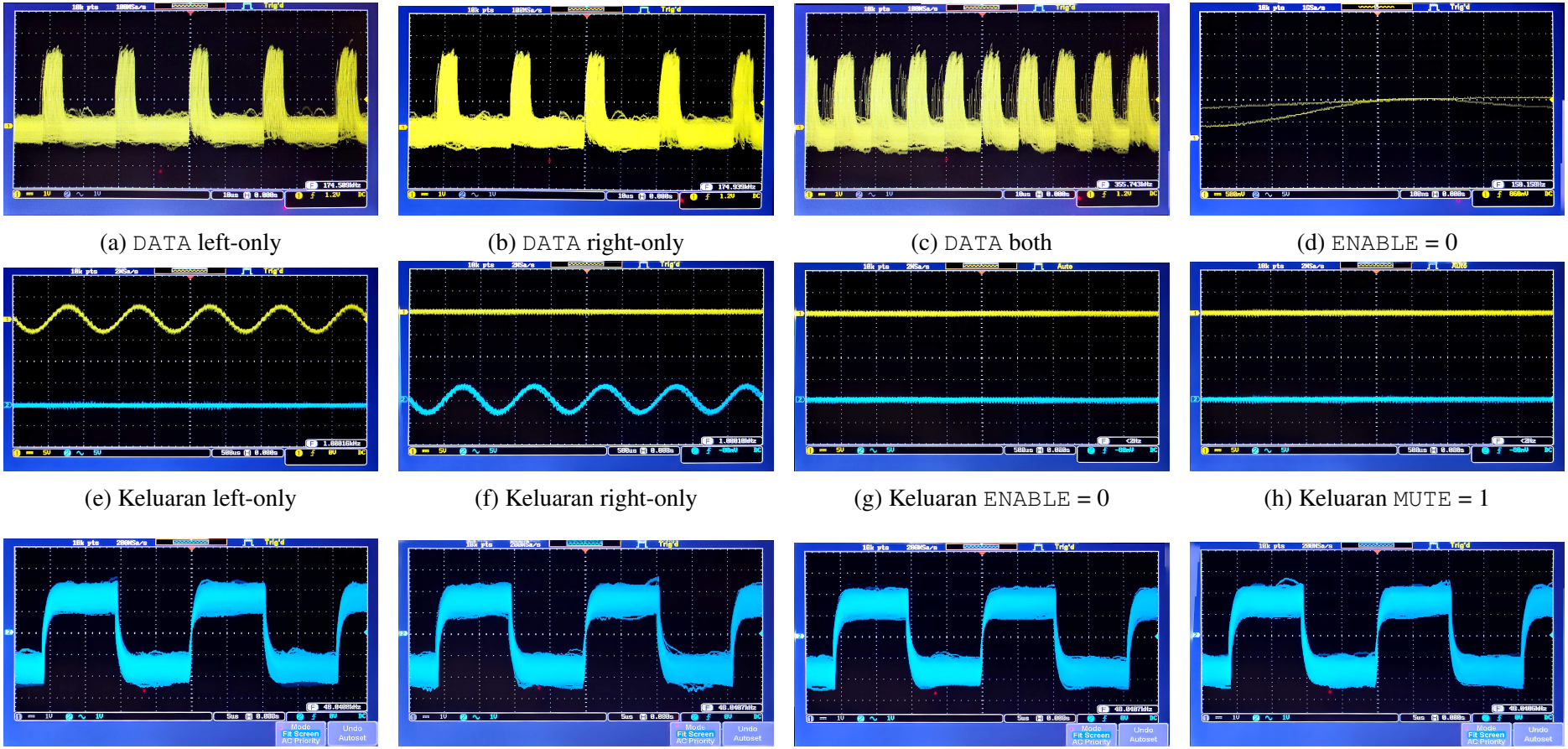
(g) Keluaran 16 bit (h) Keluaran 24 bit (i) Keluaran 32 bit

Gambar 5.11. Efek variasi lebar data sampel terhadap sinyal serial DATA dan keluaran audio analog.

Perubahan lebar bit tidak memengaruhi timing BCLK atau WS karena slot per kanal tetap 32 BCLK. Perbedaan yang terlihat pada sinyal DATA adalah jumlah bit aktif yang membawa data sebelum slot diisi padding nol, yang bertambah seiring peningkatan lebar bit. Pada keluaran analog, perbedaan lebar bit tidak terlihat secara visual pada frekuensi 1 kHz karena resolusi DAC PCM5102A sudah jauh melampaui perbedaan antara 16 dan 24 bit pada amplitudo pengujian ini.

5.9.3 Kontrol Kanal dan Status Mute

Pengujian rute kanal (left-only, right-only, both) dan status MUTE dikonfirmasi melalui pengamatan sinyal DATA, LRCK, dan keluaran analog (Gambar 5.12).



(a) DATA left-only (b) DATA right-only (c) DATA both (d) ENABLE = 0

(e) Keluaran left-only (f) Keluaran right-only (g) Keluaran ENABLE = 0 (h) Keluaran MUTE = 1

(i) LRCK left-only (j) LRCK right-only (k) LRCK both (l) LRCK saat MUTE = 1

Gambar 5.12. Validasi rute kanal, bit ENABLE, dan bit MUTE melalui sinyal digital dan keluaran analog.

Pada mode left-only, sinyal DATA aktif hanya saat WS = LOW (slot kiri) dan bernilai nol seluruhnya saat WS = HIGH (slot kanan). Kebalikannya terjadi pada mode right-only. Pada mode both, kedua slot membawa data identik. Perilaku ini sesuai dengan logika serialisasi pada fungsi `i2s_next_serial_bit` dalam RTL yang memilah sampel berdasarkan nilai `bit_count_q[5]`.

5.10 Perbandingan dengan Referensi Desain

Tabel 5.3 membandingkan fitur fungsional IP I2S Transmitter hasil penelitian ini dengan Xilinx LogiCORE I2S Transmitter [27] sebagai referensi standar industri.

Tabel 5.3. Perbandingan fitur IP I2S Transmitter.

Fitur	**Xilinx LogiCORE I2S**	**Custom IP (Penelitian)**
Jumlah register	> 15 register (0x00–0x64)	4 register (0x00–0x0C)
Antarmuka data	AXI4-Stream	AXI4-Lite (direct register write)
Konfigurasi F_s	Divider statis / PLL eksternal	NCO internal untuk frekuensi sampling integer dalam rentang RTL
Sinkronisasi	AXI-Stream backpressure / interupsi	Status-bit polling (latch toggle)
Jumlah kanal	2, 4, 6, atau 8	2 (stereo)
Lisensi	Proprietary (Xilinx)	RTL audio terbuka; integrasi tetap bergantung pada toolchain vendor
Kompleksitas	Tinggi (AES status, multi-channel, dsb.)	Rendah (minimalis, fokus fungsi inti)

Custom IP yang dirancang menawarkan kesederhanaan kontrol tanpa memerlukan DMA atau interkoneksi AXI4-Stream, dengan tetap memenuhi format transmisi audio stereo yang diuji. NCO internal menerima frekuensi sampling integer dalam rentang yang dibatasi RTL. Logika audio tidak mengandung primitif vendor-spesifik, tetapi proses packaging, integrasi Block Design, dan implementasi tetap menggunakan toolchain AMD/Xilinx.

5.11 Keterbatasan Desain

Sebagai bagian dari pembahasan yang komprehensif, berikut adalah keterbatasan yang merupakan ruang lingkup yang telah ditetapkan pada Batasan Penelitian (Bab I):

1. **TX-only.** IP dirancang sebagai transmitter dan tidak mencakup jalur penerima (RX) maupun mode full-duplex.

2. **Tanpa DMA.** Transfer data dilakukan melalui *programmed I/O* berbasis register. Streaming audio kontinu memerlukan intervensi CPU setiap frame melalui mekanisme status-bit polling.
3. **Tanpa FIFO audio.** Tidak terdapat buffer FIFO sehingga potensi *underrun* dikelola sepenuhnya oleh firmware. Pendekatan status-bit polling memitigasi risiko ini namun tidak menghilangkannya sepenuhnya pada kondisi CPU terbebani.
4. **Tanpa transfer berbasis interupsi.** Firmware menggunakan model polling sinkron sehingga CPU tidak dapat menangani tugas lain selama streaming audio berlangsung.
5. **Bare-metal only.** Pengujian dilakukan pada lingkungan bare-metal MicroBlaze V tanpa RTOS atau Linux. Integrasi ke ekosistem ASoC Linux merupakan pekerjaan masa mendatang.

Keterbatasan-keterbatasan ini menjadi landasan saran pengembangan yang diuraikan pada Bab VI, khususnya penambahan FIFO TX, mekanisme DMA berbasis AXI, dan integrasi Linux/ASoC.

BAB VI
KESIMPULAN DAN SARAN

Penelitian ini berhasil merancang dan mengimplementasikan *IP core* transmitter I2S berbasis AXI4-Lite menggunakan Verilog pada sistem berbasis MicroBlaze V. IP tersebut dikemas sebagai *custom IP*, diintegrasikan ke dalam Vivado Block Design, serta dikendalikan melalui perangkat lunak bare-metal menggunakan antarmuka AXI4-Lite.

Implementasi protokol Philips I2S pada IP menghasilkan sinyal BCLK, WS, dan DATA dengan slot 32-bit, bit tunda satu posisi, pengiriman MSB-first, dan padding nol sesuai konfigurasi lebar sampel. Pembangkitan clock berbasis *phase accumulator* mendukung frekuensi sampling integer standar maupun non-standar dalam rentang RTL tanpa pembangkit clock audio khusus.

Hasil sintesis dan implementasi sistem SoC menunjukkan penggunaan 903 Slice, 2435 LUT, 2330 Flip-Flop, dan 32 BRAM, dengan estimasi konsumsi daya total sebesar 0,209 W yang terdiri atas daya dinamis sebesar 0,135 W dan daya statis sebesar 0,073 W. Laporan timing menunjukkan WNS sebesar 0,064 ns pada clock 100 MHz. Berdasarkan slack tersebut, frekuensi operasi maksimum diperkirakan sekitar 100,64 MHz. Secara keseluruhan, penelitian ini menghasilkan IP core I2S berbasis AXI4-Lite yang modular dan dapat dikonfigurasi melalui perangkat lunak. Pengembangan berikutnya dapat menambahkan FIFO TX, DMA atau interupsi, serta dua jalur filter dengan state yang terpisah apabila pemrosesan stereo independen diperlukan.

DAFTAR PUSTAKA

[1] Philips Semiconductors, "I^{2}S bus specification," Philips Semiconductors, Tech. Rep., Feb. 1986, revision 1. Tersedia: https://www.nxp.com/docs/en/user-guide/UM11732.pdf.

[2] S. M. Trimberger, "Three ages of FPGAs: A retrospective on the first thirty years of FPGA technology," *Proceedings of the IEEE*, vol. 103, no. 3, pp. 318–331, 2015.

[3] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Prentice Hall, 2003.

[4] L. H. Crockett, R. A. Elliot, M. A. Enderwitz, and R. W. Stewart, "The Zynq book: Embedded processing with the ARM cortex-A9 on the Xilinx Zynq-7000 all programmable SoC," *Strathclyde Academic Media*, 2014, iSBN 978-0992978709. Tersedia: http://www.zynqbook.com.

[5] A. Waterman, Y. Lee, R. Avizienis, D. A. Patterson, and K. Asanović, "The RISC-V instruction set architecture," in *Hot Chips 28 Symposium (HCS)*, 2016, pp. 1–52.

[6] Advanced Micro Devices, Inc., *MicroBlaze Processor Reference Guide*, Advanced Micro Devices, Inc., 2021, tersedia: https://www.xilinx.com/support/documentation/sw_manuals/xilinx2021_2/ug984-vivado-microblaze-ref.pdf.

[7] ARM Limited, "AMBA AXI and ACE protocol specification," ARM Limited, Tech. Rep. ARM IHI 0022D, 2010, issue D. Tersedia: https://developer.arm.com/documentation/ihi0022/d/.

[8] Texas Instruments, "PCM5102A 2-VRMS, 112-dB audio stereo DAC with PLL and 32-bit, 384-kHz PCM interface," Texas Instruments, Tech. Rep. SLAS764B, 2014, datasheet. Tersedia: https://www.ti.com/lit/ds/symlink/pcm5102a.pdf.

[9] Digilent Inc., *Basys 3 FPGA Board Reference Manual*, Digilent Inc., 2020, tersedia: https://digilent.com/reference/programmable-logic/basys-3/reference-manual.

[10] Advanced Micro Devices, Inc., *Vivado Design Suite User Guide: IP Subsystems*, Advanced Micro Devices, Inc., 2025.

[11] J. Sang, D. Peng, and M. Zhou, "Design and implementation of I2S digital audio interface on FPGA," *Journal of Physics: Conference Series*, vol. 1601, no. 5, p. 052040, 2020.

[12] A. Kaviani and D. Lewis, "Embedded logic analyzer and I2S audio interface on low-cost FPGA," in *Proceedings of the IEEE International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2017, pp. 221–224.

[13] A. Lafosse and D. Gaffe, *Embedded FPGA Systems: Design and Programming with Vivado*. Wiley-ISTE, 2018.

[14] V. Sklyarov, I. Sklyarova, A. Barkalov, and L. Skliarova, "Hierarchical AXI-bus based SoC for FPGA," in *2014 24th International Conference on Field Programmable Logic and Applications (FPL)*, 2014, pp. 1–4.

[15] R. Ginosar, "Metastability and synchronizers: A tutorial," *IEEE Design & Test of Computers*, vol. 28, no. 5, pp. 23–35, 2011.

[16] A. Waterman and K. Asanović, "The RISC-V instruction set manual, volume I: Unprivileged ISA," RISC-V Foundation, Tech. Rep. Document Version 20191213, 2019, tersedia: https://github.com/riscv/riscv-isa-manual/releases/.

[17] S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture: RISC-V Edition*. Morgan Kaufmann, 2022.

[18] P. P. Chu, *FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version*. Wiley-IEEE Press, 2008.

[19] Advanced Micro Devices, Inc., *Vivado Design Suite User Guide: Creating and Packaging Custom IP*, Advanced Micro Devices, Inc., 2021, available: https://docs.amd.com/r/en-US/ug1118-vivado-creating-packaging-custom-ip.

[20] ——, *AXI Reference Guide*, Advanced Micro Devices, Inc., 2017, tersedia: https://www.xilinx.com/support/documentation/ip_documentation/axi_ref_guide/latest/ug1037-vivado-axi-reference-guide.pdf.

[21] J. Vankka and K. A. I. Halonen, *Direct Digital Synthesizers: Theory, Design and Applications*. Boston: Kluwer Academic Publishers, 2001.

[22] G. P. Ao, "The basic principle and FPGA implementation of NCO," in *Proceedings of the 2012 Second International Conference on Instrumentation, Measurement, Computer, Communication and Control*. Yantai, China: IEEE, 2012.

[23] I. Kuon, R. Tessier, and J. Rose, "FPGA architecture: Survey and challenges," *Foundations and Trends in Electronic Design Automation*, vol. 2, no. 2, pp. 135–253, 2008.

[24] Advanced Micro Devices, Inc., *Vivado Design Suite User Guide: Designing with IP*, Advanced Micro Devices, Inc., 2021, tersedia: https://www.xilinx.com/support/documentation/sw_manuals/xilinx2021_2/ug896-vivado-ip.pdf.

[25] ——, *Vitis Unified Software Platform Documentation: Embedded Software Development*, Advanced Micro Devices, Inc., 2021, tersedia: https://www.xilinx.com/support/documentation/sw_manuals/xilinx2021_2/ug1400-vitis-embedded.pdf.

[26] A. Bejo, "Tutorial microblaze v," 2025, course tutorial, IC Design Facility Laboratory, Department of Electrical Engineering and Information Engineering, Faculty of Engineering, Universitas Gadjah Mada. Version 2025/11/18.

[27] Advanced Micro Devices, Inc., *LogiCORE I2S Transmitter and Receiver Product Guide*, Advanced Micro Devices, Inc., 2021, tersedia: https://www.xilinx.com/support/documentation/ip_documentation/i2s_transmitter/v3_0/pg308-i2s-logicore.pdf.

[28] OpenCores, "I2S Interface," https://opencores.org/projects/i2s_interface, 2019, open-source I2S IP core. Last updated January 31, 2019. Accessed June 2026.

[29] Ultra-Embedded, “I2s master rtl,” https://github.com/ultraembedded/cores/tree/master/i2s, 2012, open-source Verilog implementation (GPL). Accessed June 2026.

[30] ——, “I2s master testbench,” https://github.com/ultraembedded/cores/blob/master/i2s/testbench/tb_top.v, 2012, reference testbench for I2S verification. Accessed June 2026.

LAMPIRAN

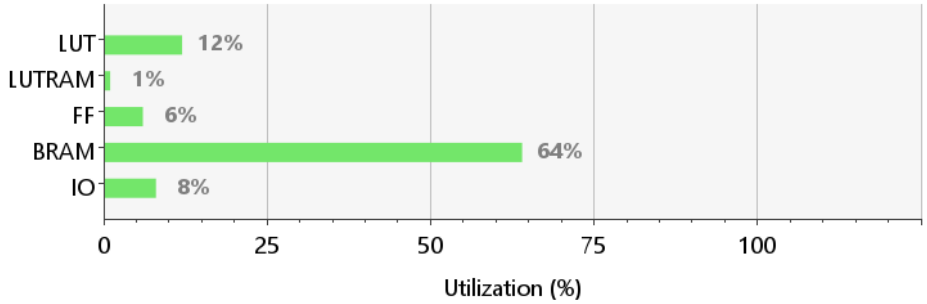
L.1 Hasil utilisasi resource

L.1.1 Post-implementation resource report

Tabel L.1. Ringkasan utilisasi resource FPGA

Resource	Used	Available	Utilization (%)
Post-implementation SoC MicroBlaze V + I2S TX			
LUT	2,435	20,800	11.71
LUTRAM	107	9,600	1.11
Flip-flop	2,330	41,600	5.60
Block RAM Tile	32	50	64.00
IO	8	106	7.55
MMCM	1	5	20.00

Resource	Utilization	Available	Utilization %
LUT	2435	20800	11.71
LUTRAM	107	9600	1.11
FF	2330	41600	5.60
BRAM	32	50	64.00
IO	8	106	7.55



Gambar L.1. Ringkasan utilisasi resource (visual).

L.1.2 Timing report

Tabel L.2. Hasil analisis timing

Parameter	**Value**
Target Clock Frequency (AXI)	100 MHz
Worst Negative Slack (WNS)	0.064 ns
Worst Hold Slack (WHS)	0.031 ns
Worst Pulse Width Slack (WPWS)	3.000 ns
Timing Met?	**Yes**

L.2 Timing diagram

L.2.1 I2S audio transmission timing

```
Mode uji utama      : Fs = 48 kHz
Sample width        : 24-bit per kanal (slot 32-bit)
Struktur frame      : 64 BCLK per frame stereo

WS/LRCLK = 0 : slot kiri  (32 BCLK)
WS/LRCLK = 1 : slot kanan (32 BCLK)

Pada setiap slot:
 bit-0   : delay 1-bit (sesuai Philips I2S)
 bit-1..24 : data audio (MSB -> LSB)
 bit-25..31: zero padding
```

L.3 Format log hasil pengujian

L.3.1 I2S audio playback test

Blok berikut menunjukkan format log yang digunakan untuk mencatat pengujian. Nilai alamat dan hasil pengamatan harus diganti dengan transkrip UART atau catatan pengukuran asli sebelum naskah diserahkan.

```
=== I2S AXI4-Lite test harness ===
Base address   : 0x00020000
Sample rate    : 48000 Hz
Sample width   : 24 bit
Registers      : DATA_LEFT=0x00 DATA_RIGHT=0x04 CONTROL=0x08 STATUS=0
```

```
[TEST] Stereo sine 440 Hz
I2S streaming active, output teramati pada BCLK/WS/DATA
Audio terverifikasi pada DAC PCM5102A
```

L.4 Deskripsi block design

Vivado IP Integrator Block Design terdiri dari komponen berikut:

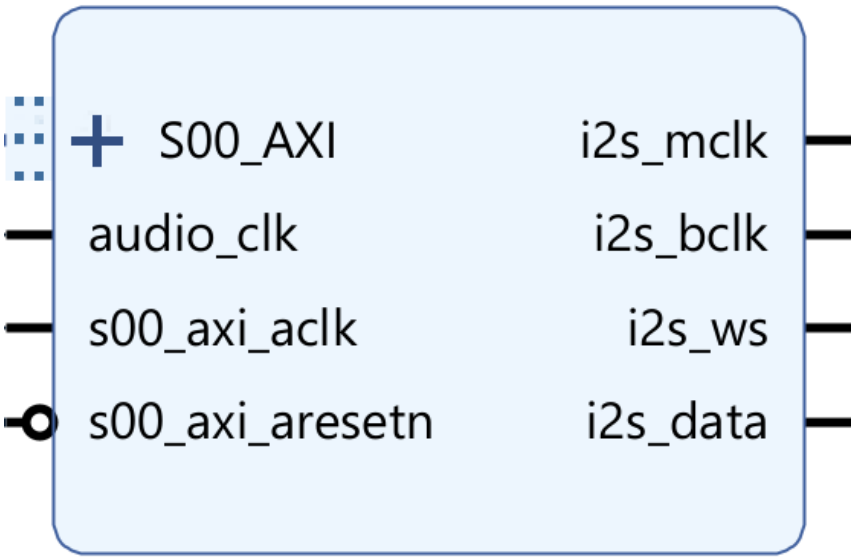
1. **MicroBlaze V (microblaze_riscv_0)**
2. **AXI SmartConnect**
3. **Local Memory / BRAM**
4. **I2S Transmitter (custom IP i2s_v1_0)**
5. **AXI UART Lite**
6. **Clocking Wizard**
7. **Processor System Reset**

Koneksi utama:

- Semua slave AXI4-Lite terhubung melalui AXI SmartConnect.
- Port `audio_clk` tetap tersedia untuk kompatibilitas wrapper, tetapi core terbaru membangkitkan `mclk/bclk/ws` dari domain `s00_axi_aclk` 100 MHz menggunakan NCO.
- Jalur AXI register dan logika audio menggunakan clock sistem 100 MHz yang sama.
- Sinyal eksternal utama adalah I2S: BCLK, LRCLK/WS, DATA, dan opsional MCLK.
- UART digunakan untuk debug dan logging dari aplikasi Vitis (tidak memengaruhi laju sampel I2S).

L.4.1 I2S IP Block (Standalone)

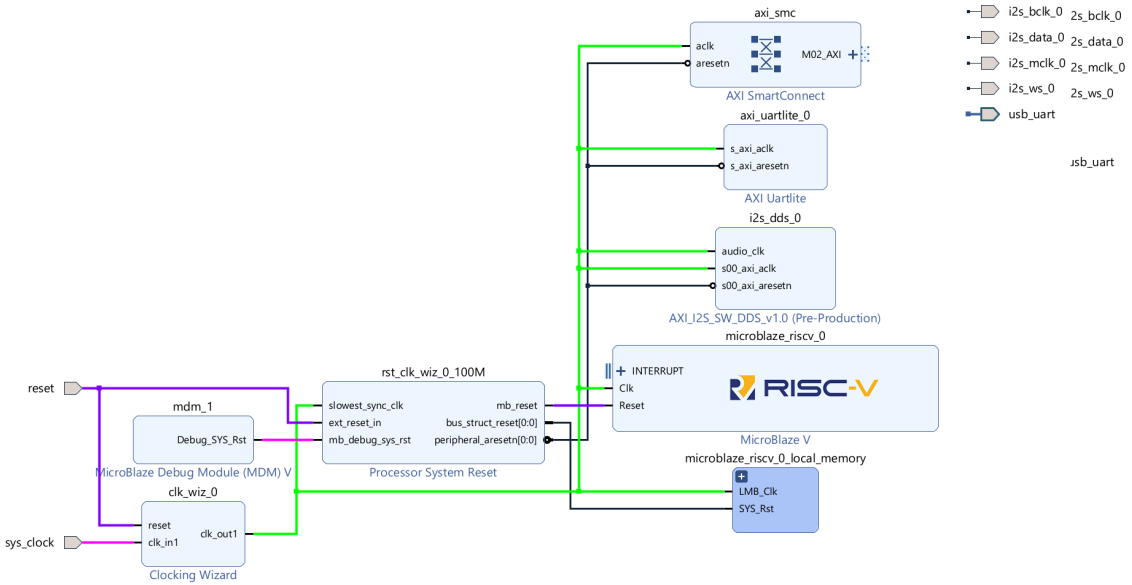
Gambar L.2 memperlihatkan blok I2S IP terpisah beserta port audio clock, sinyal I2S, dan antarmuka AXI4-Lite.



Gambar L.2. Blok IP I2S standalone.

L.4.2 Clocking View

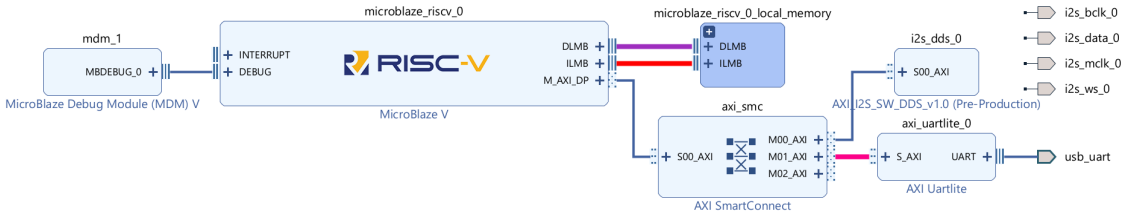
Clocking view menunjukkan Clocking Wizard untuk domain sistem 100 MHz dan koneksi ke IP I2S.



Gambar L.3. Clocking view dan jalur clock sistem pada Vivado Block Design.

L.4.3 Interface View

Interface view merinci koneksi pin PMOD, sinyal I2S, dan UART debug.



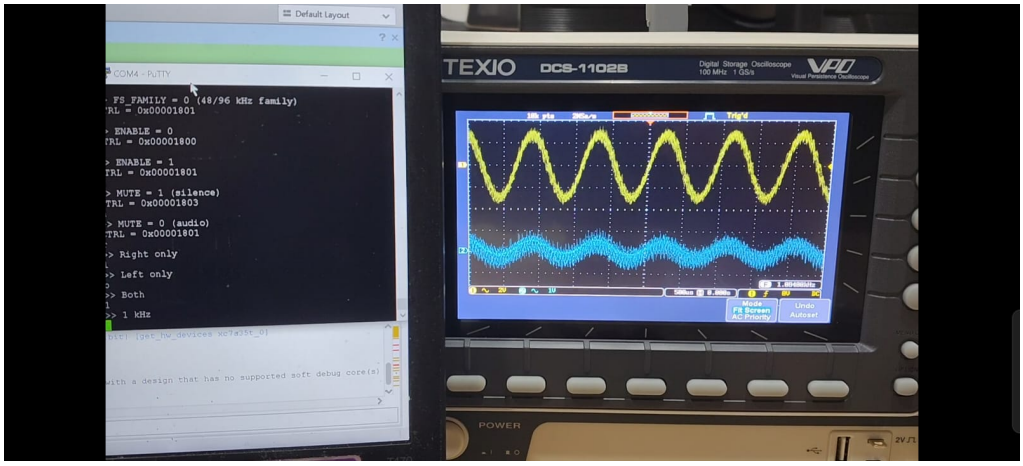
Gambar L.4. Interface view yang menunjukkan pemetaan antarmuka I2S, PMOD, dan UART.

L.5 Hasil pengujian i2s transmitter - capture osiloskop

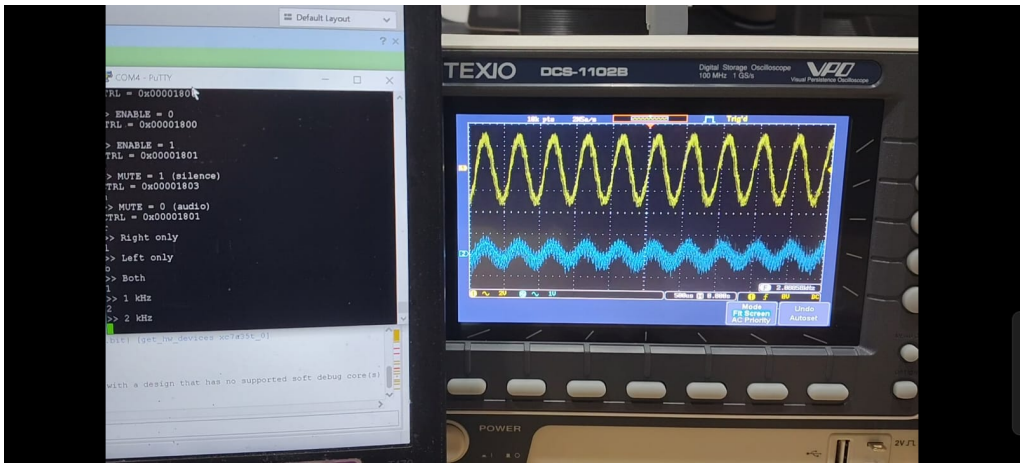
Bagian ini menampilkan hasil pengujian I2S Transmitter melalui capture osiloskop eksternal. Seluruh observasi dilakukan dari sinyal nyata pada probe osiloskop digital. Gambar berikut memperlihatkan BCLK, LRCLK/WS, dan DATA pada beberapa kondisi uji.

L.5.1 Test Frekuensi Audio

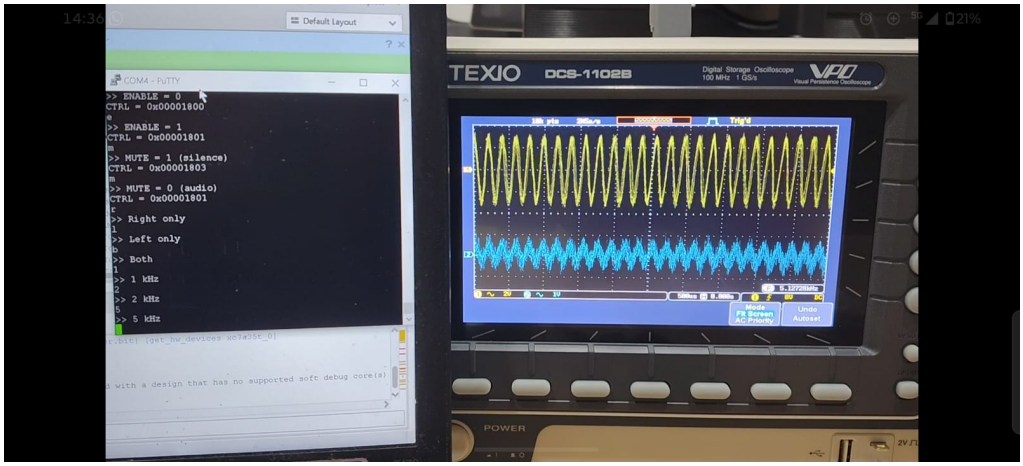
Pengujian frekuensi sinus untuk verifikasi frekuensi sampling dan integritas I2S:



Gambar L.5. Capture osiloskop: audio tone 1 kHz.



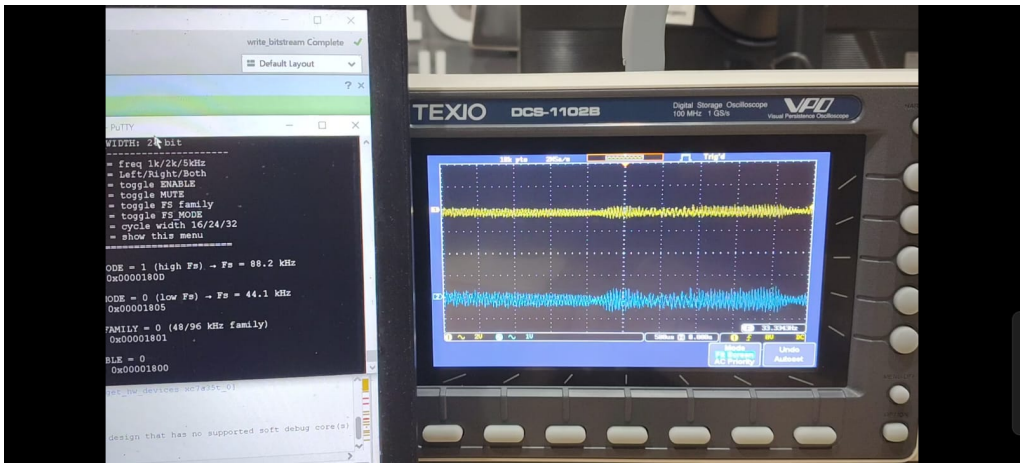
Gambar L.6. Capture osiloskop: audio tone 2 kHz.



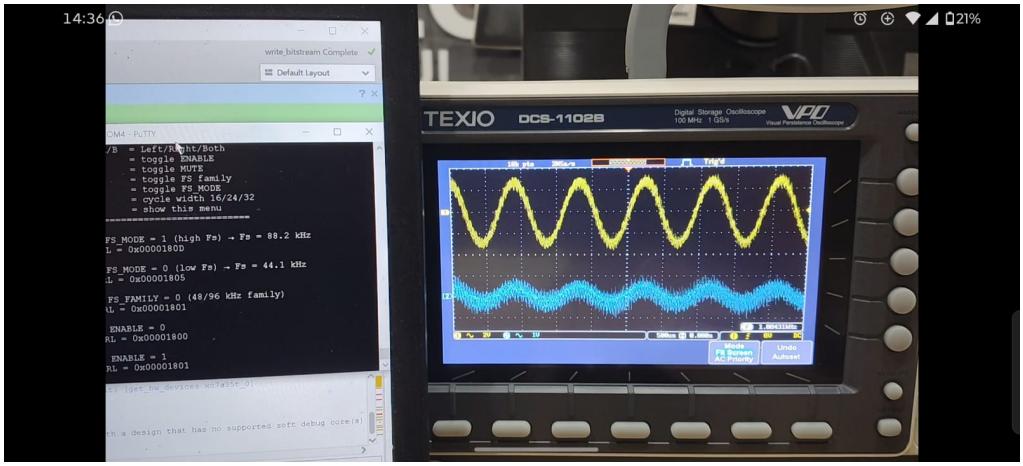
Gambar L.7. Capture osiloskop: audio tone 5 kHz.

L.5.2 Test Kontrol Enable

Pengujian sinyal enable untuk aktivasi dan deaktivasi I2S transmitter:



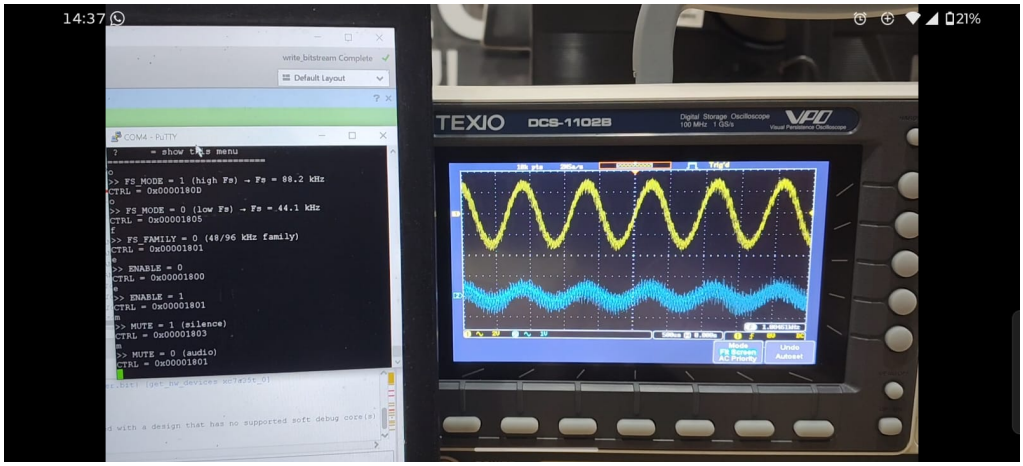
Gambar L.8. Capture osiloskop: enable=0.



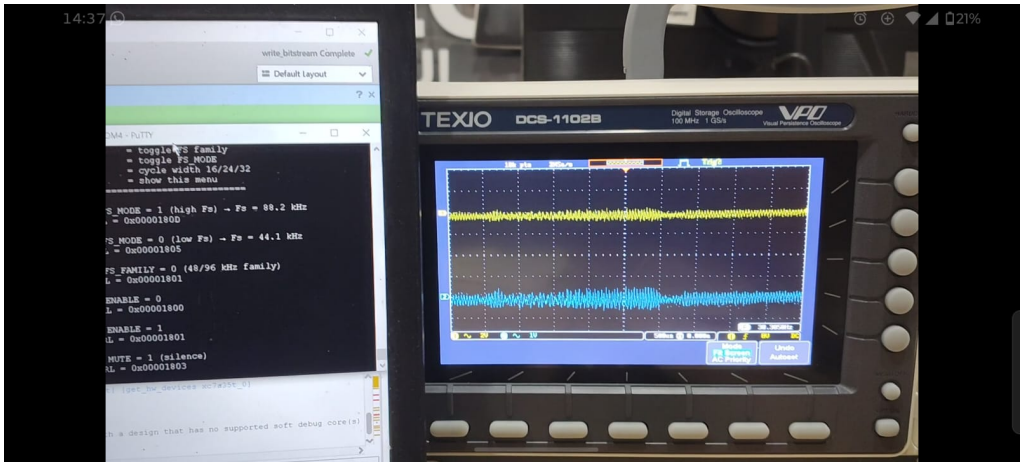
Gambar L.9. Capture osiloskop: enable=1.

L.5.3 Test Kontrol Mute

Pengujian fungsi mute untuk menghentikan keluaran audio sambil mempertahankan sinyal I2S:



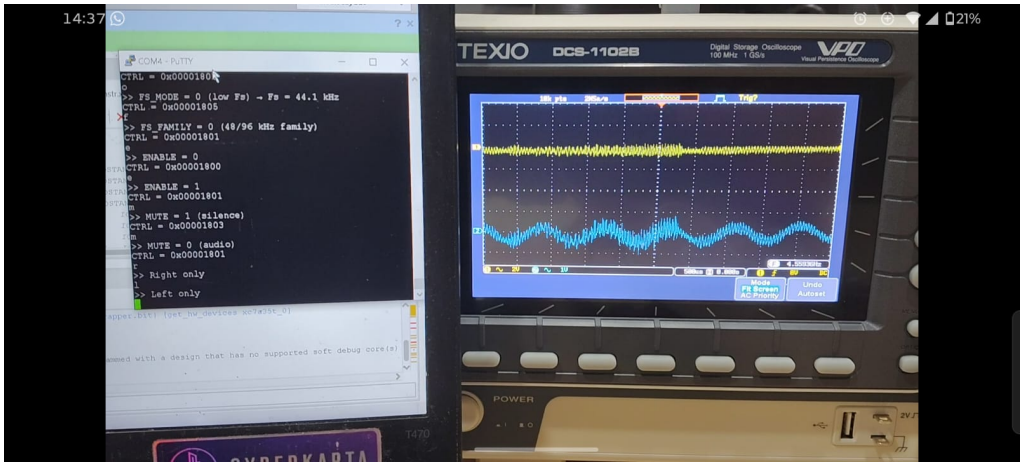
Gambar L.10. Capture osiloskop: mute=0.



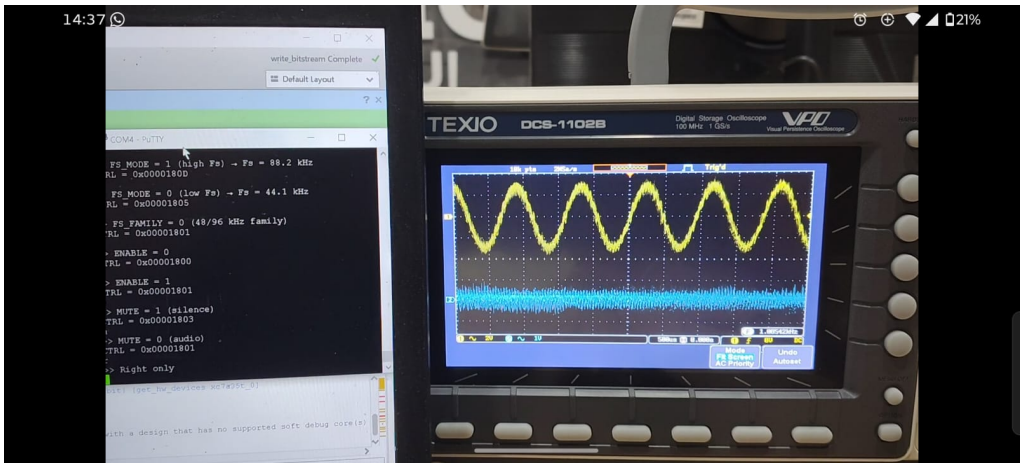
Gambar L.11. Capture osiloskop: mute=1.

L.5.4 Test Routing Stereo

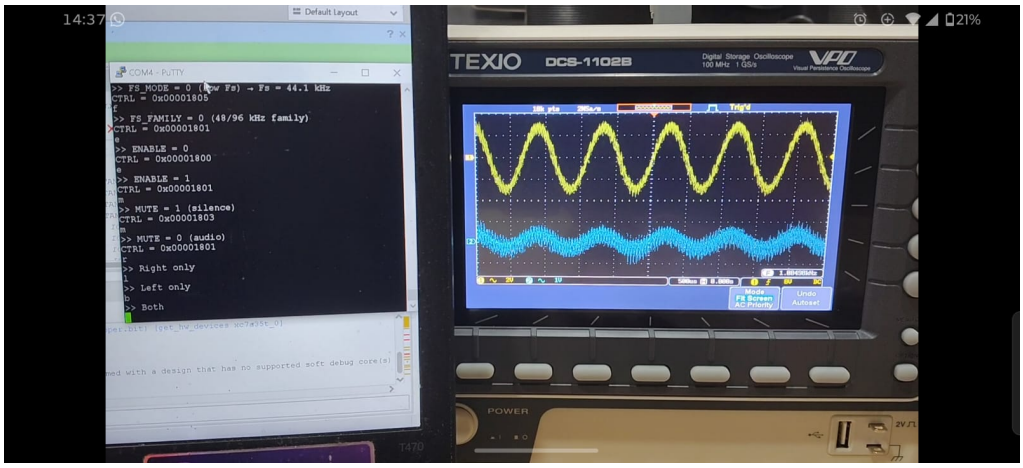
Pengujian routing kanal stereo untuk verifikasi pemisahan left dan right channel:



Gambar L.12. Hasil capture osiloskop: Left channel only - hanya kanal left dioutput



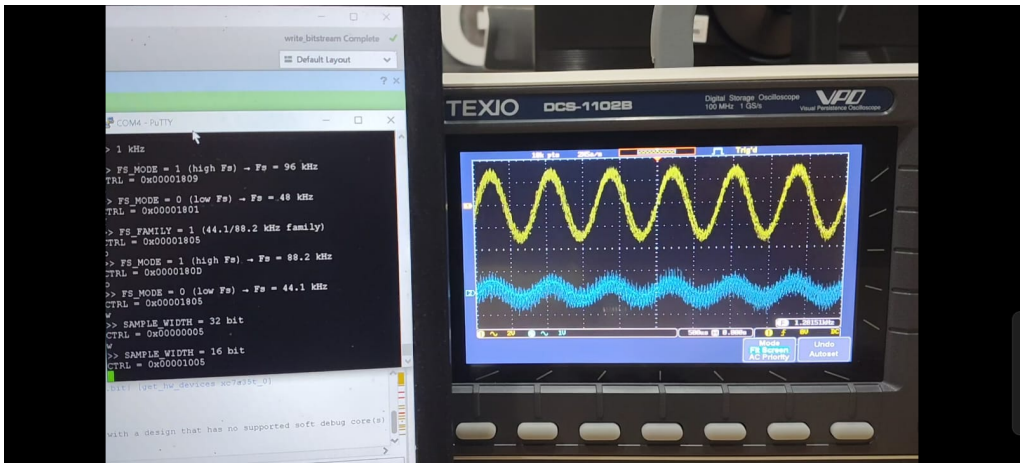
Gambar L.13. Hasil capture osiloskop: Right channel only - hanya kanal right dioutput



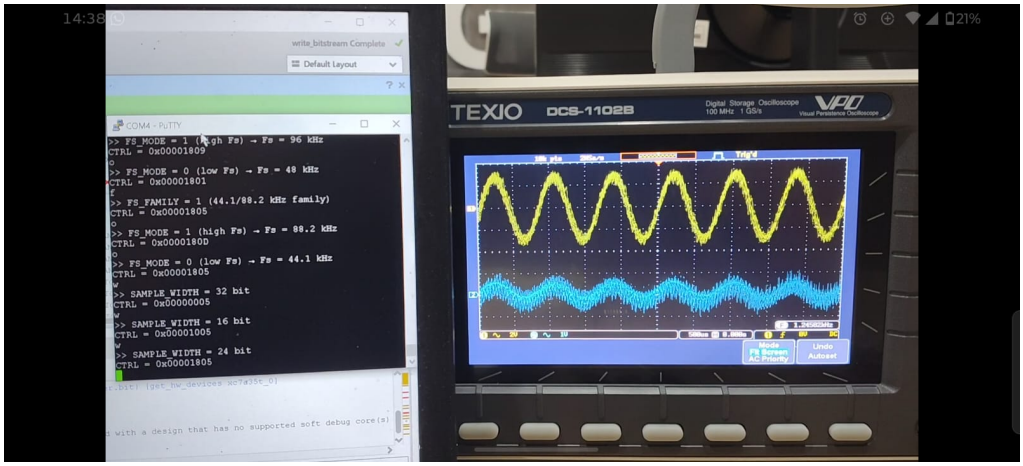
Gambar L.14. Hasil capture osiloskop: Both channels - kedua kanal left dan right aktif simultan

L.5.5 Test Lebar Sample (Sample Width)

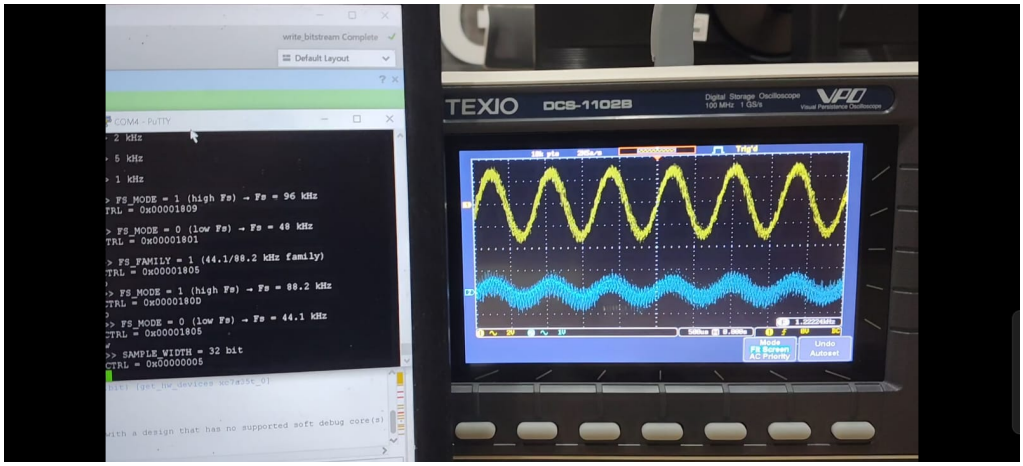
Pengujian dengan berbagai lebar bit untuk verifikasi format audio dan slot frame I2S:



Gambar L.15. Hasil capture osiloskop: Sample width 16-bit - format audio PCM 16-bit



Gambar L.16. Hasil capture osiloskop: Sample width 24-bit - format audio PCM 24-bit pada slot 32-bit



Gambar L.17. Hasil capture osiloskop: Sample width 32-bit - full 32-bit slot I2S frame

L.6 Design Timing Summary

Laporan timing summary dari hasil implementasi Vivado menunjukkan performa timing keseluruhan desain:

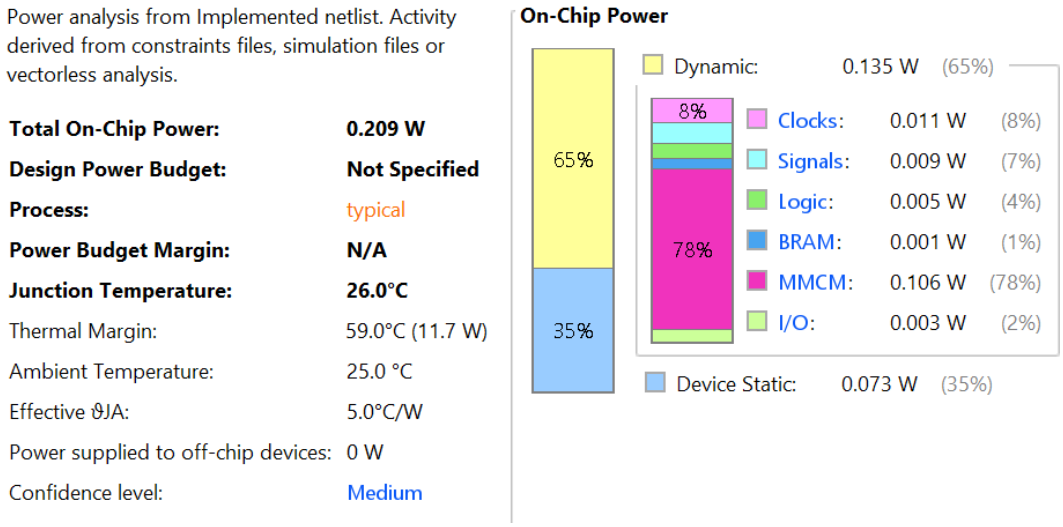
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	0.064 ns	Worst Hold Slack (WHS):	0.031 ns	Worst Pulse Width Slack (WPWS):	3.000 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	7359	Total Number of Endpoints:	7359	Total Number of Endpoints:	2606

All user specified timing constraints are met.

Gambar L.18. Design timing summary report - Worst Negative Slack (WNS), Total Negative Slack (TNS), dan status timing closure

L.7 Power Summary

Laporan analisis power dari post-implementation desain menunjukkan konsumsi daya dinamis dan statis:

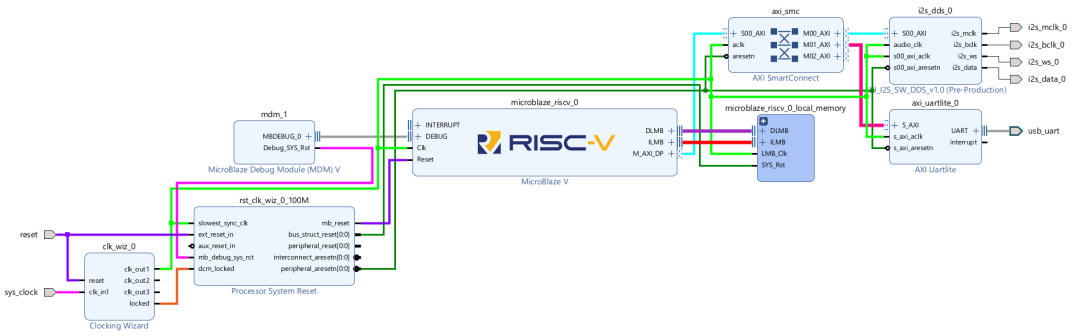


Gambar L.19. Power summary report - total on-chip power consumption, breakdown per resource (MMCM, Block RAM, Slice Logic), dan power supply margins

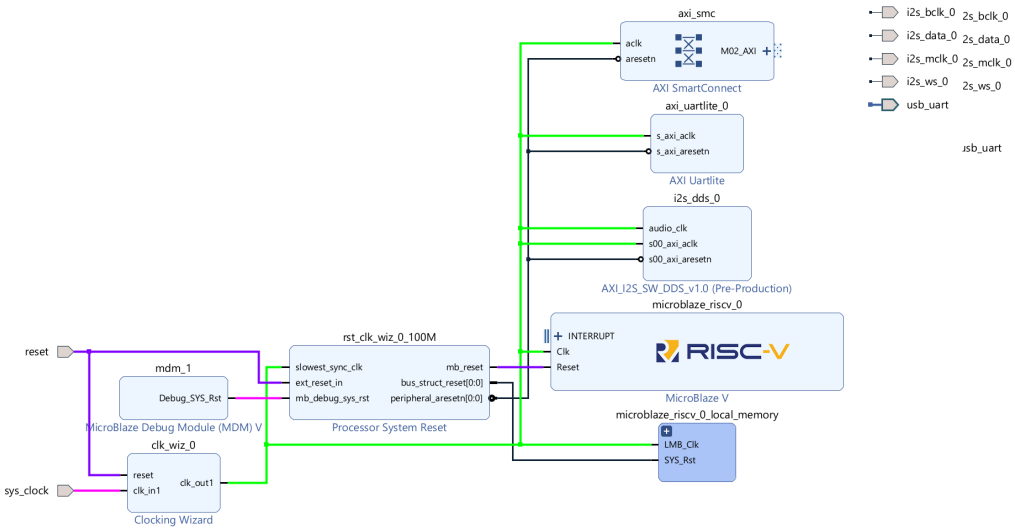
RINGKASAN IP BLOCK DESIGN

Lampiran ini merangkum IP utama pada Vivado Block Design yang digunakan dalam proyek. Berkas deskripsi block design utama dan hasil generasi IP tersedia di direktori proyek `freshi2s`.

Diagram Block Design



Gambar L.20. Diagram block design berwarna yang menunjukkan hubungan prosesor, interconnect, clock, dan IP I2S.

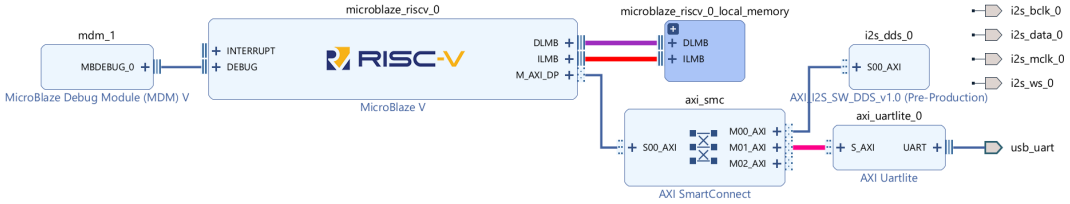


Gambar L.21. Tampilan clocking view pada block design untuk domain clock sistem dan jalur audio.

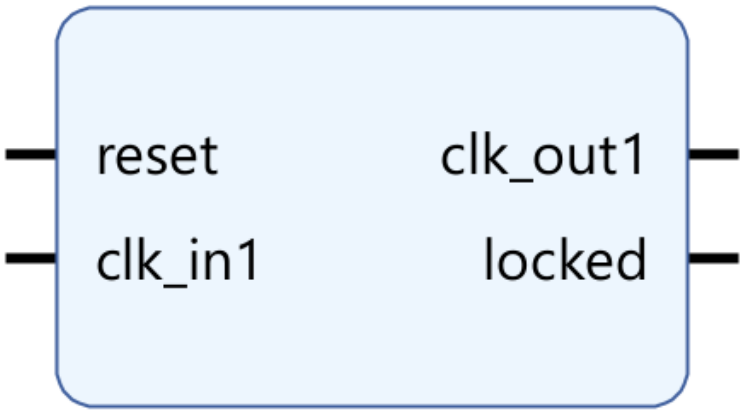
Figur Komponen Block Design

Ringkasan arsitektur

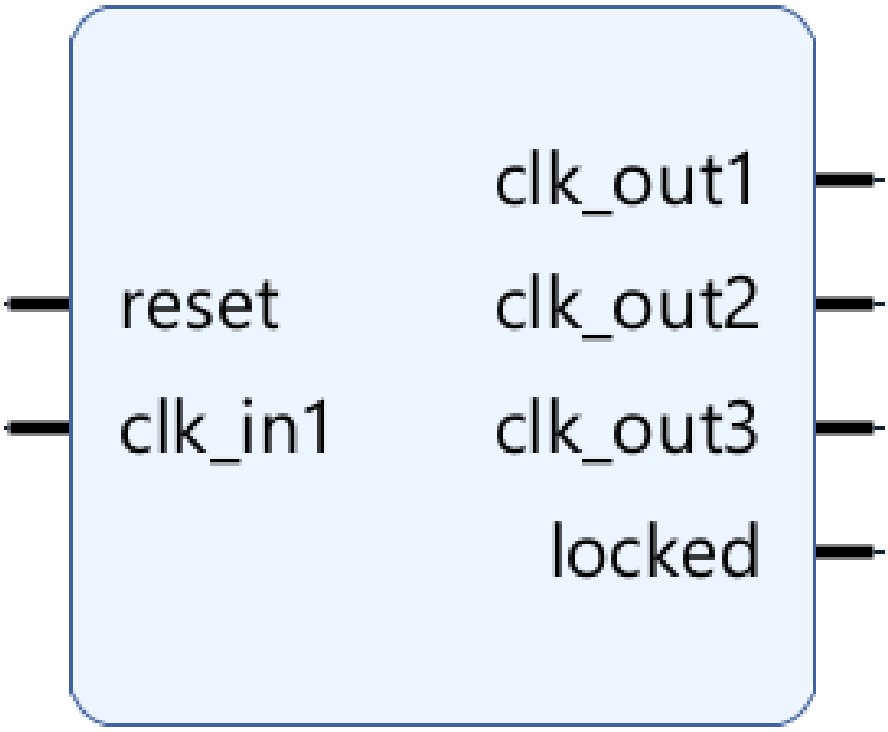
- Clock sistem berasal dari `sys_clock` 100 MHz dan masuk ke `clk_wiz_0`, yang menghasilkan domain 100 MHz untuk MicroBlaze, AXI SmartConnect, dan IP I2S.
- IP `i2s_0` membangkitkan `BCLK`, `WS`, dan `MCLK` secara internal menggunakan akumulator fase NCO secara murni digital dari clock sistem 100 MHz.



Gambar L.22. Tampilan interface view yang memperlihatkan keterhubungan MicroBlaze, UARTLite, dan IP I2S melalui AXI SmartConnect.



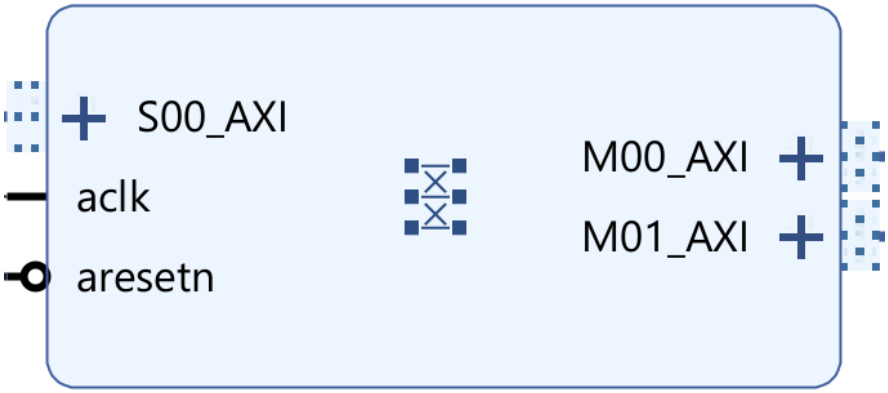
Gambar L.23. Tampilan awal konfigurasi Clocking Wizard sebelum penyesuaian frekuensi keluaran.



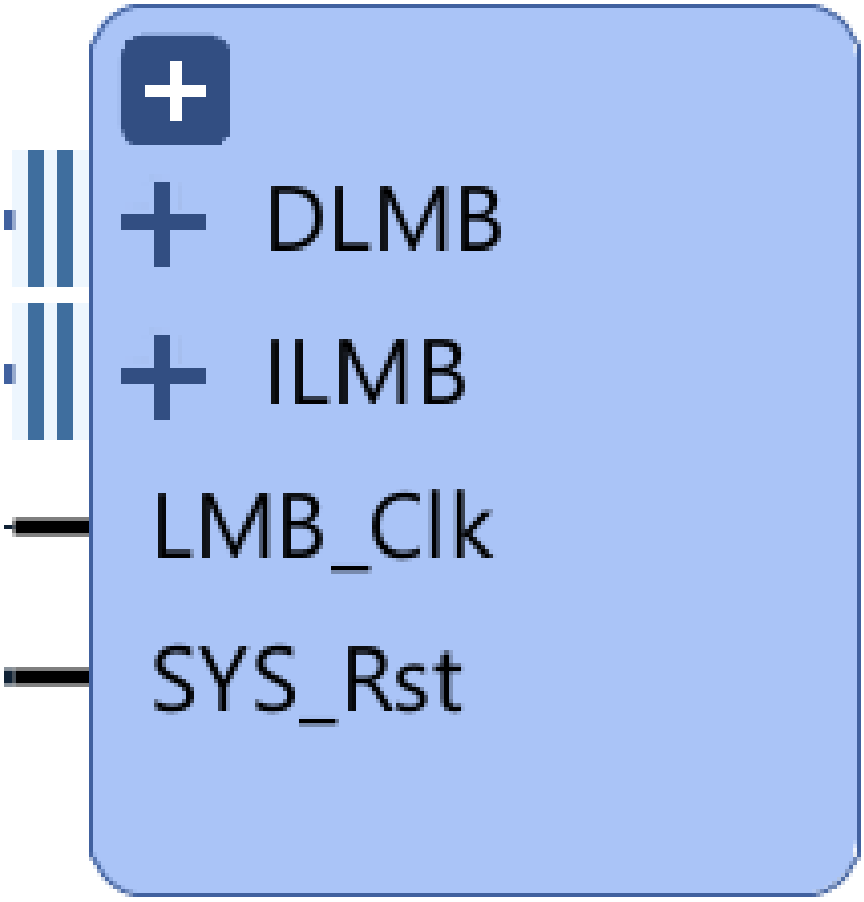
Gambar L.24. Clocking Wizard setelah dikonfigurasi untuk menghasilkan clock sistem 100 MHz.

Gambar L.25. Blok prosesor MicroBlaze V yang menjadi pengendali utama sistem.

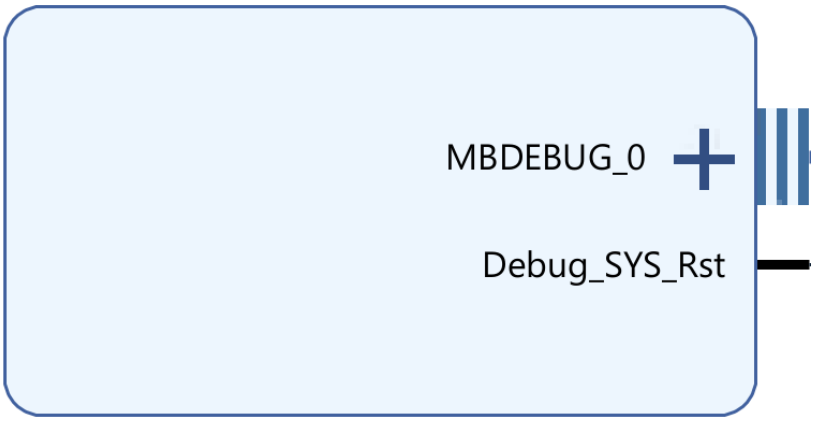
Gambar L.26. Block design reset controller yang membangkitkan sinyal reset sinkron untuk domain sistem.



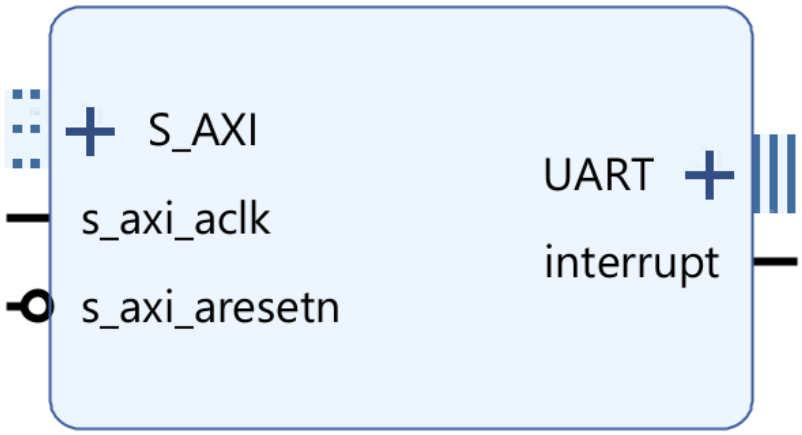
Gambar L.27. AXI SmartConnect yang merutekan transaksi dari MicroBlaze ke peripheral I2S dan UARTLite.



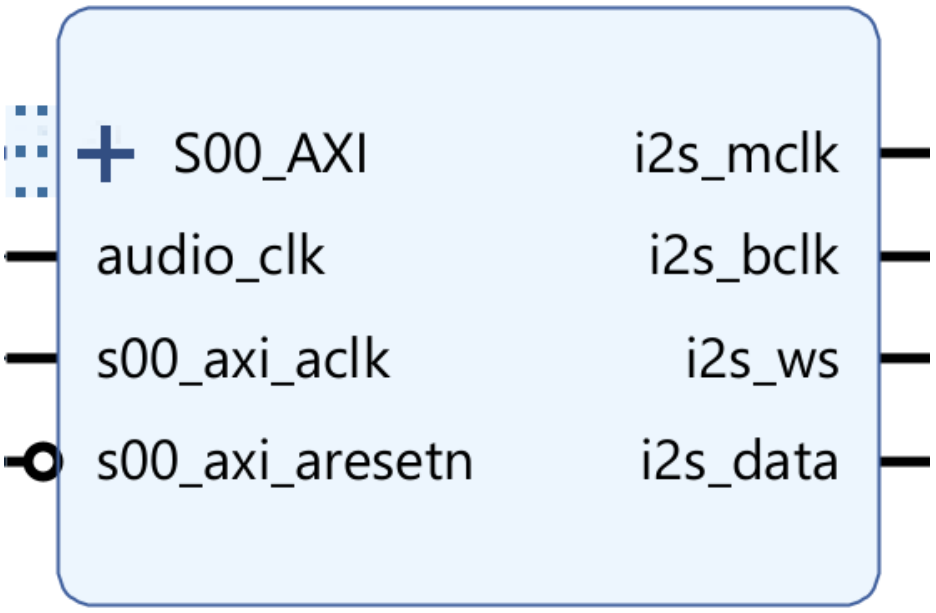
Gambar L.28. Subsistem BRAM lokal untuk instruksi dan data MicroBlaze V.



Gambar L.29. Microprocessor Debug Module yang digunakan untuk keperluan debug prosesor.



Gambar L.30. Peripheral UARTLite sebagai antarmuka terminal firmware.



Gambar L.31. Blok internal IP I2S yang digunakan sebagai peripheral utama.

- microblaze_riscv_0 bertindak sebagai master AXI dan terhubung ke axi_smc.
- axi_smc merutekan transaksi ke slave peripheral, termasuk i2s_0 dan axi_uartlite_0.
- Keluaran audio eksternal yang diekspos block design adalah i2s_mclk, i2s_bclk, i2s_ws, dan i2s_data.

Peta alamat

Tabel L.3. Segmen alamat penting pada block design bdesign_clock_core.

Peripheral	Segmen	Offset basis
i2s_0	S00_AXI_reg	0x00020000
axi_uartlite_0	SEG_axi_uartlite_0_Reg	0x04060000
localBRAM	dlmb/ilmbmemory	0x00000000

clk_wiz_0

Purpose Clocking Wizard untuk membangkitkan domain clock sistem 100 MHz.

Key parameters CLK_IN1=100MHz, CLKOUT1=100MHz. Keluaran audio tambahan (clk_out2/clk_out3) dapat dihubungkan ke port legacy IP tetapi tidak diperlukan oleh logika DDS.

Connections clk_out1 ke domain 100 MHz MicroBlaze, SmartConnect, I2S, dan UARTLite.

microblaze_riscv_0

Purpose Soft processor RISC-V untuk menjalankan firmware freshi2s / sw / helloworld.c.

Key parameters Debug aktif, jalur M_AXI_DP untuk akses peripheral, serta ILMB/DLMB untuk memori lokal.

Connections Clock 100 MHz dari clk_wiz_0; reset dari rst_clk_wiz_0_100M; master AXI ke axi_smc/S00_AXI.

microblaze_riscv_0_local_memory

Purpose Subsistem memori lokal instruksi dan data.

Sub-IP dlmb_v10, ilmb_v10, dlmb_bram_if_cntlr, ilmb_bram_if_cntlr, dan lmb_bram.

Key parameters BRAM bertipe True_Dual_Port_RAM dan dihubungkan ke kedua jalur LMB.

mdm_1

Purpose *Microprocessor Debug Module* untuk debug prosesor.

Connections Antarmuka debug langsung ke `microblaze_riscv_0/DEBUG`.

rst_clk_wiz_0_100M

Purpose Reset controller sinkron untuk domain 100 MHz.

Connections Menghasilkan `mb_reset`, `bus_struct_reset`, dan `peripheral_aresetn` untuk MicroBlaze, SmartConnect, I2S, dan UARTLite.

axi_smc

Purpose AXI SmartConnect yang menghubungkan satu master ke beberapa slave.

Connections `S00_AXI` dari `microblaze_riscv_0`; `M00_AXI` ke `i2s_0`; `M01_AXI` ke `axi_uartlite_0`.

i2s_0

Purpose Custom IP I2S transmitter berbasis NCO (`xilinx.com:user:i2s:1.0`).

Main interfaces `S00_AXI` untuk kontrol register dan empat keluaran I2S.

Key parameters Empat register AXI4-Lite 32-bit (`DATA_LEFT`, `DATA_RIGHT`, `CONTROL`, `STATUS`), `C_S00_AXI_BASEADDR=0x00020000`, seluruh logika audio pada domain 100 MHz.

axi_uartlite_0

Purpose UARTLite untuk antarmuka terminal firmware.

Main interfaces `S_AXI` untuk konfigurasi AXI dan `UART` untuk `usb_uart_txd`/`usb_uart_rxd`.

Key parameters Alamat basis `0x04060000` pada ruang alamat MicroBlaze.

L.8 Kode Testbench

Testbench berikut digunakan untuk memverifikasi transaksi AXI4-Lite, konfigurasi register IP I2S, serta keluaran clock dan sinyal audio I2S. Testbench bersifat *self-checking* dan mencatat jumlah kesalahan selama simulasi.

```
`timescale 1ns / 1ps

// Self-checking AXI4-Lite and I2S protocol testbench.
// Run as the Vivado simulation top: tb_i2s_axi_lite.
module tb_i2s_axi_lite;

    localparam integer CLK_PERIOD_NS = 10;
    localparam [3:0] ADDR_DATA_LEFT  = 4'h0;
    localparam [3:0] ADDR_DATA_RIGHT = 4'h4;
    localparam [3:0] ADDR_CONTROL    = 4'h8;
    localparam [3:0] ADDR_STATUS     = 4'hC;
    localparam [31:0] LEFT_SAMPLE    = 32'h00A5_C31E;
    localparam [31:0] RIGHT_SAMPLE   = 32'h005A_3CE1;
    localparam [31:0] CTRL_24_48K    = (32'd96000 << 7) | (32'd24 << 2) | 32'h1;
    localparam [31:0] CTRL_MUTE_24_48K = CTRL_24_48K | 32'h2;

    reg aclk = 1'b0;
    reg aresetn = 1'b0;
    always #(CLK_PERIOD_NS/2) aclk = ~aclk;

    reg [3:0] awaddr = 4'd0;
    reg [2:0] awprot = 3'd0;
    reg awvalid = 1'b0;
    wire awready;
    reg [31:0] wdata = 32'd0;
    reg [3:0] wstrb = 4'hF;
    reg wvalid = 1'b0;
    wire wready;
    wire [1:0] bresp;
    wire bvalid;
    reg bready = 1'b1;
    reg [3:0] araddr = 4'd0;
    reg [2:0] arprot = 3'd0;
    reg arvalid = 1'b0;
    wire arready;
    wire [31:0] rdata;
    wire [1:0] rresp;
    wire rvalid;
    reg rready = 1'b1;

    wire i2s_mclk;
    wire i2s_bclk;
    wire i2s_ws;
    wire i2s_data;
    wire i2s_enabled =
        dut.i2s_slave_lite_v1_0_S00_AXI_inst.slv_reg2[0];
    integer errors = 0;
    integer i;
    integer bclk_edges_since_ws = 0;
```

```
    reg ws_monitor_armed = 1'b0;
    reg [31:0] read_value;

    i2s #(
        .C_S00_AXI_DATA_WIDTH(32),
        .C_S00_AXI_ADDR_WIDTH(4)
    ) dut (
        .audio_clk(aclk),
        .i2s_mclk(i2s_mclk), .i2s_bclk(i2s_bclk),
        .i2s_ws(i2s_ws), .i2s_data(i2s_data),
        .s00_axi_aclk(aclk), .s00_axi_aresetn(aresetn),
        .s00_axi_awaddr(awaddr), .s00_axi_awprot(awprot),
        .s00_axi_awvalid(awvalid), .s00_axi_awready(awready),
        .s00_axi_wdata(wdata), .s00_axi_wstrb(wstrb),
        .s00_axi_wvalid(wvalid), .s00_axi_wready(wready),
        .s00_axi_bresp(bresp), .s00_axi_bvalid(bvalid), .s00_axi_bready(bready),
        .s00_axi_araddr(araddr), .s00_axi_arprot(arprot),
        .s00_axi_arvalid(arvalid), .s00_axi_arready(arready),
        .s00_axi_rdata(rdata), .s00_axi_rresp(rresp),
        .s00_axi_rvalid(rvalid), .s00_axi_rready(rready)
    );

    // A stereo frame has two 32-BCLK channel slots.  The first WS
    // transition arms the monitor; each following transition must occur
    // after exactly 32 receiver sampling edges.
    always @(posedge i2s_bclk)
        if (aresetn && i2s_enabled)
            bclk_edges_since_ws = bclk_edges_since_ws + 1;

    always @(i2s_ws) begin
        if (!aresetn || !i2s_enabled) begin
            bclk_edges_since_ws = 0;
            ws_monitor_armed = 1'b0;
        end else if (ws_monitor_armed) begin
            if (bclk_edges_since_ws != 32) begin
                $display("ERROR: WS changed after %0d BCLK edges, expected 32",
                         bclk_edges_since_ws);
                errors = errors + 1;
            end
            bclk_edges_since_ws = 0;
        end else begin
            ws_monitor_armed = 1'b1;
            bclk_edges_since_ws = 0;
        end
    end

    task axi_write;
        input [3:0] address;
        input [31:0] value;
        begin
            @(negedge aclk);
            awaddr = address;
            wdata = value;
            wstrb = 4'hF;
            awvalid = 1'b1;
            wvalid = 1'b1;
            while (!(awready && wready)) @(posedge aclk);
            @(negedge aclk);
            awvalid = 1'b0;
```

```
109             wvalid = 1'b0;
110             while (!bvalid) @(posedge aclk);
111             if (bresp !== 2'b00) begin
112                 $display("ERROR: AXI write response at address 0x%0h was %b", address,
    bresp);
113                 errors = errors + 1;
114             end
115             @(posedge aclk);
116         end
117     endtask
118
119     task axi_read;
120         input [3:0] address;
121         output [31:0] value;
122         begin
123             @(negedge aclk);
124             araddr = address;
125             arvalid = 1'b1;
126             while (!arready) @(posedge aclk);
127             @(negedge aclk);
128             arvalid = 1'b0;
129             while (!rvalid) @(posedge aclk);
130             value = rdata;
131             if (rresp !== 2'b00) begin
132                 $display("ERROR: AXI read response at address 0x%0h was %b", address,
    rresp);
133                 errors = errors + 1;
134             end
135             @(posedge aclk);
136         end
137     endtask
138
139     // Check one I2S channel after its WS transition.  The first BCLK rising
140     // edge is the Philips I2S one-bit delay; payload is then MSB-first.
141     task check_channel;
142         input expected_ws;
143         input [23:0] expected_sample;
144         begin
145             if (expected_ws)
146                 @(posedge i2s_ws);
147             else
148                 @(negedge i2s_ws);
149
150             @(posedge i2s_bclk);
151             #1;
152             if (i2s_data !== 1'b0) begin
153                 $display("ERROR: I2S WS=%0b did not provide the required one-bit delay",
     expected_ws);
154                 errors = errors + 1;
155             end
156
157             for (i = 23; i >= 0; i = i - 1) begin
158                 @(posedge i2s_bclk);
159                 #1;
160                 if (i2s_ws !== expected_ws || i2s_data !== expected_sample[i]) begin
161                     $display("ERROR: I2S WS=%0b bit %0d expected %b, got WS=%b DATA=%b",
162                              expected_ws, i, expected_sample[i], i2s_ws, i2s_data);
163                     errors = errors + 1;
164                 end
```

```
            end

            // 24-bit audio occupies the remaining seven positions of the
            // fixed 32-BCLK channel slot with zero padding.
            for (i = 0; i < 7; i = i + 1) begin
                @(posedge i2s_bclk);
                #1;
                if (i2s_ws !== expected_ws || i2s_data !== 1'b0) begin
                    $display("ERROR: I2S WS=%0b padding bit %0d was not zero",
    expected_ws, i);
                    errors = errors + 1;
                end
            end
        end
    endtask

    // Time complete WS periods rather than estimating frequency from a
    // screenshot. Sixteen frames keep the 10 ns simulation quantisation
    // error well below the 0.1 percent acceptance limit.
    task check_sample_rate;
        input [19:0] target_hz;
        reg [31:0] control_value;
        real start_time_ns;
        real end_time_ns;
        real measured_hz;
        real error_percent;
        integer frame;
        begin
            control_value = {5'd0, target_hz, 7'd0} |
                            (32'd24 << 2) | 32'h1;
            axi_write(ADDR_CONTROL, control_value);

            @(posedge i2s_ws);
            start_time_ns = $realtime;
            for (frame = 0; frame < 16; frame = frame + 1)
                @(posedge i2s_ws);
            end_time_ns = $realtime;

            measured_hz = 16.0e9 / (end_time_ns - start_time_ns);
            error_percent = 100.0 * (measured_hz - target_hz) / target_hz;
            $display("SAMPLE_RATE: target=%0d Hz measured=%0.3f Hz error=%0.5f%%",
                     target_hz, measured_hz, error_percent);
            if (error_percent > 0.1 || error_percent < -0.1) begin
                $display("ERROR: sample-rate error exceeds 0.1%% for %0d Hz",
                         target_hz);
                errors = errors + 1;
            end
        end
    endtask

    initial begin
        $dumpfile("tb_i2s_axi_lite.vcd");
        $dumpvars(0, tb_i2s_axi_lite);

        repeat (5) @(posedge aclk);
        aresetn = 1'b1;
        repeat (3) @(posedge aclk);

        // Disabled IP must be quiet after reset.
```

```
223         if ({i2s_mclk, i2s_bclk, i2s_ws, i2s_data} !== 4'b0000) begin
224             $display("ERROR: I2S outputs are not low while disabled");
225             errors = errors + 1;
226         end
227 
228         axi_read(ADDR_CONTROL, read_value);
229         if (read_value !== (32'd48000 << 7)) begin
230             $display("ERROR: reset CONTROL expected 0x%08h, got 0x%08h", (32'd48000 <<
    7), read_value);
231             errors = errors + 1;
232         end
233 
234         axi_write(ADDR_DATA_LEFT, LEFT_SAMPLE);
235         axi_write(ADDR_DATA_RIGHT, RIGHT_SAMPLE);
236         axi_read(ADDR_DATA_LEFT, read_value);
237         if (read_value !== LEFT_SAMPLE) begin
238             $display("ERROR: DATA_LEFT readback mismatch: 0x%08h", read_value);
239             errors = errors + 1;
240         end
241         axi_read(ADDR_DATA_RIGHT, read_value);
242         if (read_value !== RIGHT_SAMPLE) begin
243             $display("ERROR: DATA_RIGHT readback mismatch: 0x%08h", read_value);
244             errors = errors + 1;
245         end
246 
247         axi_write(ADDR_CONTROL, CTRL_24_48K);
248         axi_read(ADDR_CONTROL, read_value);
249         if (read_value !== CTRL_24_48K) begin
250             $display("ERROR: CONTROL readback mismatch: 0x%08h", read_value);
251             errors = errors + 1;
252         end
253 
254         // Check right then left.  The first left WS transition occurs only
255         // after the initial right half-frame has completed.
256         check_channel(1'b1, RIGHT_SAMPLE[23:0]);
257         check_channel(1'b0, LEFT_SAMPLE[23:0]);
258 
259         axi_read(ADDR_STATUS, read_value);
260         if (^read_value[0] === 1'bx) begin
261             $display("ERROR: STATUS[0] is unknown after frame latching");
262             errors = errors + 1;
263         end
264 
265         // Mute preserves clocks and WS but must transmit zero data.
266         axi_write(ADDR_CONTROL, CTRL_MUTE_24_48K);
267         check_channel(1'b1, 24'd0);
268 
269         // Standard and non-standard rates are measured from elapsed WS time.
270         check_sample_rate(20'd44100);
271         check_sample_rate(20'd48000);
272         check_sample_rate(20'd88200);
273         check_sample_rate(20'd96000);
274         check_sample_rate(20'd10000);
275         check_sample_rate(20'd12345);
276 
277         axi_write(ADDR_CONTROL, 32'd48000 << 7);
278         repeat (4) @(posedge aclk);
279         if ({i2s_mclk, i2s_bclk, i2s_ws, i2s_data} !== 4'b0000) begin
280             $display("ERROR: I2S outputs did not return low after ENABLE=0");
```

```
            errors = errors + 1;
        end

        if (errors == 0)
            $display("PASS: AXI access, I2S serialization, and sample-rate checks
    verified.");
        else
            $display("FAIL: %0d testbench check(s) failed.", errors);
        $finish;
    end

    initial begin
        #(5_000_000);
        $display("FAIL: simulation timeout");
        $finish;
    end
endmodule
```

Kode 1. Testbench AXI4-Lite dan protokol I2S.

Seluruh kode sumber RTL dan firmware yang dijelaskan pada Bab IV, beserta dokumentasi proyek, berkas Vivado, dan skrip pendukung, tersedia secara terbuka pada repositori GitHub: `https://github.com/AdakHaddad/thesi2s`. Repositori tersebut mencakup implementasi IP I2S Transmitter berbasis AXI4-Lite untuk MicroBlaze V, firmware pengujian, berkas konfigurasi Vivado, skrip rekonstruksi proyek, serta dokumentasi tambahan yang digunakan selama pengembangan dan validasi sistem.

Pengelolaan repositori menggunakan *Hog (HDL on Git)*, yaitu kerangka kerja berbasis Git untuk mendukung reproduksibilitas proyek FPGA dan rekonstruksi proyek AMD Vivado.

Gambar L.32. Logo Hog (*HDL on Git*) yang digunakan untuk pengelolaan proyek HDL.